Chapter 3

Constraint Based Analysis

In this chapter we present the technique of Constraint Based Analysis using a simple functional language, FUN. We begin by presenting an abstract specification of a Control Flow Analysis and then study its theoretical properties: it is correct with respect to a Structural Operational Semantics and it can be used to analyse all programs. This specification of the analysis does not immediately lend itself to an efficient algorithm for computing a solution so we proceed by developing first a syntax directed specification and then a constraint based formulation and finally we show how the constraints can be solved. We conclude by illustrating how the precision of the analysis can be improved by combining it with Data Flow Analysis and by incorporating context information thereby linking up with the development of the previous chapter.

3.1 Abstract 0-CFA Analysis

In Chapter 2 we saw how properties of data could be propagated through a program. In developing the specification we relied on the ability to identify for each program fragment all the possible successor (and predecessor) fragments via the operator *flow* (and flow^R) and the interprocedural flow $\mathit{inter\text{-}flow}_\star$ (and $\mathit{inter\text{-}flow}_\star^R$). The usefulness of the resulting specification was due to the number of successors and predecessors being small (usually just one or two except for procedure exits). This is a typical feature of imperative programs without procedures but it usually fails for more general languages, whether imperative languages with procedures as parameters, functional languages, or object-oriented languages. In particular, the interprocedural techniques of Section 2.5 provide a solution for the simpler cases where the program text allows one to limit the number of successors, as is the case when a proce-

dure call is performed by explicitly mentioning the name of the procedure. However, these techniques are not powerful enough to handle the *dynamic dispatch problem* where variables can denote procedures. In Section 1.4 we illustrated this by the functional program

```
let f = fn x => x 1;
    g = fn y => y+2;
    h = fn z => z+3
in (f g) + (f h)
```

where the function application `x 1` in the body of `f` will transfer control to the body of the function `x`, and here it is not so obvious what program fragment this actually is, since `x` is the formal parameter of `f`. The Control Flow Analysis of the present chapter will provide a solution to the dynamic dispatch problem by determining for each subexpression a hopefully small number of functions that it may evaluate to; thereby it will determine where the flow of control may be transferred to in the case where the subexpression is the operator of a function application. In short, Control Flow Analysis will determine the *interprocedural flow* information (*inter-flow*$_\star$ or *IF*) upon which the development of Section 2.5 is based.

Syntax of the FUN language. For the main part of this chapter we shall concentrate on a small functional language: the untyped lambda calculus extended with explicit operators for recursion, conditional and local definitions. The purpose of the Control Flow Analysis will be to compute for *each* subexpression the set of functions that it could evaluate to, and to express this it is important that we are able to label *all* program fragments. We shall be very explicit about this: a program fragment with a label is called an *expression* whereas a program fragment without a label is called a *term*. So we use the following syntactic categories:

e	$\in$	**Exp**	expressions (or labelled terms)
t	$\in$	**Term**	terms (or unlabelled expressions)

We assume that a countable set of variables is given and that constants (including the truth values), binary operators (including the usual arithmetic, boolean and relational operators) and labels are left unspecified:

f, x	$\in$	**Var**	variables
c	$\in$	**Const**	constants
op	$\in$	**Op**	binary operators
ℓ	$\in$	**Lab**	labels

The *abstract syntax* of the language is now given by:

$$e ::= t^\ell$$

$$t ::= c \mid x \mid \texttt{fn}\ x\ \texttt{=>}\ e_0 \mid \texttt{fun}\ f\ x\ \texttt{=>}\ e_0 \mid e_1\ e_2$$
$$\mid \texttt{if}\ e_0\ \texttt{then}\ e_1\ \texttt{else}\ e_2 \mid \texttt{let}\ x\ \texttt{=}\ e_1\ \texttt{in}\ e_2 \mid e_1\ op\ e_2$$

Here `fn` x `=>` e_0 is a function definition (or function abstraction) whereas `fun` f x `=>` e_0 is a recursive variant of `fn` x `=>` e_0 where all free occurrences of f in e_0 refer to `fun` f x `=>` e_0 itself. The construct `let` x `=` e_1 `in` e_2 is a non-recursive local definition that is semantically equivalent to (`fn` x `=>` e_2)(e_1). As usual we shall use parentheses to disambiguate the parsing whenever needed. Also we shall *assume* throughout that in all occurrences of `fun` f x `=>` e_0, f and x are *distinct* variables.

We shall need the notion of *free variables* of expressions and terms so we define the function

$$FV : (\mathbf{Term} \cup \mathbf{Exp}) \to \mathcal{P}(\mathbf{Var})$$

in the following standard way. The abstractions `fn` x `=>` e_0 and `fun` f x `=>` e_0 contain binding occurrences of variables so $FV($`fn` x `=>` $e_0) = FV(e_0) \setminus \{x\}$ and similarly $FV($`fun` f x `=>` $e_0) = FV(e_0) \setminus \{f, x\}$. Similarly, `let` x `=` e_1 `in` e_2 contains a binding occurrence of x so we have $FV($`let` x `=` e_1 `in` $e_2) = FV(e_1) \cup (FV(e_2) \setminus \{x\})$ reflecting that free occurrences of x in e_1 are bound outside the construct. The remaining clauses for FV are straightforward.

Example 3.1 The functional program (`fn x => x`) (`fn y => y`) considered in Section 1.4 is now written as:

$$((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$$

Compared with the notation of Example 1.2 we have omitted the square brackets. ∎

Example 3.2 Consider the following expression, loop, of FUN:

$$\begin{array}{l}(\texttt{let g = (fun f x => (f}^1\ (\texttt{fn y => y}^2)^3)^4)^5\\ \texttt{in (g}^6\ (\texttt{fn z => z}^7)^8)^9)^{10}\end{array}$$

It defines a function g that is applied to the identity function `fn z => z`^{7}. The function g is defined recursively: f is its local name and x is the formal parameter. Hence the function will ignore its actual parameter and call itself recursively with the argument `fn y => y`^{2}. This will happen again and again so the program loops. ∎

3.1.1 The Analysis

Abstract domains. We shall now show how to specify 0-CFA analyses. These may be regarded as the simplest possible form of Control Flow Analysis in that *no* context information is taken into account. (As will become clear in Section 3.6, this is what the number 0 is indicating.)

The result of a 0-CFA analysis is a pair $(\widehat{C}, \widehat{\rho})$ where:

- $\widehat{\mathsf{C}}$ is the *abstract cache* associating abstract values with each labelled program point.
- $\widehat{\rho}$ is the *abstract environment* associating abstract values with each variable.

This is made precise by:

$$\begin{array}{lllll} \widehat{v} & \in & \widehat{\mathbf{Val}} & = \mathcal{P}(\mathbf{Term}) & \text{abstract values} \\ \widehat{\rho} & \in & \widehat{\mathbf{Env}} & = \mathbf{Var} \to \widehat{\mathbf{Val}} & \text{abstract environments} \\ \widehat{\mathsf{C}} & \in & \widehat{\mathbf{Cache}} & = \mathbf{Lab} \to \widehat{\mathbf{Val}} & \text{abstract caches} \end{array}$$

Here an *abstract value* $\widehat{v}$ is an abstraction of a set of functions: it is a set of terms of the form `fn` x `=>` e_0 or `fun` f x `=>` e_0. We will not be recording any constants in the abstract values because the analysis we shall specify is a pure Control Flow Analysis with no Data Flow Analysis component; in Section 3.5 we shall show how to extend it with Data Flow Analysis components. Furthermore, we do *not* need to assume that all bound variables are distinct and that all labels are distinct, but clearly, greater precision is achieved if this is the case; this means that semantically equivalent programs can have different analysis results and this is a common feature of all approaches to program analysis. As we shall see an *abstract environment* is an abstraction of a set of environments occurring in closures at run-time (see the semantics in Subsection 3.2.1). In a similar way an *abstract cache* might be considered as an abstraction of a set of execution profiles: as discussed below some texts prefer to combine the abstract environment with the abstract cache.

Example 3.3 Consider the expression $((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$ of Example 3.1. The following table contains three guesses of a 0-CFA analysis:

	$(\widehat{\mathsf{C}}_e, \widehat{\rho}_e)$	$(\widehat{\mathsf{C}}'_e, \widehat{\rho}'_e)$	$(\widehat{\mathsf{C}}''_e, \widehat{\rho}''_e)$
1	{`fn y => y`^{3}}	{`fn y => y`^{3}}	{`fn x => x`^{1}, `fn y => y`^{3}}
2	{`fn x => x`^{1}}	{`fn x => x`^{1}}	{`fn x => x`^{1}, `fn y => y`^{3}}
3	$\emptyset$	$\emptyset$	{`fn x => x`^{1}, `fn y => y`^{3}}
4	{`fn y => y`^{3}}	{`fn y => y`^{3}}	{`fn x => x`^{1}, `fn y => y`^{3}}
5	{`fn y => y`^{3}}	{`fn y => y`^{3}}	{`fn x => x`^{1}, `fn y => y`^{3}}
x	{`fn y => y`^{3}}	$\emptyset$	{`fn x => x`^{1}, `fn y => y`^{3}}
y	$\emptyset$	$\emptyset$	{`fn x => x`^{1}, `fn y => y`^{3}}

Intuitively, the guess $(\widehat{\mathsf{C}}_e, \widehat{\rho}_e)$ of the first column is acceptable whereas the guess $(\widehat{\mathsf{C}}'_e, \widehat{\rho}'_e)$ of the second column is wrong: it would seem that `fn x =>` $\texttt{x}^1$ is never called since $\widehat{\rho}'_e(\texttt{x}) = \emptyset$ indicates that `x` will never be bound to any closures. Also the guess $(\widehat{\mathsf{C}}''_e, \widehat{\rho}''_e)$ of the third column would seem to be acceptable although clearly more imprecise than $(\widehat{\mathsf{C}}_e, \widehat{\rho}_e)$. ∎

Example 3.4 Let us consider the expression, loop, of Example 3.2 and introduce the following abbreviations for abstract values:

$$\begin{array}{rcl} \mathsf{f} & = & \texttt{fun f x => (f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4 \\ \mathsf{id}_y & = & \texttt{fn y => y}^2 \\ \mathsf{id}_z & = & \texttt{fn z => z}^7 \end{array}$$

One guess of a 0-CFA analysis for this program is $(\widehat{\mathsf{C}}_{\mathsf{lp}}, \widehat{\rho}_{\mathsf{lp}})$ defined by:

$$\begin{array}{lllllllll} \widehat{\mathsf{C}}_{\mathsf{lp}}(1) & = & \{\mathsf{f}\} & \widehat{\mathsf{C}}_{\mathsf{lp}}(6) & = & \{\mathsf{f}\} & \widehat{\rho}_{\mathsf{lp}}(\mathtt{f}) & = & \{\mathsf{f}\} \\ \widehat{\mathsf{C}}_{\mathsf{lp}}(2) & = & \emptyset & \widehat{\mathsf{C}}_{\mathsf{lp}}(7) & = & \emptyset & \widehat{\rho}_{\mathsf{lp}}(\mathtt{g}) & = & \{\mathsf{f}\} \\ \widehat{\mathsf{C}}_{\mathsf{lp}}(3) & = & \{\mathsf{id}_y\} & \widehat{\mathsf{C}}_{\mathsf{lp}}(8) & = & \{\mathsf{id}_z\} & \widehat{\rho}_{\mathsf{lp}}(\mathtt{x}) & = & \{\mathsf{id}_y, \mathsf{id}_z\} \\ \widehat{\mathsf{C}}_{\mathsf{lp}}(4) & = & \emptyset & \widehat{\mathsf{C}}_{\mathsf{lp}}(9) & = & \emptyset & \widehat{\rho}_{\mathsf{lp}}(\mathtt{y}) & = & \emptyset \\ \widehat{\mathsf{C}}_{\mathsf{lp}}(5) & = & \{\mathsf{f}\} & \widehat{\mathsf{C}}_{\mathsf{lp}}(10) & = & \emptyset & \widehat{\rho}_{\mathsf{lp}}(\mathtt{z}) & = & \emptyset \end{array}$$

Intuitively, this is an acceptable guess. The choice of $\widehat{\rho}_{\mathsf{lp}}(\mathtt{g}) = \{\mathsf{f}\}$ reflects that g will evaluate to a closure constructed from that abstraction. The choice of $\widehat{\rho}_{\mathsf{lp}}(\mathtt{x}) = \{\mathsf{id}_y, \mathsf{id}_z\}$ reflects that x will be bound to closures constructed from both abstractions in the course of the evaluation. The choice of $\widehat{\mathsf{C}}_{\mathsf{lp}}(10) = \emptyset$ reflects that the evaluation of the expression will never terminate. ■

We have already said that Control Flow Analysis computes the interprocedural flow information used in Section 2.5. It is also instructive to point out the similarity between Control Flow Analysis and *Use-Definition chains* (*ud*-chains) for imperative languages (see Subsection 2.1.5): in both cases we attempt to trace how definition points reach points of use. In the case of Control Flow Analysis the *definition points* are the points where the function abstractions are created, and the *use points* are the points where functions are applied; in the case of Use-Definition chains the *definition points* are the points where variables are assigned a value, and the *use points* are the points where values of variables are accessed.

Remark. Clearly an abstract cache $\widehat{\mathsf{C}} : \mathbf{Lab} \to \widehat{\mathbf{Val}}$ and an abstract environment $\widehat{\rho} : \mathbf{Var} \to \widehat{\mathbf{Val}}$ can be combined into an entity of type $(\mathbf{Var} \cup \mathbf{Lab}) \to \widehat{\mathbf{Val}}$. Some texts dispense with the labels altogether, simply using an abstract environment and no abstract cache, by ensuring that *all* subterms are properly "labelled" by variables. This type of expression frequently occurs in the internals of compilers in the form of "continuation passing style", "A-normal form" or "three address code". We have abstained from doing so in order to illustrate that the techniques not only work for compiler intermediate forms but also for general programming languages and calculi for computation; this flexibility is useful when dealing with non-standard applications (as discussed in the Concluding Remarks). ■

$$[con] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models c^{\ell} \text{ always}$$

$$[var] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models x^{\ell} \text{ iff } \widehat{\rho}(x) \subseteq \widehat{\mathsf{C}}(\ell)$$

$$[fn] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models (\texttt{fn } x \texttt{ => } e_0)^{\ell} \text{ iff } \{\texttt{fn } x \texttt{ => } e_0\} \subseteq \widehat{\mathsf{C}}(\ell)$$

$$[fun] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models (\texttt{fun } f\ x \texttt{ => } e_0)^{\ell} \text{ iff } \{\texttt{fun } f\ x \texttt{ => } e_0\} \subseteq \widehat{\mathsf{C}}(\ell)$$

$$\begin{aligned}[app] \quad (\widehat{\mathsf{C}},\widehat{\rho}) &\models (t_1^{\ell_1}\ t_2^{\ell_2})^{\ell} \\ &\text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models t_2^{\ell_2} \wedge \\ &\quad (\forall(\texttt{fn } x \texttt{ => } t_0^{\ell_0}) \in \widehat{\mathsf{C}}(\ell_1) : \\ &\qquad (\widehat{\mathsf{C}},\widehat{\rho}) \models t_0^{\ell_0} \wedge \\ &\qquad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell)) \wedge \\ &\quad (\forall(\texttt{fun } f\ x \texttt{ => } t_0^{\ell_0}) \in \widehat{\mathsf{C}}(\ell_1) : \\ &\qquad (\widehat{\mathsf{C}},\widehat{\rho}) \models t_0^{\ell_0} \wedge \\ &\qquad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell) \wedge \\ &\qquad \{\texttt{fun } f\ x \texttt{ => } t_0^{\ell_0}\} \subseteq \widehat{\rho}(f)) \end{aligned}$$

$$\begin{aligned}[if] \quad (\widehat{\mathsf{C}},\widehat{\rho}) &\models (\texttt{if } t_0^{\ell_0} \texttt{ then } t_1^{\ell_1} \texttt{ else } t_2^{\ell_2})^{\ell} \\ &\text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models t_0^{\ell_0} \wedge \\ &\quad (\widehat{\mathsf{C}},\widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models t_2^{\ell_2} \wedge \\ &\quad \widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\mathsf{C}}(\ell) \wedge \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell) \end{aligned}$$

$$\begin{aligned}[let] \quad (\widehat{\mathsf{C}},\widehat{\rho}) &\models (\texttt{let } x \texttt{ = } t_1^{\ell_1} \texttt{ in } t_2^{\ell_2})^{\ell} \\ &\text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models t_2^{\ell_2} \wedge \\ &\quad \widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\rho}(x) \wedge \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell) \end{aligned}$$

$$[op] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models (t_1^{\ell_1}\ op\ t_2^{\ell_2})^{\ell} \text{ iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models t_2^{\ell_2}$$

Table 3.1: Abstract Control Flow Analysis (Subsections 3.1.1 and 3.1.2).

Acceptability relation. It remains to determine whether or not a proposed guess $(\widehat{\mathsf{C}},\widehat{\rho})$ of an analysis results is in fact an *acceptable 0-CFA* analysis for the program considered. We shall give an *abstract specification* of what this means; having studied its theoretical properties (in Section 3.2) we then consider how to compute the desired analysis (in Sections 3.3 and 3.4).

It is instructive to point out that the abstract specification corresponds to an *implicit* formulation of the data flow equations of Chapter 2; it will be used to determine whether or not a a guess is indeed an acceptable solution to the analysis problem. The syntax directed and constraint based formulations (of Sections 3.3 and 3.4) correspond to *explicit* formulations of the data flow equations from which an iterative algorithm in the spirit of Chaotic Iteration (Section 1.7) can be used to compute an analysis result.

For the formulation of the *abstract 0-CFA* analysis we shall write

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models e$$

for when $(\widehat{\mathsf{C}}, \widehat{\rho})$ is an acceptable Control Flow Analysis of the expression e. Thus the relation "$\models$" has functionality

$$\models : (\widehat{\mathbf{Cache}} \times \widehat{\mathbf{Env}} \times \mathbf{Exp}) \to \{true, false\}$$

and its defining clauses are given in Table 3.1 (writing "always" for "iff *true*"); the clauses are explained below but the relation $\models$ will not be formally defined until Subsection 3.1.2.

The clause [*con*] places no demands on $\widehat{\mathsf{C}}(\ell)$ because we are not tracking any data values in the pure 0-CFA analysis considered here and because we assume that there are no functions among the constants; the clause can be reformulated as

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models c^{\ell} \text{ iff } \emptyset \subseteq \widehat{\mathsf{C}}(\ell)$$

thereby highlighting this point.

The clause [*var*] is responsible for linking the abstract environment into the abstract cache: so in order for $(\widehat{\mathsf{C}}, \widehat{\rho})$ to be an acceptable analysis, everything the variable x can evaluate to has to be included in what may be observed at the program point ℓ: $\widehat{\rho}(x) \subseteq \widehat{\mathsf{C}}(\ell)$.

The clauses [*fn*] and [*fun*] simply demand that in order for $(\widehat{\mathsf{C}}, \widehat{\rho})$ to be an acceptable analysis, the functional term (`fn` x `=>` e_0 or `fun` f x `=>` e_0) must be included in $\widehat{\mathsf{C}}(\ell)$; this says that the term is part of a closure that can arise at program point ℓ during evaluation. Note that these clauses do not demand that $(\widehat{\mathsf{C}}, \widehat{\rho})$ is an acceptable analysis result for the bodies of the functions; the clause for function application will take care of that.

Before turning to the more complicated clause [*app*] let us consider the clauses [*if*] and [*let*]. They contain "recursive calls" demanding that subexpressions must be analysed in consistent ways using $(\widehat{\mathsf{C}}, \widehat{\rho})$; additionally, the clauses explicitly link the values produced by subexpressions to the value of the overall expression, and in the case of [*let*] also the abstract cache is linked into the abstract environment. The interplay between the clauses [*var*] and [*let*] is illustrated in Figure 3.1; as in Chapter 2 an arrow indicates a flow of information. The clause [*op*] follows the same overall pattern.

Clause [*app*] also contains "recursive calls" demanding that the operator $t_1^{\ell_1}$ and the operand $t_2^{\ell_2}$ can be analysed using $(\widehat{\mathsf{C}}, \widehat{\rho})$. For each term `fn` x `=>` $t_0^{\ell_0}$ that may reach the operator position (ℓ_1), i.e. where

$$(\texttt{fn}\ x \texttt{ => } t_0^{\ell_0}) \in \widehat{\mathsf{C}}(\ell_1)$$

it further demands that the actual parameter (labelled ℓ_2) is linked to the formal parameter (x)

$$\widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x)$$

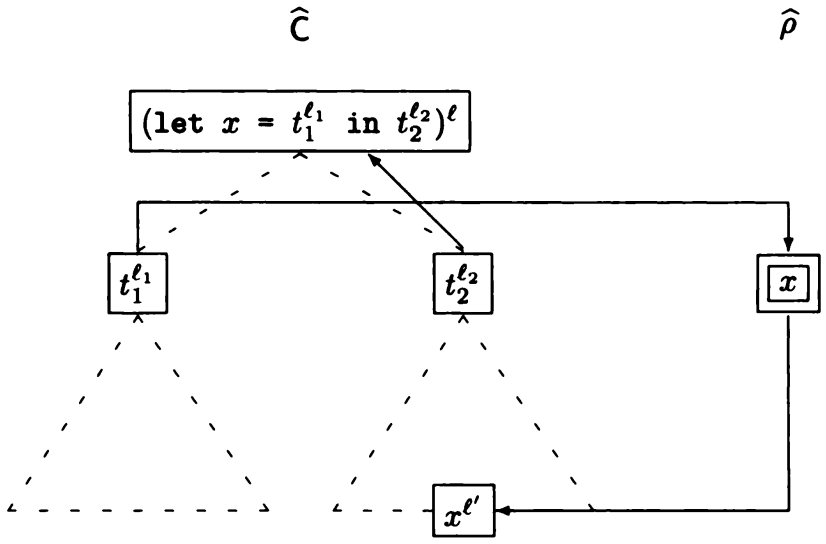

Figure 3.1: Pictorial illustration of the clauses [*let*] and [*var*].

and that the result of the function evaluation (labelled ℓ_0) is linked to the result of the application itself (labelled ℓ)

$$\widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell)$$

and finally, that the function body itself can be analysed using $(\widehat{\mathsf{C}}, \widehat{\rho})$:

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models t_0^{\ell_0}$$

This is illustrated in Figure 3.2. For terms `fun` $f\ x$ `=>` $t_0^{\ell_0}$ the demands are much the same except that the term itself additionally needs to be included in $\widehat{\rho}(f)$ in order to reflect the recursive nature of `fun` $f\ x$ `=>` $t_0^{\ell_0}$.

Example 3.5 Consider Example 3.3 and the guesses of a 0-CFA analysis for the expression. First we show that $(\widehat{\mathsf{C}}_e, \widehat{\rho}_e)$ is an acceptable guess:

$$(\widehat{\mathsf{C}}_e, \widehat{\rho}_e) \models ((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$$

Using clause [*app*] and $\widehat{\mathsf{C}}_e(2) = \{\texttt{fn x => x}^1\}$ we must check:

$$\begin{aligned}
&(\widehat{\mathsf{C}}_e, \widehat{\rho}_e) \models (\texttt{fn x => x}^1)^2\\
&(\widehat{\mathsf{C}}_e, \widehat{\rho}_e) \models (\texttt{fn y => y}^3)^4\\
&(\widehat{\mathsf{C}}_e, \widehat{\rho}_e) \models \texttt{x}^1\\
&\widehat{\mathsf{C}}_e(4) \subseteq \widehat{\rho}_e(\texttt{x})\\
&\widehat{\mathsf{C}}_e(1) \subseteq \widehat{\mathsf{C}}_e(5)
\end{aligned}$$

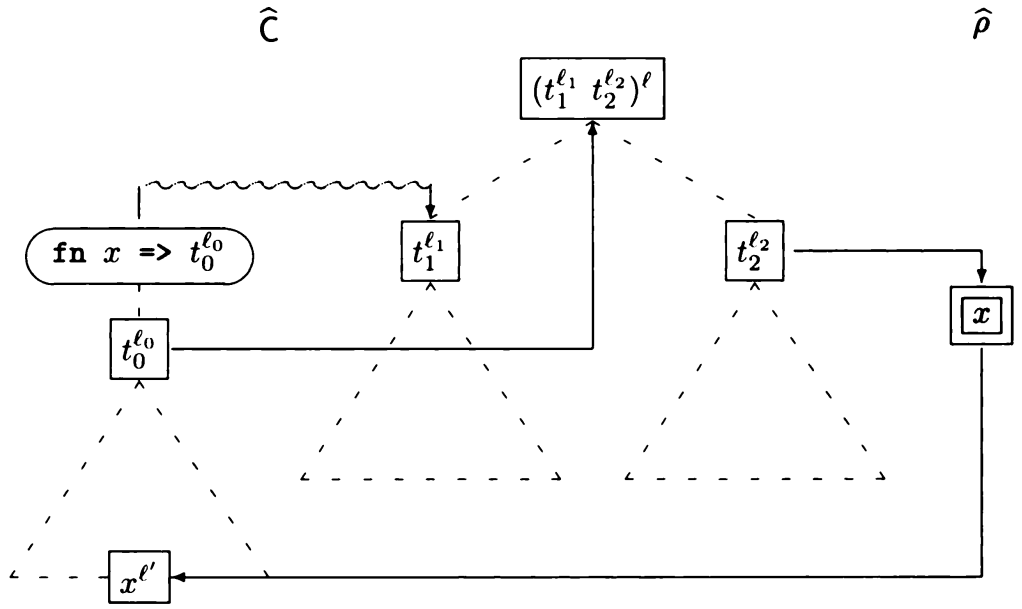

Figure 3.2: Pictorial illustration of the clauses [*app*], [*fn*] and [*var*].

All of these are easily checked using the clauses [*fn*] and [*var*].

Next we show that $(\widehat{\mathsf{C}}'_{\mathsf{e}}, \widehat{\rho}'_{\mathsf{e}})$ is *not* an acceptable guess:

$$(\widehat{\mathsf{C}}'_{\mathsf{e}}, \widehat{\rho}'_{\mathsf{e}}) \not\models ((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$$

We do so by proceeding as above and observing that $\widehat{\mathsf{C}}'_{\mathsf{e}}(4) \not\subseteq \widehat{\rho}'_{\mathsf{e}}(\mathbf{x})$. ∎

Note that the clauses contain a number of inclusions of the form

$$\mathit{lhs} \subseteq \mathit{rhs}$$

where *rhs* is of the form $\widehat{\mathsf{C}}(\ell)$ or $\widehat{\rho}(x)$ and where *lhs* is of the form $\widehat{\mathsf{C}}(\ell)$, $\widehat{\rho}(x)$, or $\{t\}$. These inclusions express how the higher-order entities may flow through the expression.

It is important to observe that the clauses [*fn*] and [*fun*] do *not* contain "recursive calls" demanding that subexpressions must be analysed. Instead one relies on the clause [*app*] demanding this for all "subexpressions" that may eventually be applied. This is a phenomenon common in program analysis, where one does *not* want to analyse *unreachable* program fragments: occasionally results obtained from these parts of the program can suppress transformations in the reachable part of the program. It also allows us to deal with *open systems* where functions may be supplied by the environment; this is particularly important for languages involving concurrency. However,

note that this perspective is different from that of type inference, where even unreachable fragments must be correctly typed.

In the terminology of Section 2.5 the analysis is *flow-insensitive* because FUN contains no side effects and because we analyse the operand to a function call even when the operator cannot evaluate to any function; see Exercise 3.3 for how to improve on this. Also the analysis is *context-insensitive* because it treats all function calls in the same way; we refer to Section 3.6 for how to improve on this.

3.1.2 Well-definedness of the Analysis

Finally, we need to clarify that the clauses of Table 3.1 do indeed define a relation. The difficulty here is that the clause [*app*] is *not* in a form that allows us to define $(\widehat{\mathsf{C}},\widehat{\rho}) \models e$ by structural induction in the expression e – it requires checking the acceptability of $(\widehat{\mathsf{C}},\widehat{\rho})$ for an expression $t_0^{\ell_0}$ that is not a subexpression of the application $(t_1^{\ell_1}\ t_2^{\ell_2})^{\ell}$. This leads to defining the relation "$\models$" of Table 3.1 by *coinduction*, that is as the *greatest fixed point* of a certain functional. An alternative will be to define the analysis as the least fixed point of the functional but, as we shall see in Example 3.6 and more formally in Proposition 3.16, this is not always appropriate.

The formal definition of $\models$. Following the approach of Appendix B we shall view Table 3.1 as defining a function:

$$\begin{aligned} \mathcal{Q} : ((\widehat{\mathbf{Cache}} \times \widehat{\mathbf{Env}} \times \mathbf{Exp}) &\to \{\mathit{true}, \mathit{false}\}) \\ &\to ((\widehat{\mathbf{Cache}} \times \widehat{\mathbf{Env}} \times \mathbf{Exp}) \to \{\mathit{true}, \mathit{false}\}) \end{aligned}$$

As an example we have:

$$\begin{aligned} &\mathcal{Q}(Q)(\widehat{\mathsf{C}},\widehat{\rho},(\mathtt{let}\ x = t_1^{\ell_1}\ \mathtt{in}\ t_2^{\ell_2})^{\ell}) \\ &\quad = Q(\widehat{\mathsf{C}},\widehat{\rho},t_1^{\ell_1}) \ \wedge\ Q(\widehat{\mathsf{C}},\widehat{\rho},t_2^{\ell_2}) \ \wedge\ \widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\rho}(x) \ \wedge\ \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell) \end{aligned}$$

We frequently refer to $\mathcal{Q}$ as a *functional* because its argument and result are themselves functions.

Now by inspecting Table 3.1 it is easy to verify that the functional $\mathcal{Q}$ constructed this way is a monotone function on the complete lattice

$$((\widehat{\mathbf{Cache}} \times \widehat{\mathbf{Env}} \times \mathbf{Exp}) \to \{\mathit{true}, \mathit{false}\}, \sqsubseteq)$$

where the ordering $\sqsubseteq$ is defined by:

$$Q_1 \sqsubseteq Q_2 \ \text{ iff } \ \forall(\widehat{\mathsf{C}},\widehat{\rho},e) : (Q_1(\widehat{\mathsf{C}},\widehat{\rho},e) = \mathit{true}) \ \Rightarrow\ (Q_2(\widehat{\mathsf{C}},\widehat{\rho},e) = \mathit{true})$$

Hence $\mathcal{Q}$ has fixed points and we shall define "$\models$" coinductively:

$$\models \text{ is the } \textit{greatest} \text{ fixed point of } \mathcal{Q}$$

The following example intuitively motivates the use of a coinductive (i.e. a greatest fixed point) definition as opposed to an inductive (i.e. a least fixed point) definition; a more formal explanation will be given in Subsection 3.2.4.

Example 3.6 Consider the expression loop of Example 3.4

$$\begin{array}{l}\texttt{(let g = (fun f x => (f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4\texttt{)}^5\\ \texttt{in (g}^6\ \texttt{(fn z => z}^7\texttt{)}^8\texttt{)}^9\texttt{)}^{10}\end{array}$$

and the suggested analysis result $(\widehat{C}_{lp}, \widehat{\rho}_{lp})$. To show $(\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models$ loop it is, according to the clause [*let*], sufficient to verify that

$$\begin{array}{l}(\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{(fun f x => (f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4\texttt{)}^5\\ (\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{(g}^6\ \texttt{(fn z => z}^7\texttt{)}^8\texttt{)}^9\end{array}$$

because $\widehat{C}_{lp}(5) \subseteq \widehat{\rho}_{lp}(\texttt{g})$ and $\widehat{C}_{lp}(9) \subseteq \widehat{C}_{lp}(10)$. The first clause follows from [*fun*] and for the second clause we use that $\widehat{C}_{lp}(6) = \{\texttt{f}\}$ so it is, according to [*app*], sufficient to verify that

$$\begin{array}{l}(\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{g}^6\\ (\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{(fn z => z}^7\texttt{)}^8\\ (\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{(f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4\end{array}$$

because $\widehat{C}_{lp}(8) \subseteq \widehat{\rho}_{lp}(\texttt{x})$, $\widehat{C}_{lp}(4) \subseteq \widehat{C}_{lp}(9)$ and $\texttt{f} \in \widehat{\rho}_{lp}(\texttt{f})$. The first two clauses now follow from [*var*] and [*fn*]. For the third clause we proceed as above and since $\widehat{C}_{lp}(1) = \{\texttt{f}\}$ it is sufficient to verify

$$\begin{array}{l}(\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{f}^1\\ (\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{(fn y => y}^2\texttt{)}^3\\ (\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{(f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4\end{array}$$

because $\widehat{C}_{lp}(3) \subseteq \widehat{\rho}_{lp}(\texttt{x})$, $\widehat{C}_{lp}(4) \subseteq \widehat{C}_{lp}(4)$ and $\texttt{f} \in \widehat{\rho}_{lp}(\texttt{f})$.

Again the first two clauses are straightforward but in the last clause we encounter a *circularity*: to verify $(\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{(f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4$ we have to verify $(\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{(f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4$!

To solve this we use coinduction: basically this amounts to assuming that $(\widehat{C}_{lp}, \widehat{\rho}_{lp}) \models \texttt{(f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4$ holds at the "inner level" and proving that it also holds at the "outer level". This will give us the required proof. ∎

Analogy. The use of coinduction and induction, or greatest and least fixed points, may be confusing at first. We therefore offer the following analogy. Imagine a company that is about to buy a large number of computers. To comply with national and international rules for commerce the deal must be offered in such a way that all vendors have a chance to make a bid. To this effect the company first makes a specification of the requirements to the computers (like the ability to execute certain benchmarks efficiently); however, the specification is necessarily loose meaning that there might be many ways of fulfilling the requirements. Among the bids made the company then chooses the bid believed to be best (in the sense of offering the required functionality as cheaply as possible or offering as much functionality as possible). Then suppose that when the actual computers are delivered the company suspects that they are not fulfilling the promises. It then starts arbitration in order to persuade the vendor that they have not fulfilled their obligations (and must take back the computers, or upgrade them or sell them at reduced price). In order for the company to be successful it must be able to use the details of the specification, and not any other information about what other vendors would have been able to provide, in order to prove that the computers delivered fail to fulfil one or more demands explicitly put forward in the specification. If the specification is too loose, like just demanding an Intel-based PC, there is no way that the inability to run Linux (or some other operating system) can be used against the vendor. — Coming back to the analysis in Table 3.1 its primary purpose is to provide a specification of when a potential analysis result is acceptable. The specification takes the form of defining a relation

$$\models : (\widehat{\mathbf{Cache}} \times \widehat{\mathbf{Env}} \times \mathbf{Exp}) \to \{\mathit{true}, \mathit{false}\}$$

and since the specification is intended to be loose it must be defined coinductively: something only fails to live up to the specification if it can be demonstrated to follow from the clauses; this line of reasoning is in fact a fundamental ingredient in algebraic specification theory. As an example, suppose we were silly and merely defined

$$(C, \rho) \models P \quad \text{iff} \quad (C, \rho) \models P$$

then any solution (C, ρ) would in fact live up to the specification; it is a simple mathematical fact that only the coinductive interpretation of the definition of $\models$ will live up to this demand (and in particular the inductive definition will not as it will not accept any solutions at all). Having clarified the meaning of the specification we can then look for the best solution (C, ρ): this is typically the least solution that satisfies the specification. So to summarise, the actual specification itself is defined by a greatest fixed point (namely coinductively) whereas algorithms for computing the intended solution are normally defined by least fixed points.

3.2 Theoretical Properties

In this section we shall investigate some more theoretical properties of the Control Flow Analysis, namely:

- semantic correctness, and
- the existence of least solutions.

The semantic correctness result is important since it ensures that the information from the analysis is indeed a safe description of what will happen during the evaluation of the program. The result about the existence of least solutions ensures that all programs can be analysed and furthermore that there is a "best" or "most precise" analysis result.

As in Section 2.2, the material of this section may be skimmed through on a first reading; however, we reiterate that it is frequently when conducting the correctness proof that the final and subtle errors in the analysis are found and corrected!

3.2.1 Structural Operational Semantics

Configurations. We shall equip the language FUN with a *Structural Operational Semantics*. We shall choose an approach based on *explicit* environments rather than substitutions because (as discussed in the Concluding Remarks) a substitution based semantics does not preserve the identity of functions (and hence abstract values) during evaluation. So a function definition will evaluate to a *closure* containing the syntax of the function definition together with an *environment* mapping its free variables to their *values*. For this we introduce the following categories

$$\begin{array}{lcll} v & \in & \mathbf{Val} & \text{values} \\ \rho & \in & \mathbf{Env} & \text{environments} \end{array}$$

defined by:

$$\begin{array}{lcl} v & ::= & c \mid \texttt{close}\ t\ \texttt{in}\ \rho \\ \rho & ::= & [\,] \mid \rho[x \mapsto v] \end{array}$$

A function abstraction `fn` x `=>` e_0 will then evaluate to a closure, written `close` (`fn` x `=>` e_0) `in` ρ; similarly, the abstraction `fun` f x `=>` e_0 will evaluate to `close` (`fun` f x `=>` e_0) `in` ρ. Our definitions do not demand that all terms t occurring in some `close` t `in` ρ in the semantics will be of the form `fn` x `=>` e_0 or `fun` f x `=>` e_0; however, it will be the case for the semantics presented below.

As in Section 2.5 we shall need intermediate configurations to handle the binding of local variables. We therefore introduce syntactic categories for *intermediate expressions* and *intermediate terms*

$$\begin{array}{lcll} ie & \in & \mathbf{IExp} & \text{intermediate expressions} \\ it & \in & \mathbf{ITerm} & \text{intermediate terms} \end{array}$$

that extend the syntax of expressions and terms as follows:

$$\begin{array}{lcl} ie & ::= & it^{\ell} \\ it & ::= & c \mid x \mid \texttt{fn}\ x\ \texttt{=>}\ e_0 \mid \texttt{fun}\ f\ x\ \texttt{=>}\ e_0 \mid ie_1\ ie_2 \\ & \mid & \texttt{if}\ ie_0\ \texttt{then}\ e_1\ \texttt{else}\ e_2 \mid \texttt{let}\ x\ \texttt{=}\ ie_1\ \texttt{in}\ e_2 \mid ie_1\ op\ ie_2 \\ & \mid & \texttt{bind}\ \rho\ \texttt{in}\ ie \mid \texttt{close}\ t\ \texttt{in}\ \rho \end{array}$$

The role of the `bind`-construct is much as in Section 2.5: `bind` ρ `in` ie records that the intermediate expression ie has to be evaluated in an environment with the bindings ρ. (The sequence of environments of nested `bind`-constructs may be viewed as an encoding of the frames of a run-time stack.) So while `close` t `in` ρ is a fully evaluated value this is not the case for `bind` ρ `in` ie. We shall need these intermediate terms because we define a small step semantics; only the `close`-constructs will be needed for a big step variant of the semantics.

Alternatively, the definitions of **Val** and **Env** could have been written $\mathbf{Val} = \mathbf{Const} + (\mathbf{Term} \times \mathbf{Env})$ and $\mathbf{Env} = \mathbf{Var} \rightarrow_{\text{fin}} \mathbf{Val}$ (for a finite mapping) but it is important to stress that all entities are defined mutually recursively in the manner of context-free grammars. Formally, we defined an environment ρ as a list but nevertheless we shall feel free to regard it as a finite mapping: we write $dom(\rho)$ for $\{x \mid \rho \text{ contains } [x \mapsto \cdots]\}$; we write $\rho(x) = v$ if $x \in dom(\rho)$ and the rightmost occurrence of $[x \mapsto \cdots]$ in ρ is $[x \mapsto v]$, and we write $\rho \mid X$ for the environment obtained from ρ by removing all occurrences of $[x \mapsto \cdots]$ with $x \notin X$. For the sake of readability we shall write $[x \mapsto v]$ for $[\,][x \mapsto v]$.

We have been very deliberate in when to use intermediate expressions and when to use expressions although it is evident that all expressions are also intermediate expressions. Since we do not evaluate the body of a function before it is applied we continue to let the body be an expression rather than an intermediate expression. Similar remarks apply to the branches of the conditional and the body of the local definitions. Note that although an environment only records the *terms* `fn` x `=>` e_0 and `fun` f x `=>` e_0 in the closures bound into it, we do not lose the identity of the function abstractions as e_0 will be of the form $t_0^{\ell_0}$ and hence ℓ_0 may be used as the *"unique" identification* of the function abstraction.

Transitions. We are now ready to define the transition rules of the Structural Operational Semantics by means of judgements of the form

$$\rho \vdash ie_1 \rightarrow ie_2$$

$$[var] \quad \rho \vdash x^\ell \rightarrow v^\ell \quad \text{if } x \in dom(\rho) \text{ and } v = \rho(x)$$

$$[fn] \quad \rho \vdash (\texttt{fn}\ x \Rightarrow e_0)^\ell \rightarrow (\texttt{close}\ (\texttt{fn}\ x \Rightarrow e_0)\ \texttt{in}\ \rho_0)^\ell \quad \text{where } \rho_0 = \rho \mid FV(\texttt{fn}\ x \Rightarrow e_0)$$

$$[fun] \quad \rho \vdash (\texttt{fun}\ f\ x \Rightarrow e_0)^\ell \rightarrow (\texttt{close}\ (\texttt{fun}\ f\ x \Rightarrow e_0)\ \texttt{in}\ \rho_0)^\ell \quad \text{where } \rho_0 = \rho \mid FV(\texttt{fun}\ f\ x \Rightarrow e_0)$$

$$[app_1] \quad \frac{\rho \vdash ie_1 \rightarrow ie_1'}{\rho \vdash (ie_1\ ie_2)^\ell \rightarrow (ie_1'\ ie_2)^\ell}$$

$$[app_2] \quad \frac{\rho \vdash ie_2 \rightarrow ie_2'}{\rho \vdash (v_1^{\ell_1}\ ie_2)^\ell \rightarrow (v_1^{\ell_1}\ ie_2')^\ell}$$

$$[app_{fn}] \quad \rho \vdash ((\texttt{close}\ (\texttt{fn}\ x \Rightarrow e_1)\ \texttt{in}\ \rho_1)^{\ell_1}\ v_2^{\ell_2})^\ell \rightarrow (\texttt{bind}\ \rho_1[x \mapsto v_2]\ \texttt{in}\ e_1)^\ell$$

$$[app_{fun}] \quad \rho \vdash ((\texttt{close}\ (\texttt{fun}\ f\ x \Rightarrow e_1)\ \texttt{in}\ \rho_1)^{\ell_1}\ v_2^{\ell_2})^\ell \rightarrow (\texttt{bind}\ \rho_2[x \mapsto v_2]\ \texttt{in}\ e_1)^\ell \quad \text{where } \rho_2 = \rho_1[f \mapsto \texttt{close}\ (\texttt{fun}\ f\ x \Rightarrow e_1)\ \texttt{in}\ \rho_1]$$

$$[bind_1] \quad \frac{\rho_1 \vdash ie_1 \rightarrow ie_1'}{\rho \vdash (\texttt{bind}\ \rho_1\ \texttt{in}\ ie_1)^\ell \rightarrow (\texttt{bind}\ \rho_1\ \texttt{in}\ ie_1')^\ell}$$

$$[bind_2] \quad \rho \vdash (\texttt{bind}\ \rho_1\ \texttt{in}\ v_1^{\ell_1})^\ell \rightarrow v_1^\ell$$

Table 3.2: The Structural Operational Semantics of FUN (part 1).

given by the axioms and inference rules of Tables 3.2 and 3.3; they are explained below. The idea is that *one step* of computation of the expression ie_1 in the environment ρ will transform it into ie_2.

The value of a variable is obtained from the environment as expressed by the axiom [*var*]. The axioms [*fn*] and [*fun*] construct the appropriate closures; they restrict the environment ρ to the free variables of the abstraction. Note that in [*fun*] it is only recorded that we have a recursively defined function; the unfolding of the recursion will not happen until it is called.

The clauses for application shows that the semantics is a *call-by-value* semantics: In an application we first evaluate the operator in a number of steps using the rule [app_1] and then we evaluate the operand in a number of steps using the rule [app_2]. The next stage is to use one of the rules [app_{fn}] or [app_{fun}] to bind the actual parameter to the formal parameter and, in the case of [app_{fun}], to unfold the recursive function so that subsequent recursive calls will be bound correctly. We shall use a `bind`-construct to contain the body of the function together with the appropriate environment. Finally, we

$$[if_1] \quad \frac{\rho \vdash ie_0 \rightarrow ie_0'}{\rho \vdash (\texttt{if } ie_0 \texttt{ then } e_1 \texttt{ else } e_2)^\ell \rightarrow (\texttt{if } ie_0' \texttt{ then } e_1 \texttt{ else } e_2)^\ell}$$

$$[if_2] \quad \rho \vdash (\texttt{if true}^{\ell_0} \texttt{ then } t_1^{\ell_1} \texttt{ else } t_2^{\ell_2})^\ell \rightarrow t_1^\ell$$

$$[if_3] \quad \rho \vdash (\texttt{if false}^{\ell_0} \texttt{ then } t_1^{\ell_1} \texttt{ else } t_2^{\ell_2})^\ell \rightarrow t_2^\ell$$

$$[let_1] \quad \frac{\rho \vdash ie_1 \rightarrow ie_1'}{\rho \vdash (\texttt{let } x = ie_1 \texttt{ in } e_2)^\ell \rightarrow (\texttt{let } x = ie_1' \texttt{ in } e_2)^\ell}$$

$$[let_2] \quad \rho \vdash (\texttt{let } x = v^{\ell_1} \texttt{ in } e_2)^\ell \rightarrow (\texttt{bind } \rho_0[x \mapsto v] \texttt{ in } e_2)^\ell \quad \text{where } \rho_0 = \rho \mid FV(e_2)$$

$$[op_1] \quad \frac{\rho \vdash ie_1 \rightarrow ie_1'}{\rho \vdash (ie_1 \ op \ ie_2)^\ell \rightarrow (ie_1' \ op \ ie_2)^\ell}$$

$$[op_2] \quad \frac{\rho \vdash ie_2 \rightarrow ie_2'}{\rho \vdash (v_1^{\ell_1} \ op \ ie_2)^\ell \rightarrow (v_1^{\ell_1} \ op \ ie_2')^\ell}$$

$$[op_3] \quad \rho \vdash (v_1^{\ell_1} \ op \ v_2^{\ell_2})^\ell \rightarrow v^\ell \qquad \text{if } v = v_1 \textbf{ op } v_2$$

Table 3.3: The Structural Operational Semantics of FUN (part 2).

evaluate the bind-construct using rule $[bind_1]$ a number of times, and we get the result of the application by using rule $[bind_2]$. The interplay between these rules is illustrated by the following example.

Example 3.7 Consider the expression $((\texttt{fn x => x}^1)^2 \ (\texttt{fn y => y}^3)^4)^5$ of Example 3.1. It has the following derivation sequence (explained below):

$$\begin{aligned}
[\,] \ \vdash \ & ((\texttt{fn x => x}^1)^2 \ (\texttt{fn y => y}^3)^4)^5 \\
\rightarrow \ & ((\texttt{close (fn x => x}^1\texttt{) in [ ]})^2 \ (\texttt{fn y => y}^3)^4)^5 \\
\rightarrow \ & ((\texttt{close (fn x => x}^1\texttt{) in [ ]})^2 \ (\texttt{close (fn y => y}^3\texttt{) in [ ]})^4)^5 \\
\rightarrow \ & (\texttt{bind [x} \mapsto \texttt{(close (fn y => y}^3\texttt{) in [ ])] in x}^1)^5 \\
\rightarrow \ & (\texttt{bind [x} \mapsto \texttt{(close (fn y => y}^3\texttt{) in [ ])] in} \\
& \quad \texttt{(close (fn y => y}^3\texttt{) in [ ])}^1)^5 \\
\rightarrow \ & (\texttt{close (fn y => y}^3\texttt{) in [ ]})^5
\end{aligned}$$

First $[app_1]$ and $[fn]$ are used to evaluate the operator, then $[app_2]$ and $[fn]$ are used to evaluate the operand and $[app_{fn}]$ introduces the bind-construct containing the local environment $[\texttt{x} \mapsto \texttt{(close (fn y => y}^3\texttt{) in [ ])}]$ needed to evaluate its body. So $\texttt{x}^1$ is evaluated using $[bind_1]$ and $[var]$, and finally $[bind_2]$ is used to get rid of the local environment. ∎

The semantics of the conditional is the usual one: first the condition is evaluated in a number of steps using rule $[if_1]$ and then the appropriate branch is selected by rules $[if_2]$ and $[if_3]$. For the local definitions we first compute the value of the bound variable in a number of steps using rule $[let_1]$ and then we introduce a `bind`-construct using rule $[let_2]$ reflecting that the body of the `let`-construct has to be evaluated in an extended environment. The rules $[bind_1]$ and $[bind_2]$ are now used to compute the result. For binary expressions we first evaluate the arguments using $[op_1]$ and $[op_2]$ and then the operation itself, denoted **op**, is performed using $[op_3]$.

As in Chapter 2 the labels have no impact on the semantics but are merely carried along. It is important to note that the outermost label never changes while inner labels may disappear; see for example the rules $[if_2]$ and $[bind_2]$. This is an important property of the semantics that is exploited by the 0-CFA analysis.

Example 3.8 Let us consider the expression, loop

$$\begin{array}{l}(\texttt{let g = (fun f x => (f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4\texttt{)}^5\\ \texttt{ in (g}^6\ \texttt{(fn z => z}^7\texttt{)}^8\texttt{)}^9\texttt{)}^{10}\end{array}$$

of Example 3.2 and see how the informal explanation of its semantics is captured in the formal semantics. First we introduce abbreviations for three closures:

$$\begin{array}{rcl}\mathbf{f} & = & \texttt{close (fun f x => (f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4\texttt{) in [ ]}\\ \mathbf{id}_y & = & \texttt{close (fn y => y}^2\texttt{) in [ ]}\\ \mathbf{id}_z & = & \texttt{close (fn z => z}^7\texttt{) in [ ]}\end{array}$$

Then we have the following derivation sequence

$$\begin{array}{ll}[\,] \vdash & \texttt{loop}\\ \rightarrow & \texttt{(let g = f}^5\ \texttt{in (g}^6\ \texttt{(fn z => z}^7\texttt{)}^8\texttt{)}^9\texttt{)}^{10}\\ \rightarrow & \texttt{(bind [g} \mapsto \texttt{f] in (g}^6\ \texttt{(fn z => z}^7\texttt{)}^8\texttt{)}^9\texttt{)}^{10}\\ \rightarrow & \texttt{(bind [g} \mapsto \texttt{f] in (f}^6\ \texttt{(fn z => z}^7\texttt{)}^8\texttt{)}^9\texttt{)}^{10}\\ \rightarrow & \texttt{(bind [g} \mapsto \texttt{f] in (f}^6\ \mathbf{id}_z^8\texttt{)}^9\texttt{)}^{10}\\ \rightarrow & \texttt{(bind [g} \mapsto \texttt{f] in}\\ & \quad \texttt{(bind [f} \mapsto \texttt{f][x} \mapsto \mathbf{id}_z\texttt{] in (f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4\texttt{)}^9\texttt{)}^{10}\\ \rightarrow^* & \texttt{(bind [g} \mapsto \texttt{f] in}\\ & \quad \texttt{(bind [f} \mapsto \texttt{f][x} \mapsto \mathbf{id}_z\texttt{] in}\\ & \quad\quad \texttt{(bind [f} \mapsto \texttt{f][x} \mapsto \mathbf{id}_y\texttt{] in (f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4\texttt{)}^4\texttt{)}^9\texttt{)}^{10}\\ \rightarrow^* & \ldots\end{array}$$

showing that the program does indeed loop. ∎

$$[\mathit{bind}] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models (\texttt{bind } \rho \texttt{ in } it_0^{\ell_0})^{\ell} \text{ iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models it_0^{\ell_0} \wedge \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell) \wedge \rho\, \mathcal{R}\, \widehat{\rho}$$

$$[\mathit{close}] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models (\texttt{close } t_0 \texttt{ in } \rho)^{\ell} \text{ iff } \{t_0\} \subseteq \widehat{\mathsf{C}}(\ell) \wedge \rho\, \mathcal{R}\, \widehat{\rho}$$

Table 3.4: Abstract Control Flow Analysis for intermediate expressions.

3.2.2 Semantic Correctness

We shall formulate semantic correctness of the Control Flow Analysis as a *subject reduction result*; this is an approach borrowed from type theory and merely says that an acceptable result of the analysis remains acceptable under evaluation. However, in order to do that we need to extend the analysis to intermediate expressions.

Analysis of intermediate expressions. The clauses for the constructs `bind` ρ `in` *ie* and `close` t_0 `in` ρ are given in Table 3.4; the remaining clauses are as in Table 3.1 (with the obvious replacements of expressions with intermediate expressions).

The clause [*bind*] reflects that its body will be executed and hence whatever it evaluates to will also be a possible value for the construct. Additionally, it expresses that there is a certain relationship $\mathcal{R}$ between the local environment (of the semantics) and the abstract environment (of the analysis). The clause [*close*] is similar in spirit to the clauses for function abstraction: the term of the closure is a possible value of the construct. Additionally, there has to be a relationship $\mathcal{R}$ between the two environments.

Correctness relation. The purpose of the global abstract environment, $\widehat{\rho}$, is to model *all* of the local environments arising during evaluation. We formalise this by defining the *correctness relation*

$$\mathcal{R} : (\mathbf{Env} \times \widehat{\mathbf{Env}}) \to \{\mathit{true}, \mathit{false}\}$$

and demanding that $\rho\, \mathcal{R}\, \widehat{\rho}$ for all local environments, ρ, occurring in the intermediate expressions. We then define:

$$\rho\, \mathcal{R}\, \widehat{\rho} \text{ iff } \mathit{dom}(\rho) \subseteq \mathit{dom}(\widehat{\rho}) \wedge \forall x \in \mathit{dom}(\rho)\ \forall t_x\ \forall \rho_x : (\rho(x) = \texttt{close } t_x \texttt{ in } \rho_x) \Rightarrow (t_x \in \widehat{\rho}(x) \wedge \rho_x\, \mathcal{R}\, \widehat{\rho})$$

This clearly demands that the function abstraction, t_x, in $\rho(x)$ must be an element of $\widehat{\rho}(x)$. It also shows that all local environments reachable from ρ, e.g. ρ_x, must be modelled by $\widehat{\rho}$ as well. Note that the relation $\mathcal{R}$ is well-defined because each recursive call is performed on a local environment that

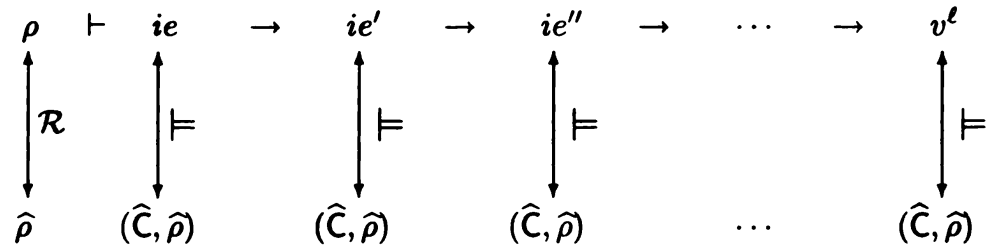

Figure 3.3: Preservation of analysis result.

is *strictly smaller* than that of the call itself; thus a simple proof by well-founded induction (Appendix B) suffices for showing the well-definedness of $\mathcal{R}$.

Example 3.9 Suppose that:

$$\begin{aligned} \rho &= [x \mapsto \texttt{close } t_1 \texttt{ in } \rho_1][y \mapsto \texttt{close } t_2 \texttt{ in } \rho_2] \\ \rho_1 &= [\,] \\ \rho_2 &= [x \mapsto \texttt{close } t_3 \texttt{ in } \rho_3] \\ \rho_3 &= [\,] \end{aligned}$$

Then $\rho \mathcal{R} \widehat{\rho}$ amounts to $\{t_1, t_3\} \subseteq \widehat{\rho}(x) \wedge \{t_2\} \subseteq \widehat{\rho}(y)$. ∎

We shall sometimes find it helpful to split the definition of $\mathcal{R}$ into two components. For this we make use of the auxiliary relation

$$\mathcal{V} : (\mathbf{Val} \times (\widehat{\mathbf{Env}} \times \widehat{\mathbf{Val}})) \to \{true, false\}$$

and define $\mathcal{V}$ and $\mathcal{R}$ by mutual recursion:

$$\begin{aligned} v\ \mathcal{V}\ (\widehat{\rho}, \widehat{v}) \quad &\text{iff} \quad \forall t\ \forall \rho : (v = \texttt{close } t \texttt{ in } \rho) \Rightarrow (t \in \widehat{v} \wedge \rho\ \mathcal{R}\ \widehat{\rho}) \\ \rho\ \mathcal{R}\ \widehat{\rho} \quad &\text{iff} \quad dom(\rho) \subseteq dom(\widehat{\rho}) \wedge \forall x \in dom(\rho) : \rho(x)\ \mathcal{V}\ (\widehat{\rho}, \widehat{\rho}(x)) \end{aligned}$$

Clearly the two definitions of $\mathcal{R}$ are equivalent.

Correctness result. The correctness result is now expressed by:

Theorem 3.10
If $\rho\ \mathcal{R}\ \widehat{\rho}$ and $\rho \vdash ie \to ie'$ then $(\widehat{C}, \widehat{\rho}) \models ie$ implies $(\widehat{C}, \widehat{\rho}) \models ie'$.

This is illustrated in Figure 3.3 for a terminating evaluation sequence $\rho \vdash ie \to^* v^\ell$; note that the result is analogous to that of Corollary 2.17 for the Live Variables Analysis in Chapter 2.

The intuitive content of the result is as follows:

> If there is a possible evaluation of the program such that the function at a call point evaluates to some abstraction, then this abstraction has to be in the set of possible abstractions computed by the analysis.

To see this assume that $\rho \vdash t^\ell \rightarrow^* (\texttt{close}\ t_0\ \texttt{in}\ \rho_0)^\ell$ and that $(\widehat{\mathsf{C}}, \widehat{\rho}) \models t^\ell$ as well as $\rho\ \mathcal{R}\ \widehat{\rho}$. Then Theorem 3.10 (and an immediate numerical induction) gives that $(\widehat{\mathsf{C}}, \widehat{\rho}) \models (\texttt{close}\ t_0\ \texttt{in}\ \rho_0)^\ell$. Now from the clause [*close*] of Table 3.4 we get that $t_0 \in \widehat{\mathsf{C}}(\ell)$ as was claimed. It is worth noticing that if the program is closed, i.e. if it does not contain free variables, then ρ will be [ ] and the condition $\rho\ \mathcal{R}\ \widehat{\rho}$ is trivially fulfilled.

Note that the theorem expresses that *all* acceptable analysis results remain acceptable under evaluation. One advantage of this is that we do not need to rely on the existence of a least or "best" solution (to be proved in Subsection 3.2.3) in order to formulate the result. Indeed the result does not say that the "best" solution remains "best" – merely that it remains acceptable. More importantly, the result opens up the possibility that the efficient realisation of Sections 3.3 and 3.4 computes a more approximate solution than the least (perhaps using the techniques of Chapter 4). Finally, note that the formulation of the theorem crucially depends on having defined the analysis for all intermediate expressions rather than just all ordinary expressions.

We shall now turn to the proof of Theorem 3.10. We first state an important observation:

Fact 3.11 If $(\widehat{\mathsf{C}}, \widehat{\rho}) \models it^{\ell_1}$ and $\widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\mathsf{C}}(\ell_2)$ then $(\widehat{\mathsf{C}}, \widehat{\rho}) \models it^{\ell_2}$. ∎

Proof By cases on the clauses for "$\models$". ∎

We then prove Theorem 3.10:

Proof We assume that $\rho\ \mathcal{R}\ \widehat{\rho}$ and $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie$ and prove $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie'$ by induction on the structure of the inference tree for $\rho \vdash ie \rightarrow ie'$. Most cases simply amount to inspecting the defining clause for $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie$; note that this method of proof applies to all fixed points of a recursive definition and in particular also to the greatest fixed point. We only give the proofs for some of the more interesting cases.

The case [*var*]. Here $\rho \vdash ie \rightarrow ie'$ is:

$$\rho \vdash x^\ell \rightarrow v^\ell \quad \text{because} \quad x \in dom(\rho) \text{ and } v = \rho(x)$$

If $v = c$ there is nothing to prove so suppose that $v = \texttt{close}\ t_0\ \texttt{in}\ \rho_0$. From $\rho\ \mathcal{R}\ \widehat{\rho}$ we get $v\ \mathcal{V}\ (\widehat{\rho}, \widehat{\rho}(x))$ and hence $t_0 \in \widehat{\rho}(x)$ and $\rho_0\ \mathcal{R}\ \widehat{\rho}$. From $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie$ we get $\widehat{\rho}(x) \subseteq \widehat{\mathsf{C}}(\ell)$, and hence $t_0 \in \widehat{\mathsf{C}}(\ell)$. Since $t_0 \in \widehat{\mathsf{C}}(\ell)$ and $\rho_0\ \mathcal{R}\ \widehat{\rho}$ we have established $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie'$.

The case [*fn*]. Here $\rho \vdash ie \to ie'$ is:

$$\rho \vdash (\texttt{fn}\ x\ \texttt{=>}\ e_0)^{\ell} \to (\texttt{close}\ (\texttt{fn}\ x\ \texttt{=>}\ e_0)\ \texttt{in}\ \rho_0)^{\ell}$$
$$\text{where } \rho_0 = \rho \mid FV(\texttt{fn}\ x\ \texttt{=>}\ e_0)$$

From $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie$ we get $(\texttt{fn}\ x\ \texttt{=>}\ e_0) \in \widehat{\mathsf{C}}(\ell)$; from $\rho\ \mathcal{R}\ \widehat{\rho}$ it is immediate to get $\rho_0\ \mathcal{R}\ \widehat{\rho}$; this then establishes $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie'$.

The case $[app_1]$. Here $\rho \vdash ie \to ie'$ is:

$$\rho \vdash (ie_1\ ie_2)^{\ell} \to (ie_1'\ ie_2)^{\ell} \quad \text{because} \quad \rho \vdash ie_1 \to ie_1'$$

The defining clauses of $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie$ and $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie'$ are equal except that the former has $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie_1$ where the latter has $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie_1'$. From the induction hypothesis applied to

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie_1, \quad \rho\ \mathcal{R}\ \widehat{\rho}, \quad \text{and} \quad \rho \vdash ie_1 \to ie_1'$$

we get $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie_1'$ and the desired result then follows.

The case $[app_{fn}]$. Here $\rho \vdash ie \to ie'$ is:

$$\rho \vdash ((\texttt{close}\ (\texttt{fn}\ x\ \texttt{=>}\ t_0^{\ell_0})\ \texttt{in}\ \rho_1)^{\ell_1}\ v_2^{\ell_2})^{\ell} \to (\texttt{bind}\ \rho_1[x \mapsto v_2]\ \texttt{in}\ t_0^{\ell_0})^{\ell}$$

From $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie$ we have $(\widehat{\mathsf{C}}, \widehat{\rho}) \models (\texttt{close}\ (\texttt{fn}\ x\ \texttt{=>}\ t_0^{\ell_0})\ \texttt{in}\ \rho_1)^{\ell_1}$ which yields:

$$(\texttt{fn}\ x\ \texttt{=>}\ t_0^{\ell_0}) \in \widehat{\mathsf{C}}(\ell_1) \quad \text{and} \quad \rho_1\ \mathcal{R}\ \widehat{\rho}$$

Further we have $(\widehat{\mathsf{C}}, \widehat{\rho}) \models v_2^{\ell_2}$; in the case where $v_2 = c$, it is immediate that

$$v_2\ \mathcal{V}\ (\widehat{\rho}, \widehat{\mathsf{C}}(\ell_2))$$

and in the case where $v_2 = \texttt{close}\ t_2\ \texttt{in}\ \rho_2$ it follows from the definition of $(\widehat{\mathsf{C}}, \widehat{\rho}) \models v_2^{\ell_2}$. Finally, the first universally quantified formula of the definition of $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie$ gives:

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models t_0^{\ell_0}, \quad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x), \quad \text{and} \quad \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell)$$

Now observe that $v_2\ \mathcal{V}\ (\widehat{\rho}, \widehat{\rho}(x))$ since $\widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x)$ follows from the clause (app_{fn}). Since $\rho_1\ \mathcal{R}\ \widehat{\rho}$ we now have

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models t_0^{\ell_0}, \quad \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell), \quad \text{and} \quad (\rho_1[x \mapsto v_2])\ \mathcal{R}\ \widehat{\rho}$$

and this establishes the desired $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie'$.

The case $[bind_2]$. Here $\rho \vdash ie \to ie'$ is:

$$\rho \vdash (\texttt{bind}\ \rho_1\ \texttt{in}\ v_1^{\ell_1})^{\ell} \to v_1^{\ell}$$

From $(\widehat{\mathsf{C}}, \widehat{\rho}) \models ie$ we have $(\widehat{\mathsf{C}}, \widehat{\rho}) \models v_1^{\ell_1}$ as well as $\widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\mathsf{C}}(\ell)$ and the desired $(\widehat{\mathsf{C}}, \widehat{\rho}) \models v_1^{\ell}$ follows from Fact 3.11.

This completes the proof. ∎

Example 3.12 From Example 3.7 we have:

$$[\,] \vdash ((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5 \rightarrow^* (\texttt{close (fn y => y}^3\texttt{) in [\,]})^5$$

Next let $(\widehat{C}_e, \widehat{\rho}_e)$ be as in Example 3.3. Clearly $[\,] \mathcal{R} \widehat{\rho}_e$ and from Example 3.5 we have:

$$(\widehat{C}_e, \widehat{\rho}_e) \models ((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$$

According to Theorem 3.10 we can now conclude:

$$(\widehat{C}_e, \widehat{\rho}_e) \models (\texttt{close (fn y => y}^3\texttt{) in [\,]})^5$$

Using Table 3.4 it is easy to check that this is indeed the case. ∎

3.2.3 Existence of Solutions

Having defined the analysis in Table 3.1 it is natural to ask the following question: Does each expression e admit a Control Flow Analysis, i.e. does there exist $(\widehat{C}, \widehat{\rho})$ such that $(\widehat{C}, \widehat{\rho}) \models e$? We shall show that the answer to this question is *yes*.

However, this does not exclude the possibility of having many different analyses for the same expression so an additional question is: Does each expression e have a "least" Control Flow Analysis, i.e. does there exists $(\widehat{C}_0, \widehat{\rho}_0)$ such that $(\widehat{C}_0, \widehat{\rho}_0) \models e$ and such that whenever $(\widehat{C}, \widehat{\rho}) \models e$ then $(\widehat{C}_0, \widehat{\rho}_0)$ is "less than" $(\widehat{C}, \widehat{\rho})$? Again, the answer will be *yes*.

Here "least" is with respect to the partial order defined by:

$$\begin{aligned}(\widehat{C}_1, \widehat{\rho}_1) \sqsubseteq (\widehat{C}_2, \widehat{\rho}_2) \quad \text{iff} \quad & (\forall \ell \in \mathbf{Lab} : \widehat{C}_1(\ell) \subseteq \widehat{C}_2(\ell)) \ \wedge \\ & (\forall x \in \mathbf{Var} : \widehat{\rho}_1(x) \subseteq \widehat{\rho}_2(x))\end{aligned}$$

It will be the topic of Sections 3.3 and 3.4 (and Mini Project 3.1) to show that the least solution can be computed efficiently for all expressions. However, it may be instructive to give a general proof for the existence of least solutions also for intermediate expressions. To this end we recall the notion of a Moore family (see Appendix A and Exercise 2.7):

> A subset Y of a complete lattice $L = (L, \sqsubseteq)$ is a *Moore family* if and only if $(\bigsqcap Y') \in Y$ for all $Y' \subseteq Y$.

This property is also called the *model intersection property* because whenever we take the "intersection" of a number of "models" we still get a "model".

Proposition 3.13
For all $ie \in \mathbf{IExp}$ the set $\{(\widehat{C}, \widehat{\rho}) \mid (\widehat{C}, \widehat{\rho}) \models ie\}$ is a Moore family.

It is an immediate corollary that all intermediate expressions ie admit a Control Flow Analysis: Let Y' be the empty set; then $\bigsqcap Y'$ is an element of $\{(\widehat{C}, \widehat{\rho}) \mid (\widehat{C}, \widehat{\rho}) \models ie\}$ showing that there exists at least one analysis of ie.

It is also an immediate corollary that all intermediate expressions have a least Control Flow Analysis: Let Y' be the set $\{(\widehat{C}, \widehat{\rho}) \mid (\widehat{C}, \widehat{\rho}) \models ie\}$; then $\bigsqcap Y'$ is an element of $\{(\widehat{C}, \widehat{\rho}) \mid (\widehat{C}, \widehat{\rho}) \models ie\}$ so it will also be an analysis of ie. Clearly $\bigsqcap Y' \sqsubseteq (\widehat{C}, \widehat{\rho})$ for all other analyses $(\widehat{C}, \widehat{\rho})$ of ie so it is the least analysis result.

In preparation for the proof of Proposition 3.13 we shall first establish an auxiliary result for $\mathcal{R}$ and $\mathcal{V}$:

Lemma 3.14

(i) For all $\rho \in \mathbf{Env}$ the set $\{\widehat{\rho} \mid \rho \mathcal{R} \widehat{\rho}\}$ is a Moore family.
(ii) For all $v \in \mathbf{Val}$ the set $\{(\widehat{\rho}, \widehat{v}) \mid v \mathcal{V} (\widehat{\rho}, \widehat{v})\}$ is a Moore family. ∎

Proof To prove (i) we proceed by well-founded induction on ρ (which is also the manner in which the existence of the predicate was proved). Now assume that

$$\forall i \in I : \rho \mathcal{R} \widehat{\rho}_i$$

for some index set I and let us show that $\rho \mathcal{R} (\bigsqcap_i \widehat{\rho}_i)$. For this consider x, t_x, and ρ_x such that:

$$\rho(x) = \mathtt{close}\ t_x\ \mathtt{in}\ \rho_x$$

We then know

$$\forall i \in I : t_x \in \widehat{\rho}_i(x) \ \wedge\ \rho_x \mathcal{R} \widehat{\rho}_i$$

and using the induction hypothesis it follows that

$$t_x \in (\bigsqcap_i \widehat{\rho}_i)(x) \ \wedge\ \rho_x \mathcal{R} (\bigsqcap_i \widehat{\rho}_i)$$

(taking care when $I = \emptyset$).

To prove (ii) we simply expand the definition of $\mathcal{V}$ and note that the result then follows from (i). ∎

We now prove Proposition 3.13 using coinduction (see Appendix B):

Proof The ternary relation $\models$ of Tables 3.1 and 3.4 is the greatest fixed point of a function $\mathcal{Q}$ as explained in Section 3.1. Now assume that

$$\forall i \in I : (\widehat{C}_i, \widehat{\rho}_i) \models ie$$

and let us prove that $\bigsqcap_i (\widehat{C}_i, \widehat{\rho}_i) \models ie$. We shall proceed by coinduction (see Appendix B) so we start by defining the ternary relation Q' by:

$$(\widehat{C}', \widehat{\rho}')\ Q'\ ie' \ \text{ iff } \ (\widehat{C}', \widehat{\rho}') = \bigsqcap_i (\widehat{C}_i, \widehat{\rho}_i) \ \wedge\ \forall i \in I : (\widehat{C}_i, \widehat{\rho}_i) \models ie'$$

It is then immediate that we have:

$$\bigsqcap_i(\widehat{\mathsf{C}}_i,\widehat{\rho}_i)\ Q'\ ie$$

The coinduction proof principle requires that we prove

$$Q' \sqsubseteq \mathcal{Q}(Q')$$

and this amounts to assuming $(\widehat{\mathsf{C}}',\widehat{\rho}')\ Q'\ ie'$ and proving that $(\widehat{\mathsf{C}}',\widehat{\rho}')\ (\mathcal{Q}(Q'))\ ie'$. So let us assume that

$$\forall i \in I : (\widehat{\mathsf{C}}_i,\widehat{\rho}_i) \models ie'$$

and let us show that:

$$\bigsqcap_i(\widehat{\mathsf{C}}_i,\widehat{\rho}_i)\ (\mathcal{Q}(Q'))\ ie'$$

For this we consider each of the clauses for ie' in turn.

Here we shall only deal with the more complicated choice $ie' = (it_1^{\ell_1}\ it_2^{\ell_2})^{\ell}$. From

$$\forall i \in I : (\widehat{\mathsf{C}}_i,\widehat{\rho}_i) \models (it_1^{\ell_1}\ it_2^{\ell_2})^{\ell}$$

we get $\forall i \in I : (\widehat{\mathsf{C}}_i,\widehat{\rho}_i) \models it_1^{\ell_1}$ and hence:

$$\bigsqcap_i(\widehat{\mathsf{C}}_i,\widehat{\rho}_i)\ Q'\ it_1^{\ell_1}$$

Similarly we get:

$$\bigsqcap_i(\widehat{\mathsf{C}}_i,\widehat{\rho}_i)\ Q'\ it_2^{\ell_2}$$

Next consider $(\texttt{fn}\ x \texttt{ => } t_0^{\ell_0}) \in \bigcap_i(\widehat{\mathsf{C}}_i(\ell_1))$ and let us prove that:

$$\bigcap_i(\widehat{\mathsf{C}}_i(\ell_2)) \subseteq \bigcap_i(\widehat{\rho}_i(x)),\quad \bigcap_i(\widehat{\mathsf{C}}_i(\ell_0)) \subseteq \bigcap_i(\widehat{\mathsf{C}}_i(\ell)),\quad \bigsqcap_i(\widehat{\mathsf{C}}_i,\widehat{\rho}_i)\ Q'\ t_0^{\ell_0} \qquad (3.1)$$

For all $i \in I$ we have that $(\widehat{\mathsf{C}}_i,\widehat{\rho}_i) \models ie'$ and since $(\texttt{fn}\ x \texttt{ => } t_0^{\ell_0}) \in \widehat{\mathsf{C}}_i(\ell_1)$ we have

$$\widehat{\mathsf{C}}_i(\ell_2) \subseteq \widehat{\rho}_i(x),\quad \widehat{\mathsf{C}}_i(\ell_0) \subseteq \widehat{\mathsf{C}}_i(\ell),\quad \text{and}\quad (\widehat{\mathsf{C}}_i,\widehat{\rho}_i) \models t_0^{\ell_0}$$

and this then gives (3.1) as desired (taking care when $I = \emptyset$). The case of $(\texttt{fun}\ f\ x \texttt{ => } t_0^{\ell_0}) \in \bigcap_i(\widehat{\mathsf{C}}_i(\ell_1))$ is similar. This completes the proof. ∎

Example 3.15 Let us return to Example 3.5 and consider the following potential analysis results for $((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$:

	$(\widehat{\mathsf{C}}_e,\widehat{\rho}_e)$	$(\widehat{\mathsf{C}}'_e,\widehat{\rho}'_e)$	$(\widehat{\mathsf{C}}''_e,\widehat{\rho}''_e)$
1	{fn y => y³}	{fn y => y³}	{fn y => y³}
2	{fn x => x¹}	{fn x => x¹}	{fn x => x¹}
3	∅	{fn x => x¹}	{fn y => y³}
4	{fn y => y³}	{fn y => y³}	{fn y => y³}
5	{fn y => y³}	{fn y => y³}	{fn y => y³}
x	{fn y => y³}	{fn y => y³}	{fn y => y³}
y	∅	{fn x => x¹}	{fn y => y³}

It is straightforward to verify that

$$(\widehat{C}'_e, \widehat{\rho}'_e) \models ((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$$
$$(\widehat{C}''_e, \widehat{\rho}''_e) \models ((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$$

Now Proposition 3.13 ensures that also:

$$(\widehat{C}'_e \sqcap \widehat{C}''_e, \widehat{\rho}'_e \sqcap \widehat{\rho}''_e) \models ((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$$

Neither $(\widehat{C}'_e, \widehat{\rho}'_e)$ nor $(\widehat{C}''_e, \widehat{\rho}''_e)$ is a least solution. Their "intersection" $(\widehat{C}'_e \sqcap \widehat{C}''_e, \widehat{\rho}'_e \sqcap \widehat{\rho}''_e)$ is smaller and equals $(\widehat{C}_e, \widehat{\rho}_e)$ which turns out to be the least analysis result for the expression. ■

3.2.4 Coinduction versus Induction

One of the important aspects of the development of the abstract Control Flow Analysis in Table 3.1 is the *coinductive definition* of the acceptability relation:

$$\models \text{ as the } \textit{greatest} \text{ fixed point of a functional } \mathcal{Q}$$

An alternative might be an *inductive definition* of an acceptability relation:

$$\models' \text{ as the } \textit{least} \text{ fixed point of the functional } \mathcal{Q}.$$

However, in Example 3.6 we argued that this might be inappropriate and here we are going to demonstrate that an important part of the development of the previous subsection actually fails for the least fixed point ($\models'$) of $\mathcal{Q}$.

Proposition 3.16
There exists $e_\star \in \mathbf{Exp}$ such that $\{(\widehat{C}, \widehat{\rho}) \mid (\widehat{C}, \widehat{\rho}) \models' e_\star\}$ is *not* a Moore family.

Proof (sketch) This proof is rather demanding and is best omitted on a first reading. To make the proof tractable we consider

$$\begin{aligned} e_\star &= t^\ell_\star \\ t_\star &= (\texttt{fn x => } (\texttt{x}^\ell\ \texttt{x}^\ell)^\ell)^\ell\ (\texttt{fn x => } (\texttt{x}^\ell\ \texttt{x}^\ell)^\ell)^\ell \end{aligned}$$

and take:

$$\begin{aligned} \mathbf{Lab}_e &= \{\ell\} \\ \mathbf{Var}_e &= \{\texttt{x}\} \\ \mathbf{Term}_e &= \{t_\star, \texttt{fn x => } (\texttt{x}^\ell\ \texttt{x}^\ell)^\ell, \texttt{x}^\ell\ \texttt{x}^\ell, \texttt{x}\} \\ \mathbf{IExp}_e &= \{t^\ell \mid t \in \mathbf{Term}_e\} \\ \widehat{\mathbf{Val}}_e &= \mathcal{P}(\{\texttt{fn x => } (\texttt{x}^\ell\ \texttt{x}^\ell)^\ell\}) = \{\emptyset, \{\texttt{fn x => } (\texttt{x}^\ell\ \texttt{x}^\ell)^\ell\}\} \end{aligned}$$

This is in line with Exercise 3.2 (and the development of Subsection 3.3.2) and as we shall see the proof of Proposition 3.13 is not invalidated.

Next let Q be the functional defined by Table 3.1 and let Q be in the domain of $\mathcal{Q}$. The condition

$$Q = \mathcal{Q}(Q)$$

is equivalent to:

$$\forall t \in \mathbf{Term_e} : \forall(\widehat{\mathsf{C}},\widehat{\rho}) : ((\widehat{\mathsf{C}},\widehat{\rho})\ Q\ t^\ell \ \text{ iff } \ (\widehat{\mathsf{C}},\widehat{\rho})\ \mathcal{Q}(Q)\ t^\ell)$$

By considering the four possibilities of $t \in \mathbf{Term_e}$ this is then equivalent to the conjunction of the following four conditions (where $(\widehat{\mathsf{C}},\widehat{\rho})$ is universally quantified):

$$\begin{aligned}
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ \mathtt{x}^\ell \quad &\text{iff} \quad \widehat{\rho}(\mathtt{x}) \subseteq \widehat{\mathsf{C}}(\ell)\\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{fn\ x => (x}^\ell\ \mathtt{x}^\ell)^\ell)^\ell \quad &\text{iff} \quad \{\mathtt{fn\ x => (x}^\ell\ \mathtt{x}^\ell)^\ell\} \subseteq \widehat{\mathsf{C}}(\ell)\\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{x}^\ell\ \mathtt{x}^\ell)^\ell \quad &\text{iff} \quad (\widehat{\mathsf{C}},\widehat{\rho})\ Q\ \mathtt{x}^\ell \ \wedge\\
&\qquad \widehat{\mathsf{C}}(\ell) \neq \emptyset \ \Rightarrow \ ((\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{x}^\ell\ \mathtt{x}^\ell)^\ell \ \wedge\\
&\qquad\qquad \widehat{\mathsf{C}}(\ell) \subseteq \widehat{\rho}(\mathtt{x}))\\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ t_\star^\ell \quad &\text{iff} \quad (\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{fn\ x => (x}^\ell\ \mathtt{x}^\ell)^\ell)^\ell \ \wedge\\
&\qquad \widehat{\mathsf{C}}(\ell) \neq \emptyset \ \Rightarrow \ ((\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{x}^\ell\ \mathtt{x}^\ell)^\ell \ \wedge\\
&\qquad\qquad \widehat{\mathsf{C}}(\ell) \subseteq \widehat{\rho}(\mathtt{x}))
\end{aligned}$$

Here we have used that $\widehat{\mathsf{C}}(\ell) \neq \emptyset$ implies that $\widehat{\mathsf{C}}(\ell) = \{\mathtt{fn\ x => (x}^\ell\ \mathtt{x}^\ell)^\ell\}$ as follows from the definition of $\widehat{\mathbf{Val_e}}$ in the beginning of this proof.

The conjunction of the above four conditions implies the conjunction of the following four conditions:

$$\begin{aligned}
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ \mathtt{x}^\ell \quad &\text{iff} \quad \widehat{\rho}(\mathtt{x}) \subseteq \widehat{\mathsf{C}}(\ell)\\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{fn\ x => (x}^\ell\ \mathtt{x}^\ell)^\ell)^\ell \quad &\text{iff} \quad \{\mathtt{fn\ x => (x}^\ell\ \mathtt{x}^\ell)^\ell\} \subseteq \widehat{\mathsf{C}}(\ell)\\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{x}^\ell\ \mathtt{x}^\ell)^\ell \quad &\text{iff} \quad \widehat{\rho}(\mathtt{x}) \subseteq \widehat{\mathsf{C}}(\ell) \ \wedge\\
&\qquad (\widehat{\mathsf{C}}(\ell) \neq \emptyset \ \Rightarrow \ (\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{x}^\ell\ \mathtt{x}^\ell)^\ell) \ \wedge\\
&\qquad \widehat{\mathsf{C}}(\ell) \subseteq \widehat{\rho}(\mathtt{x})\\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ t_\star^\ell \quad &\text{iff} \quad \{\mathtt{fn\ x => (x}^\ell\ \mathtt{x}^\ell)^\ell\} \subseteq \widehat{\mathsf{C}}(\ell) \ \wedge\\
&\qquad (\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{x}^\ell\ \mathtt{x}^\ell)^\ell \ \wedge\\
&\qquad \widehat{\mathsf{C}}(\ell) \subseteq \widehat{\rho}(\mathtt{x})
\end{aligned}$$

This implication can be reversed and this shows that also the conjunct of the above four conditions is equivalent to $Q = \mathcal{Q}(Q)$.

Using that $\widehat{\rho}(\mathtt{x})$ can only be $\emptyset$ or $\{\mathtt{fn\ x => (x}^\ell\ \mathtt{x}^\ell)^\ell\}$, and similarly for $\widehat{\mathsf{C}}(\ell)$, the above four conditions are equivalent to the following:

$$(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ \mathtt{x}^\ell \quad \text{iff} \quad \widehat{\rho}(\mathtt{x}) \subseteq \widehat{\mathsf{C}}(\ell)$$

$$
\begin{array}{rll}
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{fn\ x\ =>}\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell})^{\ell} & \text{iff} & \{\mathtt{fn\ x\ =>}\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell}\} = \widehat{\mathsf{C}}(\ell) \\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell} & \text{iff} & \widehat{\rho}(\mathtt{x}) = \widehat{\mathsf{C}}(\ell)\ \wedge \\
 & & (\widehat{\mathsf{C}}(\ell) \neq \emptyset\ \Rightarrow\ (\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell}) \\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ t_{\star}^{\ell} & \text{iff} & \{\mathtt{fn\ x\ =>}\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell}\} = \widehat{\mathsf{C}}(\ell) = \widehat{\rho}(\mathtt{x})\ \wedge \\
 & & (\widehat{\mathsf{C}},\widehat{\rho})\ Q\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell}
\end{array}
$$

It follows that the conjunct of the above four conditions is once more equivalent to $Q = \mathcal{Q}(Q)$.

The crucial case in the definition of $(\widehat{\mathsf{C}},\widehat{\rho})\ Q\ e$ is for $e = (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell}$ as this determines the truth or falsity of all other cases. We shall now try to get a handle on the candidates $Q_1, \cdots, Q_n$ for satisfying $Q_i = \mathcal{Q}(Q_i)$. Concentrating on the condition for $(\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell}$ it follows that $(\widehat{\mathsf{C}},\widehat{\rho})\ Q_i\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell}$ must demand that $\widehat{\mathsf{C}}(\ell) = \widehat{\rho}(\mathtt{x})$. Since each of $\widehat{\mathsf{C}}(\ell)$ and $\widehat{\rho}(\mathtt{x})$ can only be $\{\mathtt{fn\ x\ =>}\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell}\}$ or $\emptyset$ there are at most the following four candidates for Q_i:

$$
\begin{array}{lll}
(\widehat{\mathsf{C}},\widehat{\rho})\ Q_1\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell} & \text{iff} & \widehat{\mathsf{C}}(\ell) = \widehat{\rho}(\mathtt{x}) \\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q_2\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell} & \text{iff} & \widehat{\mathsf{C}}(\ell) = \widehat{\rho}(\mathtt{x}) = \emptyset \\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q_3\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell} & \text{iff} & \widehat{\mathsf{C}}(\ell) = \widehat{\rho}(\mathtt{x}) \neq \emptyset \\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q_4\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell} & \text{iff} & \textit{false}
\end{array}
$$

Verifying the condition

$$
\forall(\widehat{\mathsf{C}},\widehat{\rho}) : \left(\begin{array}{lll} (\widehat{\mathsf{C}},\widehat{\rho})\ Q_i\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell} & \text{iff} & \widehat{\rho}(\mathtt{x}) = \widehat{\mathsf{C}}(\ell)\ \wedge \\ & & (\widehat{\mathsf{C}}(\ell) \neq \emptyset\ \Rightarrow\ (\widehat{\mathsf{C}},\widehat{\rho})\ Q_i\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell}) \end{array} \right)
$$

for $i \in \{1,2,3,4\}$ it follows that Q_1 and Q_2 satisfy the condition whereas Q_3 and Q_4 do not.

It is now straightforward to verify also the remaining three conditions and it follows that:

$$Q_i = \mathcal{Q}(Q_i) \quad \text{for } i = 1,2$$

This means that Q_1 equals $\models$ (the greatest fixed point of $\mathcal{Q}$) and that Q_2 equals $\models'$ (the least fixed point of $\mathcal{Q}$). One can then calculate that

$$
\begin{array}{lll}
(\widehat{\mathsf{C}},\widehat{\rho})\ Q_1\ t_{\star}^{\ell} & \text{iff} & \widehat{\mathsf{C}}(\ell) = \widehat{\rho}(\mathtt{x}) \neq \emptyset \\
(\widehat{\mathsf{C}},\widehat{\rho})\ Q_2\ t_{\star}^{\ell} & \text{iff} & \textit{false}
\end{array}
$$

and this shows that

$$\{(\widehat{\mathsf{C}},\widehat{\rho}) \mid (\widehat{\mathsf{C}},\widehat{\rho})\ Q_1\ e_{\star}\} = \{(\widehat{\mathsf{C}},\widehat{\rho}) \mid \widehat{\mathsf{C}}(\ell) = \widehat{\rho}(\mathtt{x}) = \{\mathtt{fn\ x\ =>}\ (\mathtt{x}^{\ell}\ \mathtt{x}^{\ell})^{\ell}\}\}$$

which is a singleton set and in fact a Moore family, whereas

$$\{(\widehat{\mathsf{C}},\widehat{\rho}) \mid (\widehat{\mathsf{C}},\widehat{\rho})\ Q_2\ e_{\star}\} = \emptyset$$

which cannot be a Moore family (since a Moore family is never empty). This completes the proof. ∎

3.3 Syntax Directed 0-CFA Analysis

We shall now show how to obtain efficient realisations of 0-CFA analyses. So assume throughout this section that $e_\star \in \mathbf{Exp}$ is the expression of interest and that we want to find a "good" solution $(\widehat{\mathsf{C}}, \widehat{\rho})$ satisfying $(\widehat{\mathsf{C}}, \widehat{\rho}) \models e_\star$. This entails finding a solution that is *as small as possible* with respect to the partial order $\sqsubseteq$ defined in Section 3.2 by:

$$(\widehat{\mathsf{C}}_1, \widehat{\rho}_1) \sqsubseteq (\widehat{\mathsf{C}}_2, \widehat{\rho}_2) \text{ iff } (\forall \ell : \widehat{\mathsf{C}}_1(\ell) \subseteq \widehat{\mathsf{C}}_2(\ell)) \wedge (\forall x : \widehat{\rho}_1(x) \subseteq \widehat{\rho}_2(x))$$

Proposition 3.13 shows that a least solution does exist; however, the algorithm that is implicit in the proof does not have tractable (i.e. polynomial) complexity: it involves enumerating all candidate solutions, determining if they are indeed solutions, and if so taking the greatest lower bound with respect to the others found so far.

An alternative approach is somehow to obtain a finite set of constraints, say of the form *lhs* $\subseteq$ *rhs* (where *lhs* and *rhs* are much as described in Section 3.1), and then take the least solution to this system of constraints. The most obvious method is to expand the formula $(\widehat{\mathsf{C}}, \widehat{\rho}) \models e_\star$ by unfolding all "recursive calls", using memoisation to keep track of all the expansions that have been performed so far, and stopping the expansion whenever a previously expanded call is re-encountered.

Three phases. We shall take a more direct route motivated by the above considerations; it has three phases:

(i) The specification of Table 3.1 is reformulated in a syntax directed manner (Subsection 3.3.1).

(ii) The syntax directed specification is turned into an algorithm for constructing a finite set of constraints (Subsection 3.4.1).

(iii) The least solution of this set of constraints is computed (Subsection 3.4.2).

This is indeed a *common phenomenon*: a specification "$\models_A$" is reformulated into a specification "$\models_B$" ensuring that

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models_A e_\star \Leftarrow (\widehat{\mathsf{C}}, \widehat{\rho}) \models_B e_\star$$

so that "$\models_B$" is a *safe approximation* to "$\models_A$" and in particular the best (i.e. least) solution to "$\models_B e_\star$" is also a solution to "$\models_A e_\star$". This also ensures that all solutions to "$\models_B$" are semantically correct (assuming that this has already been established for all solutions to "$\models_A$"); however, we do not claim that a subject reduction result holds for $\models_B$ (even though it has been established for $\models_A$).

If additionally

$$(\widehat{C},\widehat{\rho}) \models_A e_\star \;\Rightarrow\; (\widehat{C},\widehat{\rho}) \models_B e_\star$$

then we can be assured that *no solutions are lost* and hence the best (i.e. least) solution to "$\models_B e_\star$" will also be the best (i.e. least) solution to "$\models_A e_\star$". As we shall see, it may be necessary to *restrict attention* to only solutions $(\widehat{C},\widehat{\rho})$ satisfying some additional properties (e.g. that only program fragments of $e_\star$ appear in the range of $\widehat{C}$ and $\widehat{\rho}$).

3.3.1 Syntax Directed Specification

In reformulating the specification of "$\models e_\star$" into a more computationally oriented specification "$\models_s e_\star$" we shall ensure that each function body is analysed *at most once* rather than each time the function could be applied. One way to achieve this is to analyse each function body *exactly once* as is done in the *syntax directed 0-CFA* analysis of Table 3.5; a better alternative would be to analyse only reachable function bodies and we refer to Mini Project 3.1 for how to achieve this. In Table 3.5 each function body is therefore analysed in the relevant clause for function abstraction rather than in the clause for function application; thus we now risk analysing unreachable program fragments.

The formal definition of $\models_s$. Since semantic correctness was dealt with in Section 3.2 there is no longer any need to consider intermediate expressions and consequently our specification of

$$(\widehat{C},\widehat{\rho}) \models_s e$$

in Table 3.5 considers ordinary expressions only. We shall take "$\models_s$" to be the *largest* relation that satisfies the specification; however, given the syntax directed nature of the specification there is in fact only one relation that satisfies the specification (see Exercise 3.9). Hence it would be technically correct, but intuitively misleading, to claim that we take the least relation that satisfies the specification. In other words, whether or not $\models_s$ is defined inductively or coinductively, the same relation is defined; this is a common phenomenon whenever the clauses are defined in a syntax directed manner.

Example 3.17 Consider the expression loop

$$\begin{array}{l}\texttt{(let g = (fun f x => (f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4\texttt{)}^5\\ \texttt{in (g}^6\ \texttt{(fn z => z}^7\texttt{)}^8\texttt{)}^9\texttt{)}^{10}\end{array}$$

of Example 3.4. We shall verify that $(\widehat{C}_{lp},\widehat{\rho}_{lp}) \models_s$ loop where $\widehat{C}_{lp}$ and $\widehat{\rho}_{lp}$ are as in Example 3.4. Using the clause [*let*], it is sufficient to show

$$(\widehat{C}_{lp},\widehat{\rho}_{lp}) \models_s \texttt{(fun f x => (f}^1\ \texttt{(fn y => y}^2\texttt{)}^3\texttt{)}^4\texttt{)}^5 \qquad (3.2)$$

$$(\widehat{C}_{lp},\widehat{\rho}_{lp}) \models_s \texttt{(g}^6\ \texttt{(fn z => z}^7\texttt{)}^8\texttt{)}^9 \qquad (3.3)$$

$$[con] \quad (\widehat{C},\widehat{\rho}) \models_s c^\ell \text{ always}$$

$$[var] \quad (\widehat{C},\widehat{\rho}) \models_s x^\ell \text{ iff } \widehat{\rho}(x) \subseteq \widehat{C}(\ell)$$

$$[fn] \quad (\widehat{C},\widehat{\rho}) \models_s (\texttt{fn } x \texttt{ => } e_0)^\ell \text{ iff } \{\texttt{fn } x \texttt{ => } e_0\} \subseteq \widehat{C}(\ell) \wedge (\widehat{C},\widehat{\rho}) \models_s e_0$$

$$[fun] \quad (\widehat{C},\widehat{\rho}) \models_s (\texttt{fun } f\ x \texttt{ => } e_0)^\ell \text{ iff } \{\texttt{fun } f\ x \texttt{ => } e_0\} \subseteq \widehat{C}(\ell) \wedge (\widehat{C},\widehat{\rho}) \models_s e_0 \wedge \{\texttt{fun } f\ x \texttt{ => } e_0\} \subseteq \widehat{\rho}(f)$$

$$[app] \quad (\widehat{C},\widehat{\rho}) \models_s (t_1^{\ell_1}\ t_2^{\ell_2})^\ell \text{ iff } (\widehat{C},\widehat{\rho}) \models_s t_1^{\ell_1} \wedge (\widehat{C},\widehat{\rho}) \models_s t_2^{\ell_2} \wedge (\forall(\texttt{fn } x \texttt{ => } t_0^{\ell_0}) \in \widehat{C}(\ell_1) : \widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell)) \wedge (\forall(\texttt{fun } f\ x \texttt{ => } t_0^{\ell_0}) \in \widehat{C}(\ell_1) : \widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell))$$

$$[if] \quad (\widehat{C},\widehat{\rho}) \models_s (\texttt{if } t_0^{\ell_0} \texttt{ then } t_1^{\ell_1} \texttt{ else } t_2^{\ell_2})^\ell \text{ iff } (\widehat{C},\widehat{\rho}) \models_s t_0^{\ell_0} \wedge (\widehat{C},\widehat{\rho}) \models_s t_1^{\ell_1} \wedge (\widehat{C},\widehat{\rho}) \models_s t_2^{\ell_2} \wedge \widehat{C}(\ell_1) \subseteq \widehat{C}(\ell) \wedge \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell)$$

$$[let] \quad (\widehat{C},\widehat{\rho}) \models_s (\texttt{let } x \texttt{ = } t_1^{\ell_1} \texttt{ in } t_2^{\ell_2})^\ell \text{ iff } (\widehat{C},\widehat{\rho}) \models_s t_1^{\ell_1} \wedge (\widehat{C},\widehat{\rho}) \models_s t_2^{\ell_2} \wedge \widehat{C}(\ell_1) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell)$$

$$[op] \quad (\widehat{C},\widehat{\rho}) \models_s (t_1^{\ell_1}\ op\ t_2^{\ell_2})^\ell \text{ iff } (\widehat{C},\widehat{\rho}) \models_s t_1^{\ell_1} \wedge (\widehat{C},\widehat{\rho}) \models_s t_2^{\ell_2}$$

Table 3.5: Syntax directed Control Flow Analysis.

since we have $\widehat{C}_{lp}(5) \subseteq \widehat{\rho}_{lp}(\mathtt{g})$ and $\widehat{C}_{lp}(9) \subseteq \widehat{C}_{lp}(10)$. To show (3.2) we use the clause [*fun*] and it is sufficient to show

$$(\widehat{C}_{lp},\widehat{\rho}_{lp}) \models_s (\mathtt{f}^1\ (\mathtt{fn\ y\ =>\ y}^2)^3)^4$$

since $\mathtt{f} \in \widehat{C}_{lp}(5)$ and $\mathtt{f} \in \widehat{\rho}_{lp}(\mathtt{f})$. Now $\widehat{C}_{lp}(1) = \{\mathtt{f}\}$ so, according to clause [*app*] this follows from

$$(\widehat{C}_{lp},\widehat{\rho}_{lp}) \models_s \mathtt{f}^1$$
$$(\widehat{C}_{lp},\widehat{\rho}_{lp}) \models_s (\mathtt{fn\ y\ =>\ y}^2)^3$$

since $\widehat{\mathsf{C}}_{\mathsf{lp}}(3) \subseteq \widehat{\rho}_{\mathsf{lp}}(\mathtt{x})$ and $\widehat{\mathsf{C}}_{\mathsf{lp}}(4) \subseteq \widehat{\mathsf{C}}_{\mathsf{lp}}(4)$. The first clause follows from [*var*] since $\widehat{\rho}_{\mathsf{lp}}(\mathtt{f}) \subseteq \widehat{\mathsf{C}}_{\mathsf{lp}}(1)$ and for the last clause we observe that $\mathsf{id}_y \in \widehat{\mathsf{C}}_{\mathsf{lp}}(3)$ and $(\widehat{\mathsf{C}}_{\mathsf{lp}}, \widehat{\rho}_{\mathsf{lp}}) \models_s \mathtt{y}^2$ as follows from $\widehat{\rho}_{\mathsf{lp}}(\mathtt{y}) \subseteq \widehat{\mathsf{C}}_{\mathsf{lp}}(2)$.

To show (3.3) we observe that $\widehat{\mathsf{C}}_{\mathsf{lp}}(6) = \{\mathsf{f}\}$ so using [*app*] it is sufficient to show

$$(\widehat{\mathsf{C}}_{\mathsf{lp}}, \widehat{\rho}_{\mathsf{lp}}) \models_s \mathtt{g}^6$$
$$(\widehat{\mathsf{C}}_{\mathsf{lp}}, \widehat{\rho}_{\mathsf{lp}}) \models_s (\mathtt{fn\ z=>\ z}^7)^8$$

since $\widehat{\mathsf{C}}_{\mathsf{lp}}(8) \subseteq \widehat{\rho}_{\mathsf{lp}}(\mathtt{x})$ and $\widehat{\mathsf{C}}_{\mathsf{lp}}(4) \subseteq \widehat{\mathsf{C}}_{\mathsf{lp}}(9)$. This is straightforward except for the last clause where we observe that $\mathsf{id}_z \in \widehat{\mathsf{C}}_{\mathsf{lp}}(8)$ and $(\widehat{\mathsf{C}}_{\mathsf{lp}}, \widehat{\rho}_{\mathsf{lp}}) \models_s \mathtt{z}^7$ as follows from $\widehat{\rho}_{\mathsf{lp}}(\mathtt{z}) \subseteq \widehat{\mathsf{C}}_{\mathsf{lp}}(7)$.

Note that because the analysis is syntax directed we have had *no need for coinduction*, unlike what was the case in Example 3.6. ∎

3.3.2 Preservation of Solutions

The specification of the analysis in Table 3.5 uses potentially infinite value spaces although this is not really necessary (as Exercise 3.2 demonstrates for Table 3.1). We can easily restrict ourselves to entities occurring in the original expression and this forms the basis for relating the results of the analysis of Table 3.5 to those of the analysis of Table 3.1.

So let $\mathbf{Lab}_\star \subseteq \mathbf{Lab}$ be the finite set of labels occurring in the program $e_\star$ of interest, let $\mathbf{Var}_\star \subseteq \mathbf{Var}$ be the finite set of variables occurring in $e_\star$ and let $\mathbf{Term}_\star$ be the finite set of subterms occurring in $e_\star$. Define $(\widehat{\mathsf{C}}_\star^\top, \widehat{\rho}_\star^\top)$ by:

$$\widehat{\mathsf{C}}_\star^\top(\ell) = \begin{cases} \emptyset & \text{if } \ell \notin \mathbf{Lab}_\star \\ \mathbf{Term}_\star & \text{if } \ell \in \mathbf{Lab}_\star \end{cases}$$

$$\widehat{\rho}_\star^\top(x) = \begin{cases} \emptyset & \text{if } x \notin \mathbf{Var}_\star \\ \mathbf{Term}_\star & \text{if } x \in \mathbf{Var}_\star \end{cases}$$

Then the claim

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \sqsubseteq (\widehat{\mathsf{C}}_\star^\top, \widehat{\rho}_\star^\top)$$

intuitively expresses that $(\widehat{\mathsf{C}}, \widehat{\rho})$ is concerned only with subterms occurring in the expression $e_\star$; obviously, we are primarily interested in analysis results with that property. Actually, this condition can be "reformulated" as the technically more manageable

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \in \widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star$$

where we define $\widehat{\mathbf{Cache}}_\star = \mathbf{Lab}_\star \to \widehat{\mathbf{Val}}_\star$, $\widehat{\mathbf{Env}}_\star = \mathbf{Var}_\star \to \widehat{\mathbf{Val}}_\star$ and $\widehat{\mathbf{Val}}_\star = \mathcal{P}(\mathbf{Term}_\star)$.

We can now show that all the solutions to "$\models_s e_\star$" that are "less than" $(\widehat{C}_\star^\top, \widehat{\rho}_\star^\top)$ are solutions to "$\models e_\star$" as well:

Proposition 3.18
If $(\widehat{C}, \widehat{\rho}) \models_s e_\star$ and $(\widehat{C}, \widehat{\rho}) \sqsubseteq (\widehat{C}_\star^\top, \widehat{\rho}_\star^\top)$ then $(\widehat{C}, \widehat{\rho}) \models e_\star$.

Proof Assume that $(\widehat{C}, \widehat{\rho}) \models_s e_\star$ and that $(\widehat{C}, \widehat{\rho}) \sqsubseteq (\widehat{C}_\star^\top, \widehat{\rho}_\star^\top)$. Furthermore let $\mathbf{Exp}_\star$ be the set of expressions occurring in $e_\star$ and note that

$$\forall e \in \mathbf{Exp}_\star : (\widehat{C}, \widehat{\rho}) \models_s e \tag{3.4}$$

is an immediate consequence of the syntax directed nature of the definition of $\models_s$.

To show that $(\widehat{C}, \widehat{\rho}) \models e_\star$ we proceed by coinduction. We know that "$\models$" is defined coinductively by the specification of Table 3.1, i.e. "$\models = gfp(\mathcal{Q})$" where $\mathcal{Q}$ is the function (implicitly) defined by Table 3.1. Similarly, we know that "$\models_s = gfp(\mathcal{Q}_s)$" where $\mathcal{Q}_s$ is the function (implicitly) defined by Table 3.5.

Next write $(\widehat{C}', \widehat{\rho}') \models^* e'$ for $(\widehat{C}', \widehat{\rho}') = (\widehat{C}, \widehat{\rho}) \wedge e' \in \mathbf{Exp}_\star$. It now suffices to show

$$(\mathcal{Q}_s(\models_s) \cap \models^*) \subseteq \mathcal{Q}(\models_s \cap \models^*) \tag{3.5}$$

because then "$(\models_s \cap \models^*) \subseteq \mathcal{Q}(\models_s \cap \models^*)$" follows and hence by coinduction "$(\models_s \cap \models^*) \subseteq \models$" and since $(\widehat{C}, \widehat{\rho}) \models_s e_\star$ as well as $(\widehat{C}, \widehat{\rho}) \models^* e_\star$ we then have the required $(\widehat{C}, \widehat{\rho}) \models e_\star$.

The proof of (3.5) amounts to a comparison of the right hand sides of Table 3.5 and Table 3.1: for each clause we shall assume that the right hand side of Table 3.5 holds for $(\widehat{C}, \widehat{\rho}, e)$ and that $e \in \mathbf{Exp}_\star$ and we shall show that the corresponding right hand side of Table 3.1 holds when all occurrences of "$\models$" are replaced by "$\models_s \cap \models^*$".

The clauses [*con*], [*var*], [*if*], [*let*] and [*op*] are trivial as the right hand sides of Tables 3.5 and 3.1 are similar and the subterms will all be in $\mathbf{Exp}_\star$. The clauses [*fn*] and [*fun*] are straightforward as the right hand sides of Table 3.5 imply the right-hand sides of Table 3.1. Finally, we consider the clause [*app*]. For $(\mathtt{fn}\ x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1)$ we need to show that $(\widehat{C}, \widehat{\rho}) \models_s t_0^{\ell_0}$; but since $(\widehat{C}, \widehat{\rho}) \sqsubseteq (\widehat{C}_\star^\top, \widehat{\rho}_\star^\top)$ this follows from (3.4). For $(\mathtt{fun}\ f\ x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1)$ we need to show that $(\widehat{C}, \widehat{\rho}) \models_s t_0^{\ell_0}$ and that $(\mathtt{fun}\ f\ x \Rightarrow t_0^{\ell_0}) \in \widehat{\rho}(f)$; the first follows from (3.4) and the second is an immediate consequence of $(\widehat{C}, \widehat{\rho}) \models_s (\mathtt{fun}\ f\ x \Rightarrow t_0^{\ell_0})^\ell$ (for some ℓ) that again follows from (3.4). ∎

We show an analogue of Proposition 3.13 for the syntax directed analysis:

Proposition 3.19
$\{(\widehat{C}, \widehat{\rho}) \in \widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star \mid (\widehat{C}, \widehat{\rho}) \models_s e_\star\}$ is a *Moore family*.

This result has as immediate corollaries that:

- each expression $e_\star$ has a Control Flow Analysis that is "less than" $(\widehat{\mathsf{C}}^\top_\star, \widehat{\rho}^\top_\star)$, and
- each expression $e_\star$ has a "least" Control Flow Analysis that is "less than" $(\widehat{\mathsf{C}}^\top_\star, \widehat{\rho}^\top_\star)$.

This means that the properties obtained for the analysis of Table 3.1 in Subsection 3.2.3 also hold for the analysis of Table 3.5 with the additional restriction on the range of the analysis functions. In particular, any analysis result that is acceptable with respect to Table 3.5 (and properly restricted to $\widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star$) is also an acceptable analysis result with respect to Table 3.1. The converse relationship is studied in Exercise 3.11 and Mini Project 3.1.

Proof We shall write $(\widehat{\mathsf{C}}^\top_\star, \widehat{\rho}^\top_\star)$ also for the greatest element of $\widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star$. It is immediate to show that

(a) $(\widehat{\mathsf{C}}^\top_\star, \widehat{\rho}^\top_\star) \models_s e$

(b) if $(\widehat{\mathsf{C}}_1, \widehat{\rho}_1) \models_s e$ and $(\widehat{\mathsf{C}}_2, \widehat{\rho}_2) \models_s e$ then $((\widehat{\mathsf{C}}_1, \widehat{\rho}_1) \sqcap (\widehat{\mathsf{C}}_2, \widehat{\rho}_2)) \models_s e$.

for all subexpressions e of $e_\star$ by means of structural induction on e. This establishes (a) and (b) also for $e = e_\star$. Next consider some

$$Y \subseteq \{(\widehat{\mathsf{C}}, \widehat{\rho}) \in \widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star \mid (\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star\}$$

and note than one can write $Y = \{(\widehat{\mathsf{C}}_i, \widehat{\rho}_i) \mid i \in \{1, \cdots, n\}\}$ for some $n \geq 0$ since $\widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star$ is finite. That

$$\bigsqcap Y \in \{(\widehat{\mathsf{C}}, \widehat{\rho}) \in \widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star \mid (\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star\}$$

then follows from (a) and (b) because $\bigsqcap Y = (\widehat{\mathsf{C}}^\top_\star, \widehat{\rho}^\top_\star) \sqcap (\widehat{\mathsf{C}}_1, \widehat{\rho}_1) \sqcap \cdots \sqcap (\widehat{\mathsf{C}}_n, \widehat{\rho}_n)$. ∎

3.4 Constraint Based 0-CFA Analysis

We are now ready to consider efficient ways of finding the least solution $(\widehat{\mathsf{C}}, \widehat{\rho})$ such that $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star$. To do so we first construct a finite set $\mathcal{C}_\star[\![e_\star]\!]$ of *constraints* and *conditional constraints* of the form

$$\mathit{lhs} \subseteq \mathit{rhs} \tag{3.6}$$

$$\{t\} \subseteq \mathit{rhs}' \Rightarrow \mathit{lhs} \subseteq \mathit{rhs} \tag{3.7}$$

where *rhs* is of the form $\mathsf{C}(\ell)$ or $\mathsf{r}(x)$, and *lhs* is of the form $\mathsf{C}(\ell), \mathsf{r}(x)$, or $\{t\}$, and all occurrences of t are of the form $\texttt{fn}\ x \Rightarrow e_0$ or $\texttt{fun}\ f\ x \Rightarrow e_0$. To simplify the technical development we shall read (3.7) as

$$(\{t\} \subseteq \mathit{rhs}' \Rightarrow \mathit{lhs}) \subseteq \mathit{rhs}$$

and we shall write *ls* for *lhs* as well as $\{t\} \subseteq \mathit{rhs}' \Rightarrow \mathit{lhs}$.

$$[con] \quad C_\star[\![c^\ell]\!] = \emptyset$$

$$[var] \quad C_\star[\![x^\ell]\!] = \{\mathsf{r}(x) \subseteq \mathsf{C}(\ell)\}$$

$$[fn] \quad C_\star[\![(\mathtt{fn}\ x \Rightarrow e_0)^\ell]\!] = \{\{\mathtt{fn}\ x \Rightarrow e_0\} \subseteq \mathsf{C}(\ell)\} \cup C_\star[\![e_0]\!]$$

$$[fun] \quad C_\star[\![(\mathtt{fun}\ f\ x \Rightarrow e_0)^\ell]\!] = \{\{\mathtt{fun}\ f\ x \Rightarrow e_0\} \subseteq \mathsf{C}(\ell)\} \cup C_\star[\![e_0]\!] \cup \{\{\mathtt{fun}\ f\ x \Rightarrow e_0\} \subseteq \mathsf{r}(f)\}$$

$$[app] \quad C_\star[\![(t_1^{\ell_1}\ t_2^{\ell_2})^\ell]\!] = C_\star[\![t_1^{\ell_1}]\!] \cup C_\star[\![t_2^{\ell_2}]\!] \cup \{\{t\} \subseteq \mathsf{C}(\ell_1) \Rightarrow \mathsf{C}(\ell_2) \subseteq \mathsf{r}(x) \mid t = (\mathtt{fn}\ x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_\star\} \cup \{\{t\} \subseteq \mathsf{C}(\ell_1) \Rightarrow \mathsf{C}(\ell_0) \subseteq \mathsf{C}(\ell) \mid t = (\mathtt{fn}\ x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_\star\} \cup \{\{t\} \subseteq \mathsf{C}(\ell_1) \Rightarrow \mathsf{C}(\ell_2) \subseteq \mathsf{r}(x) \mid t = (\mathtt{fun}\ f\ x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_\star\} \cup \{\{t\} \subseteq \mathsf{C}(\ell_1) \Rightarrow \mathsf{C}(\ell_0) \subseteq \mathsf{C}(\ell) \mid t = (\mathtt{fun}\ f\ x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_\star\}$$

$$[if] \quad C_\star[\![(\mathtt{if}\ t_0^{\ell_0}\ \mathtt{then}\ t_1^{\ell_1}\ \mathtt{else}\ t_2^{\ell_2})^\ell]\!] = C_\star[\![t_0^{\ell_0}]\!] \cup C_\star[\![t_1^{\ell_1}]\!] \cup C_\star[\![t_2^{\ell_2}]\!] \cup \{\mathsf{C}(\ell_1) \subseteq \mathsf{C}(\ell)\} \cup \{\mathsf{C}(\ell_2) \subseteq \mathsf{C}(\ell)\}$$

$$[let] \quad C_\star[\![(\mathtt{let}\ x = t_1^{\ell_1}\ \mathtt{in}\ t_2^{\ell_2})^\ell]\!] = C_\star[\![t_1^{\ell_1}]\!] \cup C_\star[\![t_2^{\ell_2}]\!] \cup \{\mathsf{C}(\ell_1) \subseteq \mathsf{r}(x)\} \cup \{\mathsf{C}(\ell_2) \subseteq \mathsf{C}(\ell)\}$$

$$[op] \quad C_\star[\![(t_1^{\ell_1}\ op\ t_2^{\ell_2})^\ell]\!] = C_\star[\![t_1^{\ell_1}]\!] \cup C_\star[\![t_2^{\ell_2}]\!]$$

Table 3.6: Constraint based Control Flow Analysis.

Informally, the constraints are obtained by expanding the clauses defining $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star$ into a finite set of constraints of the above form and then letting $C_\star[\![e_\star]\!]$ be the set of individual conjuncts. One caveat is that all occurrences of "$\widehat{\mathsf{C}}$" are changed into "C" and that all occurrences of "$\widehat{\rho}$" are changed into "r" to avoid confusion: $\widehat{\mathsf{C}}(\ell)$ will be a *set of terms* whereas $\mathsf{C}(\ell)$ is *pure syntax* and similarly for $\widehat{\rho}(x)$ and $\mathsf{r}(x)$.

Formally, the *constraint based 0-CFA* analysis is defined by the function $C_\star$ of Table 3.6: it makes use of the set $\mathbf{Term}_\star$ of subterms occurring in the expression $e_\star$ in order to generate only a finite number of constraints in the clause for application; this is justified by Propositions 3.18 and 3.19.

If the size of the expression $e_\star$ is n then it might seem that there could be $O(n^2)$ constraints of the form (3.6) and $O(n^4)$ constraints of the form (3.7).

However, inspection of the definition of $\mathcal{C}_\star$ ensures that at most $O(n)$ constraints of the form (3.6) and $O(n^2)$ constraints of the form (3.7) are ever generated: each of the $O(n)$ constituents only generate $O(1)$ constraints of the form (3.6) and $O(n)$ constraints of the form (3.7).

Example 3.20 Consider the expression

$$((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$$

of Example 3.7. We generate the following set of constraints

$$\begin{aligned}
&\mathcal{C}_\star[\![((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5]\!] = \\
&\quad \{\ \{\texttt{fn x => x}^1\} \subseteq \mathsf{C}(2), \\
&\quad\ \ \mathsf{r}(\texttt{x}) \subseteq \mathsf{C}(1), \\
&\quad\ \ \{\texttt{fn y => y}^3\} \subseteq \mathsf{C}(4), \\
&\quad\ \ \mathsf{r}(\texttt{y}) \subseteq \mathsf{C}(3), \\
&\quad\ \ \{\texttt{fn x => x}^1\} \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(4) \subseteq \mathsf{r}(\texttt{x}), \\
&\quad\ \ \{\texttt{fn x => x}^1\} \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(1) \subseteq \mathsf{C}(5), \\
&\quad\ \ \{\texttt{fn y => y}^3\} \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(4) \subseteq \mathsf{r}(\texttt{y}), \\
&\quad\ \ \{\texttt{fn y => y}^3\} \subseteq \mathsf{C}(2) \Rightarrow \mathsf{C}(3) \subseteq \mathsf{C}(5)\ \}
\end{aligned}$$

where we use that $\texttt{fn x => x}^1$ and $\texttt{fn y => y}^3$ are the only abstractions in $\mathbf{Term}_\star$. ∎

3.4.1 Preservation of Solutions

It is important to stress that while $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star$ is a logical formula, $\mathcal{C}_\star[\![e_\star]\!]$ is a set of syntactic entities. To give meaning to the syntax we first translate the "C" and "r" symbols into the sets "$\widehat{\mathsf{C}}$" and "$\widehat{\rho}$":

$$\begin{aligned}
(\widehat{\mathsf{C}}, \widehat{\rho})[\![\mathsf{C}(\ell)]\!] &= \widehat{\mathsf{C}}(\ell) \\
(\widehat{\mathsf{C}}, \widehat{\rho})[\![\mathsf{r}(x)]\!] &= \widehat{\rho}(x)
\end{aligned}$$

To deal with the possible forms of *ls* we additionally take:

$$\begin{aligned}
(\widehat{\mathsf{C}}, \widehat{\rho})[\![\{t\}]\!] &= \{t\} \\
(\widehat{\mathsf{C}}, \widehat{\rho})[\![\{t\} \subseteq rhs' \Rightarrow lhs]\!] &= \begin{cases} (\widehat{\mathsf{C}}, \widehat{\rho})[\![lhs]\!] & \text{if } \{t\} \subseteq (\widehat{\mathsf{C}}, \widehat{\rho})[\![rhs']\!] \\ \emptyset & \text{otherwise} \end{cases}
\end{aligned}$$

Next we define a satisfaction relation $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c (ls \subseteq rhs)$ on the individual constraints:

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c (ls \subseteq rhs) \quad \text{iff} \quad (\widehat{\mathsf{C}}, \widehat{\rho})[\![ls]\!] \subseteq (\widehat{\mathsf{C}}, \widehat{\rho})[\![rhs]\!]$$

This definition can be lifted to a set C of constraints by:

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c C \text{ iff } \forall (ls \subseteq rhs) \in C : (\widehat{\mathsf{C}}, \widehat{\rho}) \models_c (ls \subseteq rhs)$$

We then have the following result showing that all solutions to the set $\mathcal{C}_\star[\![e_\star]\!]$ of constraints also satisfy the syntax directed specification of the Control Flow Analysis and vice versa:

Proposition 3.21
If $(\widehat{\mathsf{C}}, \widehat{\rho}) \sqsubseteq (\widehat{\mathsf{C}}_\star^\top, \widehat{\rho}_\star^\top)$ then

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star \text{ if and only if } (\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}_\star[\![e_\star]\!]$$

Thus the least solution $(\widehat{\mathsf{C}}, \widehat{\rho})$ to $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star$ equals the least solution to $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}_\star[\![e_\star]\!]$.

Proof A simple structural induction on e shows that

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e \text{ iff } (\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}_\star[\![e]\!]$$

for all subexpressions e of $e_\star$; in the case of the function application $(t_1^{\ell_1}\ t_2^{\ell_2})^\ell$ the assumption $(\widehat{\mathsf{C}}, \widehat{\rho}) \sqsubseteq (\widehat{\mathsf{C}}_\star^\top, \widehat{\rho}_\star^\top)$ is used to ensure that $\widehat{\mathsf{C}}(\ell_1) \subseteq \mathbf{Term}_\star$. ∎

3.4.2 Solving the Constraints

We shall present two approaches to solving the set of constraints $\mathcal{C}_\star[\![e_\star]\!]$. First we shall show that finding the least solution to $\mathcal{C}_\star[\![e_\star]\!]$ is equivalent to finding the least fixed point of a certain function; straightforward techniques allow us to compute the least fixed point in time $O(n^5)$ when the size of the expression $e_\star$ is n. Improvements upon this are possible, but to obtain the best known result we shall consider a graph representation of the problem; this will give us a $O(n^3)$ algorithm. This is indeed a *common phenomenon* in program analysis: syntax directed specifications are appropriate for correctness considerations but often they need to be "massaged" in order to obtain efficient implementations.

Fixed point formulation. To show that finding the solution of the set $\mathcal{C}_\star[\![e_\star]\!]$ of constraints is a *fixed point problem* we shall define a function

$$F_\star : \widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star \to \widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star$$

and show that it has a least fixed point $lfp(F_\star)$ that is indeed the least solution whose existence is guaranteed by Propositions 3.18 and 3.21.

We define the function $F_\star$ by

$$F_\star(\widehat{\mathsf{C}},\widehat{\rho}) = (F_1(\widehat{\mathsf{C}},\widehat{\rho}), F_2(\widehat{\mathsf{C}},\widehat{\rho}))$$

where:

$$\begin{aligned} F_1(\widehat{\mathsf{C}},\widehat{\rho})(\ell) &= \bigcup\{(\widehat{\mathsf{C}},\widehat{\rho})[\![ls]\!] \mid (ls \subseteq \mathsf{C}(\ell)) \in \mathcal{C}_\star[\![e_\star]\!]\} \\ F_2(\widehat{\mathsf{C}},\widehat{\rho})(x) &= \bigcup\{(\widehat{\mathsf{C}},\widehat{\rho})[\![ls]\!] \mid (ls \subseteq \mathsf{r}(x)) \in \mathcal{C}_\star[\![e_\star]\!]\} \end{aligned}$$

To see that this defines a monotone function it suffices to consider a constraint

$$lhs' \subseteq rhs' \Rightarrow lhs \subseteq rhs$$

in $\mathcal{C}_\star[\![e_\star]\!]$ and to observe that lhs' is of the form $\{t\}$; this ensures that $(\widehat{\mathsf{C}}_1,\widehat{\rho}_1) \sqsubseteq (\widehat{\mathsf{C}}_2,\widehat{\rho}_2)$ implies $F_i(\widehat{\mathsf{C}}_1,\widehat{\rho}_1) \sqsubseteq F_i(\widehat{\mathsf{C}}_2,\widehat{\rho}_2)$ (for $i = 1,2$) because if $\{t\} \subseteq (\widehat{\mathsf{C}}_1,\widehat{\rho}_1)[\![rhs']\!]$ then also $\{t\} \subseteq (\widehat{\mathsf{C}}_2,\widehat{\rho}_2)[\![rhs']\!]$. Since $\widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star$ is a complete lattice this means that $F_\star$ has a least fixed point and it turns out also to be the least solution to the set $\mathcal{C}_\star[\![e_\star]\!]$ of constraints:

Proposition 3.22
$lfp(F_\star) = \bigsqcap\{(\widehat{\mathsf{C}},\widehat{\rho}) \mid (\widehat{\mathsf{C}},\widehat{\rho}) \models_c \mathcal{C}_\star[\![e_\star]\!]\}$

Proof It is easy to verify that:

$$F_\star(\widehat{\mathsf{C}},\widehat{\rho}) \sqsubseteq (\widehat{\mathsf{C}},\widehat{\rho}) \text{ iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models_c \mathcal{C}_\star[\![e_\star]\!]$$

Using the formula $lfp(f) = \bigsqcap\{x \mid f(x) \sqsubseteq x\}$ (see Appendix A) the result then follows. ∎

If the size of $e_\star$ is n then an element $(\widehat{\mathsf{C}},\widehat{\rho})$ of $\widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star$ may be viewed as an $O(n)$-tuple of values from $\widehat{\mathbf{Val}}_\star$. Since $\widehat{\mathbf{Val}}_\star$ is a lattice of height $O(n)$ this means that $\widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Val}}_\star$ has height $O(n^2)$ and hence the formula

$$lfp(F_\star) = \bigsqcup_m F_\star^m(\bot)$$

may be used to compute the least fixed point in at most $O(n^2)$ iterations. A naive approach will need to consider all $O(n^2)$ constraints to determine the value of each of the $O(n)$ components of the new iterant; this yields an overall $O(n^5)$ bound on the cost.

Graph formulation. An alternative method for computing the least solution to the set $\mathcal{C}_\star[\![e_\star]\!]$ of constraints is to use a *graph formulation of the constraints.* The graph will have nodes $\mathsf{C}(\ell)$ and $\mathsf{r}(x)$ for $\ell \in \mathbf{Lab}_\star$ and $x \in \mathbf{Var}_\star$. Associated with each node p we have a data field $\mathsf{D}[\mathsf{p}]$ that initially is given by:

$$\mathsf{D}[\mathsf{p}] = \{t \mid (\{t\} \subseteq p) \in \mathcal{C}_\star[\![e_\star]\!]\}$$

The graph will have edges for a subset of the constraints in $\mathcal{C}_\star[\![e_\star]\!]$; each edge will be decorated with the constraint that gives rise to it:

INPUT: $\mathcal{C}_\star[\![e_\star]\!]$

OUTPUT: $(\widehat{\mathsf{C}}, \widehat{\rho})$

METHOD: Step 1: Initialisation

```
W := nil;
for q in Nodes do D[q] := ∅;
for q in Nodes do E[q] := nil;
```

Step 2: Building the graph

```
for cc in C⋆[⋆] do
    case cc of
        {t} ⊆ p: add(p,{t});
        p1 ⊆ p2: E[p1] := cons(cc,E[p1]);
        {t} ⊆ p ⇒ p1 ⊆ p2:
                E[p1] := cons(cc,E[p1]);
                E[p] := cons(cc,E[p]);
```

Step 3: Iteration

```
while W ≠ nil do
    q := head(W); W := tail(W);
    for cc in E[q] do
        case cc of
            p1 ⊆ p2: add(p2, D[p1]);
            {t} ⊆ p ⇒ p1 ⊆ p2:
                if t ∈ D[p] then add(p2, D[p1]);
```

Step 4: Recording the solution

```
for ℓ in Lab⋆ do Ĉ(ℓ) := D[C(ℓ)];
for x in Var⋆ do ρ̂(x) := D[r(x)];
```

USING:

```
procedure add(q,d) is
    if ¬ (d ⊆ D[q])
    then  D[q] := D[q] ∪ d;
          W := cons(q,W);
```

Table 3.7: Algorithm for solving constraints.

- a constraint $p_1 \subseteq p_2$ gives rise to an edge from p_1 to p_2, and
- a constraint $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$ gives rise to an edge from p_1 to p_2 and an edge from p to p_2.

Having constructed the graph we now traverse all edges in order to propagate information from one data field to another. We make certain only to traverse

p	D[p]	E[p]
C(1)	$\emptyset$	[$\mathsf{id}_x \subseteq$ C(2) $\Rightarrow$ C(1) $\subseteq$ C(5)]
C(2)	id_x	[$\mathsf{id}_y \subseteq$ C(2) $\Rightarrow$ C(3) $\subseteq$ C(5), $\mathsf{id}_y \subseteq$ C(2) $\Rightarrow$ C(4) $\subseteq$ r(y), $\mathsf{id}_x \subseteq$ C(2) $\Rightarrow$ C(1) $\subseteq$ C(5), $\mathsf{id}_x \subseteq$ C(2) $\Rightarrow$ C(4) $\subseteq$ r(x)]
C(3)	$\emptyset$	[$\mathsf{id}_y \subseteq$ C(2) $\Rightarrow$ C(3) $\subseteq$ C(5)]
C(4)	id_y	[$\mathsf{id}_y \subseteq$ C(2) $\Rightarrow$ C(4) $\subseteq$ r(y), $\mathsf{id}_x \subseteq$ C(2) $\Rightarrow$ C(4) $\subseteq$ r(x)]
C(5)	$\emptyset$	[ ]
r(x)	$\emptyset$	[r(x) $\subseteq$ C(1)]
r(y)	$\emptyset$	[r(y) $\subseteq$ C(3)]

Figure 3.4: Initialisation of data structures for example program.

an edge from p_1 to p_2 when D[p_1] is extended with a term not previously there (and this incorporates the situation where D[p_1] is initially set to a non-empty set). Furthermore, an edge decorated with $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$ is only traversed if in fact $t \in$ D[p].

To be more specific consider the algorithm of Table 3.7. It takes as *input* a set $\mathcal{C}_\star[\![e_\star]\!]$ of constraints and produces as *output* a solution $(\widehat{\mathsf{C}}, \widehat{\rho}) \in \widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star$. It operates on the following main *data structures*:

- a *worklist* W, i.e. a list of nodes whose outgoing edges should be traversed;
- a *data array* D that for each node gives an element of $\widehat{\mathbf{Val}}_\star$; and
- an *edge array* E that for each node gives a list of constraints from which a list of the successor nodes can be computed.

The set Nodes consists of C(ℓ) for all ℓ in $\mathbf{Lab}_\star$ and r(x) for all x in $\mathbf{Var}_\star$.

The first step of the algorithm is to initialise the data structures. The second step is to build the graph and to perform the initial assignments to the data fields. This is established using the procedure add(q,d) that incorporates d into D[q] and adds q to the worklist if d was not part of D[q]. The third step is to continue propagating contributions along edges as long as the worklist is non-empty. The fourth and final step is to record the solution in a more familiar form.

Example 3.23 Let us consider how the algorithm operates on the expression $((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5$ of Example 3.20. After step 2 the data structure W has been initialised to

$$\mathsf{W} = [\mathsf{C}(4), \mathsf{C}(2)],$$

W	[C(4),C(2)]	[r(x),C(2)]	[C(1),C(2)]	[C(5),C(2)]	[C(2)]	[ ]
p	D[p]	D[p]	D[p]	D[p]	D[p]	D[p]
C(1)	$\emptyset$	$\emptyset$	id_y	id_y	id_y	id_y
C(2)	id_x	id_x	id_x	id_x	id_x	id_x
C(3)	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
C(4)	id_y	id_y	id_y	id_y	id_y	id_y
C(5)	$\emptyset$	$\emptyset$	$\emptyset$	id_y	id_y	id_y
r(x)	$\emptyset$	id_y	id_y	id_y	id_y	id_y
r(y)	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Figure 3.5: Iteration steps of example program.

and the data structures D and E have been initialised as in Figure 3.4 where we have written id_x for {`fn x => x`^{1}} and id_y for {`fn y => y`^{3}}. The algorithm will now iterate through the worklist and update the data structures W and D as described by step 3. The various intermediate stages are recorded in Figure 3.5. The algorithm computes the solution in the last column and this agrees with the solution presented in Example 3.5. ∎

The following result shows that the algorithm of Table 3.7 does indeed compute the solution we want:

Proposition 3.24

Given input $\mathcal{C}_\star[\![e_\star]\!]$ the algorithm of Table 3.7 terminates and the result $(\widehat{\mathsf{C}}, \widehat{\rho})$ produced by the algorithm satisfies

$$(\widehat{\mathsf{C}}, \widehat{\rho}) = \bigsqcap \{(\widehat{\mathsf{C}}', \widehat{\rho}') \mid (\widehat{\mathsf{C}}', \widehat{\rho}') \models_c \mathcal{C}_\star[\![e_\star]\!]\}$$

and hence it is the least solution to $\mathcal{C}_\star[\![e_\star]\!]$.

Proof It is immediate that steps 1, 2 and 4 terminate, and this leaves us with step 3. It is immediate that the values of D[q] never decrease and that they can be increased at most a finite number of times. It is also immediate that a node q is added to the worklist only if some value of D[q] actually increased. For each node placed on the worklist only a finite amount of calculation (bounded by the number of outgoing edges) needs to be performed in order to remove the node from the worklist. This guarantees termination.

Next let $(\widehat{\mathsf{C}}', \widehat{\rho}')$ be a solution to $(\widehat{\mathsf{C}}', \widehat{\rho}') \models_c \mathcal{C}_\star[\![e_\star]\!]$. It is possible to show that the following invariant

$$\forall \ell \in \mathbf{Lab}_\star : \mathsf{D}[\mathsf{C}(\ell)] \subseteq \widehat{\mathsf{C}}'(\ell)$$
$$\forall x \in \mathbf{Var}_\star : \mathsf{D}[\mathsf{r}(x)] \subseteq \widehat{\rho}'(x)$$

is maintained at all points after step 1. It follows that $(\widehat{\mathsf{C}}, \widehat{\rho}) \sqsubseteq (\widehat{\mathsf{C}}', \widehat{\rho}')$ upon completion of the algorithm.

We prove that $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}_\star[\![e_\star]\!]$ by contradiction. So suppose there exists $cc \in \mathcal{C}_\star[\![e_\star]\!]$ such that $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c cc$ does *not* hold. If cc is $\{t\} \subseteq p$ then step 2 ensures that $\{t\} \subseteq \mathsf{D}[p]$ and this is maintained throughout the algorithm; hence cc cannot have this form. If cc is $p_1 \subseteq p_2$ it must be the case that the final value of D satisfies $\mathsf{D}[p_1] \neq \emptyset$ since otherwise $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c cc$ would hold; now consider the last time $\mathsf{D}[p_1]$ was modified and note that p_1 was placed on the worklist at that time (by the procedure **add**); since the final worklist is empty we must have considered the constraint cc (which is in $\mathsf{E}[p_1]$) and updated $\mathsf{D}[p_2]$ accordingly; hence cc cannot have this form. If cc is $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$ it must be the case that the final value of D satisfies $\mathsf{D}[p] \neq \emptyset$ as well as $\mathsf{D}[p_1] \neq \emptyset$; now consider the last time one of $\mathsf{D}[p]$ and $\mathsf{D}[p_1]$ was modified and note that p or p_1 was placed on the worklist at that time; since the final worklist is empty we must have considered the constraint cc and updated $\mathsf{D}[p_2]$ accordingly; hence cc cannot have this form. Thus $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c cc$ for all $cc \in \mathcal{C}_\star[\![e_\star]\!]$.

We have now shown that $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}_\star[\![e_\star]\!]$ and that $(\widehat{\mathsf{C}}, \widehat{\rho}) \sqsubseteq (\widehat{\mathsf{C}}', \widehat{\rho}')$ whenever $(\widehat{\mathsf{C}}', \widehat{\rho}') \models_c \mathcal{C}_\star[\![e_\star]\!]$. It now follows that

$$(\widehat{\mathsf{C}}, \widehat{\rho}) = \bigsqcap \{(\widehat{\mathsf{C}}', \widehat{\rho}') \mid (\widehat{\mathsf{C}}', \widehat{\rho}') \models_c \mathcal{C}_\star[\![e_\star]\!]\}$$

as required. ∎

The proof showing that the algorithm terminates can be refined to show that it takes at most $O(n^3)$ steps if the original expression $e_\star$ has size n. To see this recall that $\mathcal{C}_\star[\![e_\star]\!]$ contains at most $O(n)$ constraints of the form $\{t\} \subseteq p$ or $p_1 \subseteq p_2$, and at most $O(n^2)$ constraints of the form $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$. We therefore know that the graph has at most $O(n)$ nodes and $O(n^2)$ edges and that each data field can be enlarged at most $O(n)$ times. Assuming that the operations upon $\mathsf{D}[p]$ take unit time we can perform the following calculations: step 1 takes time $O(n)$, step 2 takes time $O(n^2)$, and step 4 takes time $O(n)$; step 3 traverses each of the $O(n^2)$ edges at most $O(n)$ times and hence takes time $O(n^3)$; it follows that the overall algorithm takes no more than $O(n^3)$ basic steps.

Combining the three phases. From Proposition 3.24 we get that the pair $(\widehat{\mathsf{C}}, \widehat{\rho})$ computed by the algorithm of Table 3.7 is the least solution to $\mathcal{C}[\![e_\star]\!]$, so in particular $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}[\![e_\star]\!]$. Proposition 3.21 shows that a solution to the constraints will also be an acceptable analysis result for the syntax directed specification, hence $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star$. Proposition 3.18 shows that a solution that only involves program fragments of $e_\star$ and that is acceptable for the syntax directed specification, also is acceptable for the abstract specification, and therefore $(\widehat{\mathsf{C}}, \widehat{\rho}) \models e_\star$. Thus we have the following important corollary:

Corollary 3.25 Assume that $(\widehat{C}, \widehat{\rho})$ is the solution to the constraints $\mathcal{C}[\![e_\star]\!]$ computed by the algorithm of Table 3.7; then $(\widehat{C}, \widehat{\rho}) \models e_\star$. ∎

It is not the case that any $(\widehat{C}, \widehat{\rho})$ satisfying $(\widehat{C}, \widehat{\rho}) \models e_\star$ can be obtained using the above approach – see Exercise 3.11 and Mini Project 3.1.

For many applications it is the ability to compute the least $(\widehat{C}, \widehat{\rho})$ satisfying $(\widehat{C}, \widehat{\rho}) \models e_\star$ that is of primary interest, rather than the ability to check $(\widehat{C}, \widehat{\rho}) \models e_\star$ for a proposed guess $(\widehat{C}, \widehat{\rho})$. However, there are applications where it is the ability to check $(\widehat{C}, \widehat{\rho}) \models e_\star$ that is of primary concern. As an example consider an *open system* e_o that utilises resources of an unspecified library $e_\star$. Then analyses and optimisations of e_o must rely on $(\widehat{C}, \widehat{\rho})$ expressing all properties of interest about the library $e_\star$, i.e. $(\widehat{C}, \widehat{\rho}) \models e_\star$. This ensures that the unspecified library $e_\star$ can be replaced by another library $e'_\star$ as long as $(\widehat{C}, \widehat{\rho})$ continues to express all properties of interest about the library $e'_\star$, i.e. $(\widehat{C}, \widehat{\rho}) \models e'_\star$.

3.5 Adding Data Flow Analysis

In Section 3.1 we indicated that our Control Flow Analysis could be extended with Data Flow Analysis components. Basically, this amounts to extending the set $\widehat{\mathbf{Val}}$ to contain abstract values other than just abstractions. We shall first see how this can be done when the data flow component is a powerset and next we shall see how it can be generalised to complete lattices. We shall present the two approaches as abstract specifications only (in the manner of Section 3.1), leaving the syntax directed formulations to the exercises.

3.5.1 Abstract Values as Powersets

Abstract domains. There are several ways to extend the value domain $\widehat{\mathbf{Val}}$ so as to specify both Control Flow Analysis and Data Flow Analysis. A particularly simple approach is to use a set **Data** of *abstract data values* (i.e. abstract properties of booleans and integers) since this allows us to define:

$$\widehat{v} \in \widehat{\mathbf{Val}}_d = \mathcal{P}(\mathbf{Term} \cup \mathbf{Data}) \quad \text{abstract values}$$

For each constant $c \in \mathbf{Const}$ we need an element $d_c \in \mathbf{Data}$ specifying the abstract property of c. Similarly, for each operator $op \in \mathbf{Op}$ we need a total function

$$\widehat{\mathsf{op}} : \widehat{\mathbf{Val}}_d \times \widehat{\mathbf{Val}}_d \to \widehat{\mathbf{Val}}_d$$

telling how op operates on abstract properties. Typically, $\widehat{\mathsf{op}}$ will have a definition of the form

$$\widehat{v}_1 \ \widehat{\mathsf{op}} \ \widehat{v}_2 = \bigsqcup\{d_{op}(d_1, d_2) \mid d_1 \in \widehat{v}_1 \cap \mathbf{Data}, d_2 \in \widehat{v}_2 \cap \mathbf{Data}\}$$

for some function d_{op} : **Data** $\times$ **Data** $\rightarrow$ $\mathcal{P}$(**Data**) specifying how the operator computes with the abstract properties of integers and booleans.

Example 3.26 For a *Detection of Signs Analysis* we take $\mathbf{Data}_{\mathsf{sign}} = \{\mathsf{tt}, \mathsf{ff}, -, 0, +\}$ where tt and ff stand for the two truth values and -, 0, and + for the negative numbers, the number 0, and the positive numbers, respectively. It is then natural to define $d_{\mathtt{true}} = \mathsf{tt}$ and $d_7 = +$ and similarly for the other constants. Taking $\widehat{+}$ as an example, we can base its definition on the following table

d_+	tt	ff	-	0	+
tt	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
ff	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
-	$\emptyset$	$\emptyset$	{-}	{-}	{-, 0, +}
0	$\emptyset$	$\emptyset$	{-}	{0}	{+}
+	$\emptyset$	$\emptyset$	{-, 0, +}	{+}	{+}

and similarly for the other operators. ∎

Acceptability relation. The acceptability relation of the combined analysis has the form

$$(\widehat{C}, \widehat{\rho}) \models_d e$$

and is presented in Table 3.8. Compared with the analysis of Table 3.1 the clause [*con*] now records that d_c is a possible value of c and the clause [*op*] makes use of the function $\widehat{\mathsf{op}}$ described above. In the case of [*if*] we have made sure only to analyse those branches of the conditional that the analysis of the condition indicates the need for; hence we can be more precise than in the pure Control Flow Analysis – the Data Flow Analysis component of the analysis can influence the outcome of the Control Flow Analysis. In the manner of Exercise 3.3 similar improvements can be made to many of the clauses (see Exercise 3.14) and thereby the specification becomes more flow-sensitive.

Example 3.27 Consider the expression:

$$\begin{array}{l}(\mathtt{let\ f\ =\ (fn\ x\ =>\ (if\ (x^1\ >\ 0^2)^3\ then\ (fn\ y\ =>\ y^4)^5}\\ \qquad\qquad\qquad\qquad\mathtt{else\ (fn\ z\ =>\ 25^6)^7)^8)^9}\\ \mathtt{in\ ((f^{10}\ 3^{11})^{12}\ 0^{13})^{14})^{15}}\end{array}$$

A pure 0-CFA analysis will not be able to discover that the `else`-branch of the conditional will never be executed so it will conclude that the subterm with label 12 may evaluate to `fn y => y`^{4} as well as `fn z => 25`^{6} as shown in the first column of Figure 3.6. The second column of Figure 3.6 shows that when we combine the analysis with a Detection of Signs Analysis (outlined

$$[con] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models_d c^\ell \text{ iff } \{d_c\} \subseteq \widehat{\mathsf{C}}(\ell)$$

$$[var] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models_d x^\ell \text{ iff } \widehat{\rho}(x) \subseteq \widehat{\mathsf{C}}(\ell)$$

$$[fn] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (\texttt{fn } x \texttt{ => } e_0)^\ell \text{ iff } \{\texttt{fn } x \texttt{ => } e_0\} \subseteq \widehat{\mathsf{C}}(\ell)$$

$$[fun] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (\texttt{fun } f\ x \texttt{ => } e_0)^\ell \text{ iff } \{\texttt{fun } f\ x \texttt{ => } e_0\} \subseteq \widehat{\mathsf{C}}(\ell)$$

$$\begin{array}{ll}[app] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (t_1^{\ell_1}\ t_2^{\ell_2})^\ell \\ & \quad \text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_2^{\ell_2} \wedge \\ & \qquad (\forall(\texttt{fn } x \texttt{ => } t_0^{\ell_0}) \in \widehat{\mathsf{C}}(\ell_1) : \\ & \qquad\quad (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_0^{\ell_0} \wedge \\ & \qquad\quad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell)) \wedge \\ & \qquad (\forall(\texttt{fun } f\ x \texttt{ => } t_0^{\ell_0}) \in \widehat{\mathsf{C}}(\ell_1) : \\ & \qquad\quad (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_0^{\ell_0} \wedge \\ & \qquad\quad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell) \wedge \\ & \qquad\quad \{\texttt{fun } f\ x \texttt{ => } t_0^{\ell_0}\} \subseteq \widehat{\rho}(f))\end{array}$$

$$\begin{array}{ll}[if] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (\texttt{if } t_0^{\ell_0} \texttt{ then } t_1^{\ell_1} \texttt{ else } t_2^{\ell_2})^\ell \\ & \quad \text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_0^{\ell_0} \wedge \\ & \qquad (d_{\texttt{true}} \in \widehat{\mathsf{C}}(\ell_0) \Rightarrow ((\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_1^{\ell_1} \wedge \widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\mathsf{C}}(\ell))) \wedge \\ & \qquad (d_{\texttt{false}} \in \widehat{\mathsf{C}}(\ell_0) \Rightarrow ((\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_2^{\ell_2} \wedge \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell)))\end{array}$$

$$\begin{array}{ll}[let] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (\texttt{let } x \texttt{ = } t_1^{\ell_1} \texttt{ in } t_2^{\ell_2})^\ell \\ & \quad \text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_2^{\ell_2} \wedge \\ & \qquad \widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\rho}(x) \wedge \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell)\end{array}$$

$$\begin{array}{ll}[op] & (\widehat{\mathsf{C}},\widehat{\rho}) \models_d (t_1^{\ell_1}\ op\ t_2^{\ell_2})^\ell \\ & \quad \text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models_d t_2^{\ell_2} \wedge \\ & \qquad \widehat{\mathsf{C}}(\ell_1)\ \widehat{\mathsf{op}}\ \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell)\end{array}$$

Table 3.8: Abstract values as powersets.

in Example 3.26) then the analysis can determine that only `fn y => y`^{4} is a possible abstraction at label 12. Note that the Detection of Signs Analysis (correctly) determines that the expression will evaluate to a value with property $\{0\}$. ∎

The proof techniques introduced in Section 3.2 should suffice for proving the correctness of the analysis with respect to the operational semantics. A slight extension of the algorithmic techniques presented in Sections 3.3 and 3.4 (and in Mini Project 3.1) suffices for obtaining an implementation of the analysis provided that the set **Data** is finite.

	Section 3.1	Subsection 3.5.1	Subsection 3.5.2	
	$(\widehat{\mathsf{C}},\widehat{\rho})$	$(\widehat{\mathsf{C}},\widehat{\rho})$	$(\widehat{\mathsf{C}},\widehat{\rho})$	$(\widehat{\mathsf{D}},\widehat{\delta})$
1	$\emptyset$	{+}	$\emptyset$	{+}
2	$\emptyset$	{0}	$\emptyset$	{0}
3	$\emptyset$	{tt}	$\emptyset$	{tt}
4	$\emptyset$	{0}	$\emptyset$	{0}
5	{fn y => y^4}	{fn y => y^4}	{fn y => y^4}	$\emptyset$
6	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
7	{fn z => 25^6}	$\emptyset$	$\emptyset$	$\emptyset$
8	{fn y => y^4, fn z => 25^6}	{fn y => y^4}	{fn y => y^4}	$\emptyset$
9	{fn x => $(\cdots)^8$}	{fn x => $(\cdots)^8$}	{fn x => $(\cdots)^8$}	$\emptyset$
10	{fn x => $(\cdots)^8$}	{fn x => $(\cdots)^8$}	{fn x => $(\cdots)^8$}	$\emptyset$
11	$\emptyset$	{+}	$\emptyset$	{+}
12	{fn y => y^4, fn z => 25^6}	{fn y => y^4}	{fn y => y^4}	$\emptyset$
13	$\emptyset$	{0}	$\emptyset$	{0}
14	$\emptyset$	{0}	$\emptyset$	{0}
15	$\emptyset$	{0}	$\emptyset$	{0}
f	{fn x => $(\cdots)^8$}	{fn x => $(\cdots)^8$}	{fn x => $(\cdots)^8$}	$\emptyset$
x	$\emptyset$	{+}	$\emptyset$	{+}
y	$\emptyset$	{0}	$\emptyset$	{0}
z	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Figure 3.6: Control Flow and Data Flow Analysis for example program.

Finally, we should stress that a solution to the analysis of Table 3.8 does *not* immediately give a solution to the analysis of Table 3.1. More precisely, $(\widehat{\mathsf{C}},\widehat{\rho}) \models_d e$ does *not* guarantee that $(\widehat{\mathsf{C}}',\widehat{\rho}') \models e$ where $\forall \ell : \widehat{\mathsf{C}}'(\ell) = \widehat{\mathsf{C}}(\ell) \cap \mathbf{Term}$ and $\forall x : \widehat{\rho}'(x) = \widehat{\rho}(x) \cap \mathbf{Term}$. The reason is that the Control Flow Analysis part of Table 3.8 is influenced by the Data Flow Analysis part in the clause [*if*]: if for example the abstract value of the condition does not include $d_{\mathtt{true}}$ then the **then**-branch will not be analysed.

3.5.2 Abstract Values as Complete Lattices

Abstract domains. Clearly $\widehat{\mathbf{Val}}_d = \mathcal{P}(\mathbf{Term} \cup \mathbf{Data})$ is isomorphic to $\mathcal{P}(\mathbf{Term}) \times \mathcal{P}(\mathbf{Data})$. This suggests that the abstract cache $\widehat{\mathsf{C}} : \mathbf{Lab} \to \widehat{\mathbf{Val}}_d$

could be split into a term component and a data component and similarly for the abstract environment $\widehat{\rho}$: **Var** $\rightarrow \widehat{\mathbf{Val}}_d$.

Having decoupled $\mathcal{P}$(**Term**) and $\mathcal{P}$(**Data**) we can now consider replacing $\mathcal{P}$(**Data**) by a more general collection of properties. An obvious possibility is to replace $\mathcal{P}$(**Data**) by a complete lattice L and perform a development closely related to that of the (forward) Monotone Frameworks of Chapter 2.

So let us define a *monotone structure* to consist of:

- a complete lattice L, and
- a set $\mathcal{F}$ of monotone functions of $L \times L \rightarrow L$.

An *instance* of a monotone structure then consists of the structure $(L, \mathcal{F})$ and

- a mapping ι. from the constants $c \in$ **Const** to values in L, and
- a mapping f. from the binary operators $op \in$ **Op** to functions of $\mathcal{F}$.

Compared with the instances of the Monotone Frameworks of Section 2.3 we omit the flow component since it will be the responsibility of the Control Flow Analysis to determine this. The component ι has been replaced by the mapping ι. giving the *extremal value* for all the constants and the component f. mapping labels to transfer functions has been replaced by a mapping of the binary operators to their interpretation.

Example 3.28 A monotone structure corresponding to the development of Subsection 3.5.1 will have L to be $\mathcal{P}$(**Data**) and $\mathcal{F}$ to be the monotone functions of $\mathcal{P}$(**Data**) $\times$ $\mathcal{P}$(**Data**) $\rightarrow$ $\mathcal{P}$(**Data**).

An instance of the monotone structure is then obtained by taking

$$\iota_c = \{d_c\}$$

for all constants c (and with $d_c \in$ **Data** as above) and

$$f_{op}(l_1, l_2) = \bigcup\{d_{op}(d_1, d_2) \mid d_1 \in l_1, d_2 \in l_2\}$$

for all binary operators op (and where d_{op} : **Data** $\times$ **Data** $\rightarrow$ $\mathcal{P}$(**Data**) is as above). ∎

Example 3.29 A monotone structure for *Constant Propagation Analysis* will have L to be $\mathbf{Z}_{\perp}^{\top} \times \mathcal{P}(\{\mathrm{tt}, \mathrm{ff}\})$ and $\mathcal{F}$ to be the monotone functions of $L \times L \rightarrow L$.

An instance of the monotone structure is obtained by taking e.g. $\iota_7 = (7, \emptyset)$ and $\iota_{\mathbf{true}} = (\bot, \{\mathbf{tt}\})$. For a binary operator such as + we can take:

$$f_+(l_1, l_2) = \begin{cases} (z_1 + z_2, \emptyset) & \text{if } l_1 = (z_1, \cdots), l_2 = (z_2, \cdots), \\ & \text{and } z_1, z_2 \in \mathbf{Z} \\ (\bot, \emptyset) & \text{if } l_1 = (z_1, \cdots), l_2 = (z_2, \cdots), \\ & \text{and } z_1 = \bot \text{ or } z_2 = \bot \\ (\top, \emptyset) & \text{otherwise} \end{cases}$$

■

We can now define the following abstract domains

$\widehat{v}$	$\in$	$\widehat{\mathbf{Val}}$	$= \mathcal{P}(\mathbf{Term})$	abstract values
$\widehat{\rho}$	$\in$	$\widehat{\mathbf{Env}}$	$= \mathbf{Var} \to \widehat{\mathbf{Val}}$	abstract environments
$\widehat{\mathsf{C}}$	$\in$	$\widehat{\mathbf{Cache}}$	$= \mathbf{Lab} \to \widehat{\mathbf{Val}}$	abstract caches

to take care of the Control Flow Analysis and furthermore

$\widehat{d}$	$\in$	$\widehat{\mathbf{Data}}$	$= L$	abstract data values
$\widehat{\delta}$	$\in$	$\widehat{\mathbf{DEnv}}$	$= \mathbf{Var} \to \widehat{\mathbf{Data}}$	abstract data environments
$\widehat{\mathsf{D}}$	$\in$	$\widehat{\mathbf{DCache}}$	$= \mathbf{Lab} \to \widehat{\mathbf{Data}}$	abstract data caches

to take care of the Data Flow Analysis.

Acceptability relation. The acceptability relation now has the form

$$(\widehat{\mathsf{C}}, \widehat{\mathsf{D}}, \widehat{\rho}, \widehat{\delta}) \models_D e$$

and it is defined by the clauses of Table 3.9. In the clause [*con*] we see that the $\iota_\cdot$ component of the instance is used to restrict the value of the $\widehat{\mathsf{D}}$ component of the analysis and in the clause [*op*] we see how the $f_\cdot$ component is used. The clause [*if*] has explicit tests for the two branches as in the previous approach thereby allowing the Control Flow Analysis to benefit from results obtained by the Data Flow Analysis component. As in the previous subsection, similar improvements can be made to many of the other clauses so as to produce a more flow-sensitive analysis.

Example 3.30 Returning to the expression of Example 3.27 and the Detection of Signs Analysis we now get the analysis result of the last column of Figure 3.6. So we see that the result is as before. ■

The proof techniques introduced in Section 3.2 should suffice for proving the correctness of the analysis with respect to the operational semantics. A slight extension of the algorithmic techniques presented in Sections 3.3 and 3.4 (and in Mini Project 3.1) suffices for obtaining an implementation of the analysis provided that L satisfies the Ascending Chain Condition (as is the case for Monotone Frameworks).

$$[con] \quad (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D c^{\ell} \text{ iff } \iota_c \sqsubseteq \widehat{\mathsf{D}}(\ell)$$

$$[var] \quad (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D x^{\ell} \text{ iff } \widehat{\rho}(x) \subseteq \widehat{\mathsf{C}}(\ell) \ \wedge \ \widehat{\delta}(x) \sqsubseteq \widehat{\mathsf{D}}(\ell)$$

$$[fn] \quad (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D (\texttt{fn}\ x \texttt{ => } e_0)^{\ell} \text{ iff } \{\texttt{fn}\ x \texttt{ => } e_0\} \subseteq \widehat{\mathsf{C}}(\ell)$$

$$[fun] \quad (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D (\texttt{fun}\ f\ x \texttt{ => } e_0)^{\ell} \text{ iff } \{\texttt{fun}\ f\ x \texttt{ => } e_0\} \subseteq \widehat{\mathsf{C}}(\ell)$$

$$\begin{array}{ll}[app] & (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D (t_1^{\ell_1}\ t_2^{\ell_2})^{\ell} \\ & \quad \text{iff} \quad (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_1^{\ell_1} \ \wedge \ (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_2^{\ell_2} \ \wedge \\ & \qquad (\forall(\texttt{fn}\ x \texttt{ => } t_0^{\ell_0}) \in \widehat{\mathsf{C}}(\ell_1) : \\ & \qquad\qquad (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_0^{\ell_0} \ \wedge \\ & \qquad\qquad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x) \ \wedge \ \widehat{\mathsf{D}}(\ell_2) \sqsubseteq \widehat{\delta}(x) \ \wedge \\ & \qquad\qquad \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell) \ \wedge \ \widehat{\mathsf{D}}(\ell_0) \sqsubseteq \widehat{\mathsf{D}}(\ell)) \ \wedge \\ & \qquad (\forall(\texttt{fun}\ f\ x \texttt{ => } t_0^{\ell_0}) \in \widehat{\mathsf{C}}(\ell_1) : \\ & \qquad\qquad (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_0^{\ell_0} \ \wedge \\ & \qquad\qquad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\rho}(x) \ \wedge \ \widehat{\mathsf{D}}(\ell_2) \sqsubseteq \widehat{\delta}(x) \ \wedge \\ & \qquad\qquad \widehat{\mathsf{C}}(\ell_0) \subseteq \widehat{\mathsf{C}}(\ell) \ \wedge \ \widehat{\mathsf{D}}(\ell_0) \sqsubseteq \widehat{\mathsf{D}}(\ell) \ \wedge \\ & \qquad\qquad \{\texttt{fun}\ f\ x \texttt{ => } t_0^{\ell_0}\} \subseteq \widehat{\rho}(f)) \end{array}$$

$$\begin{array}{ll}[if] & (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D (\texttt{if}\ t_0^{\ell_0}\ \texttt{then}\ t_1^{\ell_1}\ \texttt{else}\ t_2^{\ell_2})^{\ell} \\ & \quad \text{iff} \quad (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_0^{\ell_0} \ \wedge \\ & \qquad (\iota_{\texttt{true}} \sqsubseteq \widehat{\mathsf{D}}(\ell_0) \Rightarrow (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_1^{\ell_1} \ \wedge \\ & \qquad\qquad\qquad\qquad \widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\mathsf{C}}(\ell) \wedge \\ & \qquad\qquad\qquad\qquad \widehat{\mathsf{D}}(\ell_1) \sqsubseteq \widehat{\mathsf{D}}(\ell)) \ \wedge \\ & \qquad (\iota_{\texttt{false}} \sqsubseteq \widehat{\mathsf{D}}(\ell_0) \Rightarrow (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_2^{\ell_2} \ \wedge \\ & \qquad\qquad\qquad\qquad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell) \wedge \\ & \qquad\qquad\qquad\qquad \widehat{\mathsf{D}}(\ell_2) \sqsubseteq \widehat{\mathsf{D}}(\ell)) \end{array}$$

$$\begin{array}{ll}[let] & (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D (\texttt{let}\ x = t_1^{\ell_1}\ \texttt{in}\ t_2^{\ell_2})^{\ell} \\ & \quad \text{iff} \quad (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_1^{\ell_1} \ \wedge \ (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_2^{\ell_2} \ \wedge \\ & \qquad \widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\rho}(x) \wedge \widehat{\mathsf{D}}(\ell_1) \sqsubseteq \widehat{\delta}(x) \ \wedge \\ & \qquad \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell) \wedge \widehat{\mathsf{D}}(\ell_2) \sqsubseteq \widehat{\mathsf{D}}(\ell) \end{array}$$

$$\begin{array}{ll}[op] & (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D (t_1^{\ell_1}\ op\ t_2^{\ell_2})^{\ell} \\ & \quad \text{iff} \quad (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_1^{\ell_1} \ \wedge \ (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models_D t_2^{\ell_2} \ \wedge \\ & \qquad f_{op}(\widehat{\mathsf{D}}(\ell_1),\widehat{\mathsf{D}}(\ell_2)) \sqsubseteq \widehat{\mathsf{D}}(\ell) \end{array}$$

Table 3.9: Abstract values as complete lattices.

Staging the specification. Let us briefly consider the following alternative clause for [*if*] where the data flow component *cannot* influence the control flow component because we always make sure that the analysis result is acceptable for both branches:

$$\begin{array}{rl}(\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models'_D (\texttt{if } t_0^{\ell_0} \texttt{ then } t_1^{\ell_1} \texttt{ else } t_2^{\ell_2})^{\ell} & \\ \quad \text{iff} & (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models'_D t_0^{\ell_0} \wedge \\ & (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models'_D t_1^{\ell_1} \wedge \widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\mathsf{C}}(\ell) \wedge \widehat{\mathsf{D}}(\ell_1) \sqsubseteq \widehat{\mathsf{D}}(\ell) \wedge \\ & (\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models'_D t_2^{\ell_2} \wedge \widehat{\mathsf{C}}(\ell_2) \subseteq \widehat{\mathsf{C}}(\ell) \wedge \widehat{\mathsf{D}}(\ell_2) \sqsubseteq \widehat{\mathsf{D}}(\ell)\end{array}$$

Unlike what was the case for the analyses of Tables 3.8 and 3.9, a solution to the analysis modified in this way *does* give rise to a solution to the analysis of Table 3.1; to be precise $(\widehat{\mathsf{C}},\widehat{\mathsf{D}},\widehat{\rho},\widehat{\delta}) \models'_D e$ guarantees $(\widehat{\mathsf{C}},\widehat{\rho}) \models e$.

In terms of implementation this modification means that the constraints for the control flow component ($\widehat{\mathsf{C}}$ and $\widehat{\rho}$) can be solved first, and based on this the constraints for the data flow component ($\widehat{\mathsf{D}}$ and $\widehat{\delta}$) can be solved next. If both sets of constraints are solved for their least solution this will still yield the least solution of the combined set of constraints.

Example 3.31 Let us return to Example 3.30. If we modify the clause for [*if*] as discussed above then the resulting analysis will have $\widehat{\mathsf{C}}$ and $\widehat{\rho}$ as in the column for the pure analysis from Section 3.1 and $\widehat{\mathsf{D}}$ and $\widehat{\delta}$ will associate slightly larger sets with some of the labels and variables:

$$\begin{array}{rcl}\widehat{\mathsf{D}}(6) & = & \{+\} \\ \widehat{\mathsf{D}}(14) & = & \{0,+\} \\ \widehat{\mathsf{D}}(15) & = & \{0,+\} \\ \widehat{\delta}(\mathsf{z}) & = & \{0\}\end{array}$$

This analysis is less precise than those of Tables 3.8 and 3.9: it will only determine that the expression will evaluate to a value with the property $\{0,+\}$. ■

Imperative constructs and data structures. Mini Project 3.2 shows one way of extending the development to track creation points of data structures. Mini Project 3.4 shows how to deal with imperative constructs in the manner of the imperative language WHILE of Chapter 2.

3.6 Adding Context Information

The Control Flow Analyses presented so far are imprecise in that they cannot distinguish the various instances of function calls from one another. In the

terminology of Section 2.5 the 0-CFA analysis is *context-insensitive* and in the terminology of Control Flow Analysis it is *monovariant.*

Example 3.32 Consider the expression:

$$\begin{array}{l}(\texttt{let f = (fn x => x}^1)^2 \\ \texttt{ in ((f}^3\ \texttt{f}^4)^5\ (\texttt{fn y => y}^6)^7)^8)^9\end{array}$$

The least 0-CFA analysis is given by $(\widehat{C}_{id}, \widehat{\rho}_{id})$:

$$\begin{array}{lcl}
\widehat{C}_{id}(1) & = & \{\texttt{fn x => x}^1, \texttt{fn y => y}^6\} \\
\widehat{C}_{id}(2) & = & \{\texttt{fn x => x}^1\} \\
\widehat{C}_{id}(3) & = & \{\texttt{fn x => x}^1\} \\
\widehat{C}_{id}(4) & = & \{\texttt{fn x => x}^1\} \\
\widehat{C}_{id}(5) & = & \{\texttt{fn x => x}^1, \texttt{fn y => y}^6\} \\
\widehat{C}_{id}(6) & = & \{\texttt{fn y => y}^6\} \\
\widehat{C}_{id}(7) & = & \{\texttt{fn y => y}^6\} \\
\widehat{C}_{id}(8) & = & \{\texttt{fn x => x}^1, \texttt{fn y => y}^6\} \\
\widehat{C}_{id}(9) & = & \{\texttt{fn x => x}^1, \texttt{fn y => y}^6\} \\
\widehat{\rho}_{id}(\texttt{f}) & = & \{\texttt{fn x => x}^1\} \\
\widehat{\rho}_{id}(\texttt{x}) & = & \{\texttt{fn x => x}^1, \texttt{fn y => y}^6\} \\
\widehat{\rho}_{id}(\texttt{y}) & = & \{\texttt{fn y => y}^6\}
\end{array}$$

So we see that `x` can be bound to $\texttt{fn x => x}^1$ as well as $\texttt{fn y => y}^6$ and hence the overall expression (label 9) may evaluate to either of these two abstractions. However, it is easy to see that in fact only $\texttt{fn y => y}^6$ is a possible result. ■

To get a more precise analysis it is useful to introduce a mechanism that distinguishes different dynamic instances of variables and labels from one another. This results in a *context-sensitive* analysis and in the terminology of Control Flow Analysis the term *polyvariant* is used. There are several approaches to how this can be done. One simple possibility is to expand the program such that the problem does not arise.

Example 3.33 For the expression of Example 3.32 we could for example consider

```
let f1 = (fn x1 => x1)
in let f2 = (fn x2 => x2)
   in (f1 f2) (fn y => y)
```

and then analyse the expanded expression: the 0-CFA analysis is now able to deduce that `x1` can only be bound to `fn x2 => x2` and that `x2` can only be bound to `fn y => y` so the overall expression will evaluate to `fn y => y` only. ∎

A more satisfactory solution to the problem is to extend the analysis with *context information* allowing it to distinguish between the various instances of variables and program points and still analyse the original expression. Examples of such analyses include k-CFA analyses, uniform k-CFA analyses, polynomial k-CFA analyses (mainly of interest for $k > 0$) and the Cartesian Product Algorithm.

3.6.1 Uniform k-CFA Analysis

Abstract domains. A key idea is to introduce context to distinguish between the various dynamic instances of variables and program points. There are many choices concerning how to model contexts and how they can be modified in the course of the analysis. In a *uniform k-CFA* analysis (as well as in a k-CFA analysis) a context δ records the last k dynamic call points; hence in this case contexts will be sequences of labels of length at most k and they will be updated whenever a function application is analysed. This is modelled by taking:

$$\delta \in \Delta = \mathbf{Lab}^{\leq k} \quad \text{context information}$$

Since the contexts will be used to distinguish between the various instances of the variables we will need a *context environment* to determine the context associated with the current instance of a variable:

$$ce \in \mathbf{CEnv} = \mathbf{Var} \to \Delta \quad \text{context environments}$$

The context environment will play a role similar to the environment of the semantics; in particular, this means that we shall extend the *abstract values* to contain a context environment:

$$\widehat{v} \in \widehat{\mathbf{Val}} = \mathcal{P}(\mathbf{Term} \times \mathbf{CEnv}) \quad \text{abstract values}$$

So in addition to recording the abstractions (`fn` x `=>` e and `fun` f x `=>` e) we will also record the context environment at the *definition point* for the free variables of the term. This should be compared with the Structural Operational Semantics of Section 3.2 where the closures contain information about the abstraction as well as the environment determining the values of the free variables at the definition point.

The *abstract environment* $\widehat{\rho}$ will now map a variable and a context to an abstract value:

$$\widehat{\rho} \in \widehat{\mathbf{Env}} = (\mathbf{Var} \times \Delta) \to \widehat{\mathbf{Val}} \quad \text{abstract environments}$$

$$[con] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} c^{\ell} \text{ always}$$

$$[var] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} x^{\ell} \text{ iff } \widehat{\rho}(x, ce(x)) \subseteq \widehat{\mathsf{C}}(\ell,\delta)$$

$$[fn] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} (\texttt{fn}\ x\ \texttt{=>}\ e_0)^{\ell} \text{ iff } \{(\texttt{fn}\ x\ \texttt{=>}\ e_0, ce_0)\} \subseteq \widehat{\mathsf{C}}(\ell,\delta)$$
$$\text{where } ce_0 = ce \mid FV(\texttt{fn}\ x\ \texttt{=>}\ e_0)$$

$$[fun] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} (\texttt{fun}\ f\ x\ \texttt{=>}\ e_0)^{\ell} \text{ iff } \{(\texttt{fun}\ f\ x\ \texttt{=>}\ e_0, ce_0)\} \subseteq \widehat{\mathsf{C}}(\ell,\delta)$$
$$\text{where } ce_0 = ce \mid FV(\texttt{fun}\ f\ x\ \texttt{=>}\ e_0)$$

$$[app] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} (t_1^{\ell_1}\ t_2^{\ell_2})^{\ell}$$
$$\text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} t_2^{\ell_2} \wedge$$
$$(\forall(\texttt{fn}\ x\ \texttt{=>}\ t_0^{\ell_0}, ce_0) \in \widehat{\mathsf{C}}(\ell_1,\delta):$$
$$(\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce'_0}_{\delta_0} t_0^{\ell_0} \wedge$$
$$\widehat{\mathsf{C}}(\ell_2,\delta) \subseteq \widehat{\rho}(x,\delta_0) \wedge \widehat{\mathsf{C}}(\ell_0,\delta_0) \subseteq \widehat{\mathsf{C}}(\ell,\delta)$$
$$\text{where } \delta_0 = \lceil \delta, \ell \rceil_k$$
$$\text{and } ce'_0 = ce_0[x \mapsto \delta_0]) \wedge$$
$$(\forall(\texttt{fun}\ f\ x\ \texttt{=>}\ t_0^{\ell_0}, ce_0) \in \widehat{\mathsf{C}}(\ell_1,\delta):$$
$$(\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce'_0}_{\delta_0} t_0^{\ell_0} \wedge$$
$$\widehat{\mathsf{C}}(\ell_2,\delta) \subseteq \widehat{\rho}(x,\delta_0) \wedge \widehat{\mathsf{C}}(\ell_0,\delta_0) \subseteq \widehat{\mathsf{C}}(\ell,\delta) \wedge$$
$$\{(\texttt{fun}\ f\ x\ \texttt{=>}\ t_0^{\ell_0}, ce_0)\} \subseteq \widehat{\rho}(f,\delta_0)$$
$$\text{where } \delta_0 = \lceil \delta, \ell \rceil_k$$
$$\text{and } ce'_0 = ce_0[f \mapsto \delta_0, x \mapsto \delta_0])$$

$$[if] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} (\texttt{if}\ t_0^{\ell_0}\ \texttt{then}\ t_1^{\ell_1}\ \texttt{else}\ t_2^{\ell_2})^{\ell}$$
$$\text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} t_0^{\ell_0} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} t_2^{\ell_2} \wedge$$
$$\widehat{\mathsf{C}}(\ell_1,\delta) \subseteq \widehat{\mathsf{C}}(\ell,\delta) \wedge \widehat{\mathsf{C}}(\ell_2,\delta) \subseteq \widehat{\mathsf{C}}(\ell,\delta)$$

$$[let] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} (\texttt{let}\ x = t_1^{\ell_1}\ \texttt{in}\ t_2^{\ell_2})^{\ell}$$
$$\text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce'}_{\delta} t_2^{\ell_2} \wedge$$
$$\widehat{\mathsf{C}}(\ell_1,\delta) \subseteq \widehat{\rho}(x,\delta) \wedge \widehat{\mathsf{C}}(\ell_2,\delta) \subseteq \widehat{\mathsf{C}}(\ell,\delta)$$
$$\text{where } ce' = ce[x \mapsto \delta]$$

$$[op] \quad (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} (t_1^{\ell_1}\ op\ t_2^{\ell_2})^{\ell} \text{ iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} t_1^{\ell_1} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models^{ce}_{\delta} t_2^{\ell_2}$$

Table 3.10: Uniform k-CFA analysis.

Typically we will use a context environment to find the context associated with the variable of interest and then use it together with the variable to access the abstract environment. This means that indirectly we get the effect of having local abstract environments in the abstract values although $\widehat{\rho}$ is still a global entity as in the previous sections.

The uniform k-CFA analysis differs from the k-CFA analysis in performing a similar development for the *abstract cache* that now maps a label and a context to an abstract value:

$$\widehat{\mathsf{C}} \in \widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \Delta) \to \widehat{\mathbf{Val}} \quad \text{abstract caches}$$

Given information about the context of interest we can determine the abstract value associated with a label. Again we indirectly get the effect of having a cache for each possible context although it is still a global entity. (In k-CFA one has $\widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \mathbf{CEnv}) \to \widehat{\mathbf{Val}}$.)

Acceptability relation. The acceptability relation for *uniform k-CFA* is presented in Table 3.10. It is defined by formulae of the form

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models^{ce}_{\delta} e$$

where ce is the current context environment and δ is the current context. The formula expresses that $(\widehat{\mathsf{C}}, \widehat{\rho})$ is an acceptable analysis of e in the *context* specified by ce and δ. The clauses for the various constructs are very much as those in Table 3.1 and will be explained below.

In the clause [*var*] we use the current context environment ce to determine the context $ce(x)$ of the current instance of the variable x and then the abstract value of the variable is given by $\widehat{\rho}(x, ce(x))$. The current context is δ so we have to ensure that $\widehat{\rho}(x, ce(x)) \subseteq \widehat{\mathsf{C}}(\ell, \delta)$.

In the clause [*fn*] we record the current context environment as part of the abstract value and (as in the Structural Operational Semantics of Table 3.2) we restrict the context environment to the set of variables of interest for the abstraction. The clause [*fun*] is similar.

In the clause [*app*] we analyse the two subexpressions using the same context and context environment as the composite expression. When we find a potential abstract value, say $(\texttt{fn}\ x \texttt{=>}\ t_0^{\ell_0}, ce_0)$, that the operator may evaluate to, it will contain a local context environment ce_0 that was created at its definition point. When analysing $t_0^{\ell_0}$ we will have passed through the application point ℓ, so the current context will be updated to include ℓ and this will also be the context associated with the variable x in the updated version of the context environment ce_0 used for the analysis of $t_0^{\ell_0}$. The new context is $\lceil \delta, \ell \rceil_k$ which (as in Section 2.5) denotes the sequence $[\delta, \ell]$ but possibly truncated (by omitting elements on the left) so as have length at most k. In the case where the operator has the form $(\texttt{fun}\ f\ x \texttt{=>}\ t_0^{\ell_0}, ce_0)$ we proceed in a similar way and note that f as well as x will be associated with the new context in the analysis of the body of the function.

The clauses for [*if*], [*let*] and [*op*] are fairly straightforward modifications of those of Table 3.1; however, note that the context of the bound variable of the `let`-construct is the current context (as no application point is passed). We

shall dispense with proving the correctness of the analysis and with showing how it can be implemented.

Example 3.34 We shall now specify a uniform 1-CFA analysis for the expression of Example 3.33:

$$(\texttt{let f = (fn x => x}^1)^2 \texttt{ in } ((\texttt{f}^3\ \texttt{f}^4)^5\ (\texttt{fn y => y}^6)^7)^8)^9$$

The initial context will be Λ, the empty sequence of labels. In the course of the analysis the current context will be modified at the two application points with labels 5 and 8; since we only records call strings of length at most one the only contexts of interest will therefore be Λ, 5 and 8. There are four context environments of interest:

$\mathsf{ce}_0 = [\,]$	the initial (empty) context environment,
$\mathsf{ce}_1 = \mathsf{ce}_0[\texttt{f} \mapsto \Lambda]$	the context environment for the analysis of the body of the `let`-construct,
$\mathsf{ce}_2 = \mathsf{ce}_0[\texttt{x} \mapsto 5]$	the context environment used for the analysis of the body of `f` initiated at the application point 5, and
$\mathsf{ce}_3 = \mathsf{ce}_0[\texttt{x} \mapsto 8]$	the context environment used for the analysis of the body of `f` initiated at the application point 8.

Let us take $\widehat{\mathsf{C}}'_{\mathsf{id}}$ and $\widehat{\rho}'_{\mathsf{id}}$ to be:

$$\begin{array}{ll}
\widehat{\mathsf{C}}'_{\mathsf{id}}(1,5) = \{(\texttt{fn x => x}^1,\mathsf{ce}_0)\} & \widehat{\mathsf{C}}'_{\mathsf{id}}(1,8) = \{(\texttt{fn y => y}^6,\mathsf{ce}_0)\} \\
\widehat{\mathsf{C}}'_{\mathsf{id}}(2,\Lambda) = \{(\texttt{fn x => x}^1,\mathsf{ce}_0)\} & \widehat{\mathsf{C}}'_{\mathsf{id}}(3,\Lambda) = \{(\texttt{fn x => x}^1,\mathsf{ce}_0)\} \\
\widehat{\mathsf{C}}'_{\mathsf{id}}(4,\Lambda) = \{(\texttt{fn x => x}^1,\mathsf{ce}_0)\} & \widehat{\mathsf{C}}'_{\mathsf{id}}(5,\Lambda) = \{(\texttt{fn x => x}^1,\mathsf{ce}_0)\} \\
\widehat{\mathsf{C}}'_{\mathsf{id}}(7,\Lambda) = \{(\texttt{fn y => y}^6,\mathsf{ce}_0)\} & \widehat{\mathsf{C}}'_{\mathsf{id}}(8,\Lambda) = \{(\texttt{fn y => y}^6,\mathsf{ce}_0)\} \\
\widehat{\mathsf{C}}'_{\mathsf{id}}(9,\Lambda) = \{(\texttt{fn y => y}^6,\mathsf{ce}_0)\} & \\
\widehat{\rho}'_{\mathsf{id}}(\texttt{f},\Lambda) = \{(\texttt{fn x => x}^1,\mathsf{ce}_0)\} & \\
\widehat{\rho}'_{\mathsf{id}}(\texttt{x},5) = \{(\texttt{fn x => x}^1,\mathsf{ce}_0)\} & \widehat{\rho}'_{\mathsf{id}}(\texttt{x},8) = \{(\texttt{fn y => y}^6,\mathsf{ce}_0)\}
\end{array}$$

We shall now show that this is an acceptable analysis result for the example expression:

$$(\widehat{\mathsf{C}}'_{\mathsf{id}}, \widehat{\rho}'_{\mathsf{id}}) \models^{\mathsf{ce}_0}_{\Lambda} (\texttt{let f = (fn x => x}^1)^2 \texttt{ in } ((\texttt{f}^3\ \texttt{f}^4)^5\ (\texttt{fn y => y}^6)^7)^8)^9$$

According to clause [*let*], it is sufficient to verify that

$$(\widehat{\mathsf{C}}'_{\mathsf{id}}, \widehat{\rho}'_{\mathsf{id}}) \models^{\mathsf{ce}_0}_{\Lambda} (\texttt{fn x => x}^1)^2$$
$$(\widehat{\mathsf{C}}'_{\mathsf{id}}, \widehat{\rho}'_{\mathsf{id}}) \models^{\mathsf{ce}_1}_{\Lambda} ((\texttt{f}^3\ \texttt{f}^4)^5\ (\texttt{fn y => y}^6)^7)^8$$

because $\widehat{\mathsf{C}}'_{\mathsf{id}}(2,\Lambda) \subseteq \widehat{\rho}'_{\mathsf{id}}(\texttt{f},\Lambda)$ and $\widehat{\mathsf{C}}'_{\mathsf{id}}(8,\Lambda) \subseteq \widehat{\mathsf{C}}'_{\mathsf{id}}(9,\Lambda)$. This is straightforward except for the last clause. Since $\widehat{\mathsf{C}}'_{\mathsf{id}}(5,\Lambda) = \{(\texttt{fn x => x}^1,\mathsf{ce}_0)\}$ it is, according to [*app*], sufficient to verify that

$$(\widehat{C}'_{id}, \widehat{\rho}'_{id}) \models^{ce_1}_{\Lambda} (\mathtt{f}^3\ \mathtt{f}^4)^5$$
$$(\widehat{C}'_{id}, \widehat{\rho}'_{id}) \models^{ce_1}_{\Lambda} (\mathtt{fn\ y\ =>\ y}^6)^7$$
$$(\widehat{C}'_{id}, \widehat{\rho}'_{id}) \models^{ce_3}_{8} \mathtt{x}^1$$

because $\widehat{C}'_{id}(7,\Lambda) \subseteq \widehat{\rho}'_{id}(\mathtt{x},8)$ and $\widehat{C}'_{id}(1,8) \subseteq \widehat{C}'_{id}(8,\Lambda)$. This is straightforward except for the first clause. Proceeding as above we see that $\widehat{C}'_{id}(3,\Lambda) = \{(\mathtt{fn\ x\ =>\ x}^1, ce_0)\}$ and it is sufficient to verify

$$(\widehat{C}'_{id}, \widehat{\rho}'_{id}) \models^{ce_1}_{\Lambda} \mathtt{f}^3$$
$$(\widehat{C}'_{id}, \widehat{\rho}'_{id}) \models^{ce_1}_{\Lambda} \mathtt{f}^4$$
$$(\widehat{C}'_{id}, \widehat{\rho}'_{id}) \models^{ce_2}_{5} \mathtt{x}^1$$

because $\widehat{C}'_{id}(4,\Lambda) \subseteq \widehat{\rho}'_{id}(\mathtt{x},5)$ and $\widehat{C}'_{id}(1,5) \subseteq \widehat{C}'_{id}(5,\Lambda)$. This is straightforward.

The importance of this example is that it shows that the uniform 1-CFA analysis is strong enough to determine that $\mathtt{fn\ y\ =>\ y}^6$ is the *only* result possible for the overall expression unlike what was the case for the 0-CFA analysis in Example 3.32. We can also see that, since $\widehat{\rho}'_{id}(\mathtt{y},\delta) = \emptyset$ for all $\delta \in \{\Lambda, 5, 8\}$ it follows that $\mathtt{fn\ y\ =>\ y}^6$ is never called upon a function. ∎

The resulting analysis will have exponential worst case complexity even for the case where $k = 1$. To see this assume that the expression has size n and that it has p different variables. Then Δ has $O(n)$ elements and hence there will be $O(p \cdot n)$ different pairs (x, δ) and $O(n^2)$ different pairs (ℓ, δ). This means that $(\widehat{C}, \widehat{\rho})$ can be seen as an $O(n^2)$ tuple of values from $\widehat{\mathbf{Val}}$. Since $\widehat{\mathbf{Val}}$ itself is a powerset of pairs of the form (t, ce) and there are $O(n \cdot n^p)$ such pairs it follows that $\widehat{\mathbf{Val}}$ has height $O(n \cdot n^p)$. Since $p = O(n)$ we have the exponential worst case complexity claimed above.

This should be contrasted with the 0-CFA analysis developed in the previous sections. It corresponds to letting Δ be a singleton. Repeating the above calculations we can see $(\widehat{C}, \widehat{\rho})$ as an $O(p+n)$ tuple of values from $\widehat{\mathbf{Val}}$, and $\widehat{\mathbf{Val}}$ will be a lattice of height $O(n)$. In total this gives us a polynomial analysis as we already saw in Section 3.4.

The worst case complexity of the uniform k-CFA analysis (as well as the k-CFA analysis) can be improved in different ways. One possibility is to reduce the height of the lattice $\widehat{\mathbf{Val}}$ using the techniques of Chapter 4. Another possibility is to replace all context environments with contexts, i.e. to have $\widehat{\mathbf{Val}} = \mathcal{P}(\mathbf{Term} \times \Delta)$; clearly this will give a lattice of polynomial height. This idea is closely related to the so-called *polynomial k-CFA* analysis where the analogues of context environments are forced to be constant functions, i.e. to map all variables to the same context. In the case of polynomial 1-CFA the analysis is of complexity $O(n^6)$.

Interprocedural analysis revisited. Let us compare the above development with that of Section 2.5 where we considered *interprocedural analysis* for a simple imperative procedure language.

Recall that the abstract domain of interest in Section 2.5 has the form

$$\Delta \to L$$

where Δ is the context information and L is the complete lattice of abstract values of interest. For each label ℓ the analysis will determine two elements $A_\circ(\ell)$ and $A_\bullet(\ell)$ of $\Delta \to L$ describing the situation before and after the elementary block labelled ℓ is executed. So we have

$$A_\circ, A_\bullet : \mathbf{Lab} \to (\Delta \to L)$$

and in the terminology of the present chapter we may regard these functions as abstract caches. There is no analogue of the abstract environment in Section 2.5 – the reason is that the procedure language is so simple that it is not needed: the abstract environment records the context of the free variables and since all free variables in the procedures are global variables there is no need for this component.

We can now reformulate the above development as follows. We can take the abstract domain of interest to be

$$\Delta \to \mathcal{P}(\mathbf{Term} \times \mathbf{CEnv})$$

and reformulate the abstract cache and the abstract environment as having the functionalities:

$$\widehat{\mathsf{C}} : \mathbf{Lab} \to \Delta \to \mathcal{P}(\mathbf{Term} \times \mathbf{CEnv})$$

$$\widehat{\rho} : \mathbf{Var} \to \Delta \to \mathcal{P}(\mathbf{Term} \times \mathbf{CEnv})$$

Thus the abstract caches of the interprocedural analysis and the uniform k-CFA analysis (using call strings) have the same overall functionality and we may conclude that the two analyses are variations over a theme.

3.6.2 The Cartesian Product Algorithm

The *Cartesian Product Algorithm*, abbreviated *CPA*, has been developed for object-oriented languages but the main ideas can be expressed using a variation of our functional language in which functions take m arguments ($m > 0$):

$$t ::= \cdots \mid \mathtt{fn}\ x_1, \cdots, x_m \Rightarrow e_b \mid e_0(e_1, \cdots, e_m)$$

For notational simplicity we shall dispense with recursive functions throughout this subsection. To be faithful to the official description of CPA we shall furthermore impose the well-formedness condition that all function abstractions are closed, i.e. $FV(\mathtt{fn}\ x_1, \cdots, x_m \Rightarrow e_b) = \emptyset$, much as was the case for the procedural language considered in Section 2.5. We leave a more general treatment to Exercise 3.17.

Rephrasing the 0-CFA analysis. The first step is to adapt the specification of the 0-CFA analysis to deal with functions taking m arguments. Even though CPA is a very practical oriented algorithm it will be appropriate to consider the abstract specification in Table 3.1 (rather than the syntax directed specification in Table 3.5). We modify it as follows:

$$(\widehat{\mathsf{C}},\widehat{\rho}) \models (\mathtt{fn}\ x_1,\cdots,x_m \Rightarrow e_b)^{\ell} \text{ iff } \{\mathtt{fn}\ x_1,\cdots,x_m \Rightarrow e_b\} \subseteq \widehat{\mathsf{C}}(\ell)$$

$$\begin{array}{l}(\widehat{\mathsf{C}},\widehat{\rho}) \models (t_0^{\ell_0}(t_1^{\ell_1},\cdots,t_m^{\ell_m}))^{\ell} \\ \quad \text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models t_0^{\ell_0} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models t_1^{\ell_1} \wedge \cdots \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models t_m^{\ell_m} \wedge \\ \quad\quad \forall(\mathtt{fn}\ x_1,\cdots,x_m \Rightarrow t_b^{\ell_b}) \in \widehat{\mathsf{C}}(\ell_0): \\ \quad\quad\quad \widehat{\mathsf{C}}(\ell_1)\times\cdots\times\widehat{\mathsf{C}}(\ell_m) \subseteq \widehat{\rho}(x_1)\times\cdots\times\widehat{\rho}(x_m) \wedge \\ \quad\quad\quad (\widehat{\mathsf{C}},\widehat{\rho}) \models t_b^{\ell_b} \wedge \\ \quad\quad\quad \widehat{\mathsf{C}}(\ell_b) \subseteq \widehat{\mathsf{C}}(\ell)\end{array}$$

The third last conjunct in the clause for function application can be written

$$\widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\rho}(x_1) \wedge \cdots \wedge \widehat{\mathsf{C}}(\ell_m) \subseteq \widehat{\rho}(x_m)$$

in the case where no $\widehat{\mathsf{C}}(\ell_i)$ is empty.

The Cartesian Product Algorithm. We next extend the analysis to take context into account. This will be in the form of the actual arguments supplied to the function:

$$\delta \in \Delta = \mathbf{Term}^m = \mathbf{Term}\times\cdots\times\mathbf{Term} \qquad (m \text{ times})$$

Recalling that $\widehat{\mathbf{Val}} = \mathcal{P}(\mathbf{Term})$ we then redefine the abstract domains as follows:

$$\begin{array}{lcll}\widehat{\rho} & \in & \widehat{\mathbf{Env}} & = (\mathbf{Var}\times\Delta)\to\widehat{\mathbf{Val}} \\ \widehat{\mathsf{C}} & \in & \widehat{\mathbf{Cache}} & = (\mathbf{Lab}\times\Delta)\to\widehat{\mathbf{Val}}\end{array}$$

The key clauses in the CPA analysis then are:

$$(\widehat{\mathsf{C}},\widehat{\rho}) \models^{\delta}_{\mathsf{CPA}} x^{\ell} \text{ iff } \widehat{\rho}(x,\delta) \subseteq \widehat{\mathsf{C}}(\ell,\delta)$$

$$(\widehat{\mathsf{C}},\widehat{\rho}) \models^{\delta}_{\mathsf{CPA}} (\mathtt{fn}\ x_1,\cdots,x_m \Rightarrow e_b)^{\ell} \text{ iff } \{\mathtt{fn}\ x_1,\cdots,x_m \Rightarrow e_b\} \subseteq \widehat{\mathsf{C}}(\ell,\delta)$$

$$\begin{array}{l}(\widehat{\mathsf{C}},\widehat{\rho}) \models^{\delta}_{\mathsf{CPA}} (t_0^{\ell_0}(t_1^{\ell_1},\cdots,t_m^{\ell_m})^{\ell} \\ \quad \text{iff } (\widehat{\mathsf{C}},\widehat{\rho}) \models^{\delta}_{\mathsf{CPA}} t_0^{\ell_0} \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models^{\delta}_{\mathsf{CPA}} t_1^{\ell_1} \wedge \cdots \wedge (\widehat{\mathsf{C}},\widehat{\rho}) \models^{\delta}_{\mathsf{CPA}} t_m^{\ell_m} \wedge \\ \quad\quad \forall(\mathtt{fn}\ x_1,\cdots,x_m \Rightarrow t_b^{\ell_b}) \in \widehat{\mathsf{C}}(\ell_0,\delta) \\ \quad\quad\quad \forall\delta_b \in \widehat{\mathsf{C}}(\ell_1,\delta)\times\cdots\times\widehat{\mathsf{C}}(\ell_m,\delta): \\ \quad\quad\quad\quad \{\delta_b\} \subseteq \widehat{\rho}(x_1)\times\cdots\times\widehat{\rho}(x_m) \wedge \\ \quad\quad\quad\quad (\widehat{\mathsf{C}},\widehat{\rho}) \models^{\delta_b}_{\mathsf{CPA}} t_b^{\ell_b} \wedge \\ \quad\quad\quad\quad \widehat{\mathsf{C}}(\ell_b,\delta_b) \subseteq \widehat{\mathsf{C}}(\ell,\delta)\end{array}$$

It is clear from this specification that the body of the function is analysed separately for each possible tuple of arguments and that no merging of data takes place. To be practical we need to implement the analysis using memoisation so that the bodies are only analysed once. This can be done by organising each $(\widehat{\mathsf{C}},\widehat{\rho}) \models^{\delta_b}_{\mathsf{CPA}} t_b^{\ell_b}$ into so-called *templates* and to maintain them in a global pool; when a template is created it is only added to the pool if it is not already present.

The Cartesian Product Algorithm derives its name from the cartesian product $\widehat{\mathsf{C}}(\ell_1,\delta)\times\cdots\times\widehat{\mathsf{C}}(\ell_m,\delta)$ over which δ_b ranges. The analysis can be implemented in a straightforward *lazy* manner because the product grows monotonically: $\widehat{\mathsf{C}}(\ell_1,\delta) \times \cdots \times \widehat{\mathsf{C}}(\ell_m,\delta)$ will increase each time one of the $\widehat{\mathsf{C}}(\ell_i,\delta)$ increases (assuming that none is empty). A mild generalisation is studied in Exercise 3.17.

Interprocedural analysis revisited. Let us once more compare the development to Section 2.5 but this time to the development based on assumption sets. The development in Section 2.5 is based on

$$A_\circ, A_\bullet : \mathbf{Lab} \to (\Delta \to L)$$

where Δ is the context information and $L = \mathcal{P}(D)$ is the powerset of interest, and the current development can be reformulated as operating on

$$\widehat{\mathsf{C}} : \mathbf{Lab} \to \Delta \to \widehat{\mathbf{Val}}$$
$$\widehat{\rho} : \mathbf{Var} \to \Delta \to \widehat{\mathbf{Val}}$$

where $\widehat{\mathbf{Val}} = \mathcal{P}(\mathbf{Term})$ is the powerset of interest. Clearly D and **Term** can be taken to be equal and the fact that $\Delta = \mathbf{Term}$ then shows that $\Delta = D$ and we may conclude that "small assumption sets" and the Cartesian Product Algorithm are variations over a theme.

Concluding Remarks

Control Flow Analysis for functional languages. Many of the key ideas for Control Flow Analysis have been developed within the context of functional languages. The concept of *k-CFA* analysis seems due to Shivers [156, 157, 158]; other works on 0-CFA-like analyses include [163, 136, 57, 56]. The ideas behind k-CFA and *polynomial k-CFA* analysis were further clarified in [79] that also established the exponential complexity of k-CFA analysis for $k > 0$; it also related a form of *Set Based Analysis* [70] to 0-CFA. The *uniform k-CFA* analyses were introduced in [122] as a simplification of the k-CFA analyses; an obvious variation over this is to record the set of the last k distinct call points (see Exercise 3.15). Yet another variation over the

same theme is *Closure Analysis*; an early and often neglected development may be found in [152].

The formulation of the analyses (as well as the one presented in Table 3.1) would often seem to be more appropriate for a dynamically scoped than for a statically scoped language: The 0-CFA analysis coalesces information about variables having several defining occurrences even if they differ in their scope; clearly the analysis can easily be modified so that it more directly models static scope (see Exercise 3.7) rather than relying on no variable having more than one defining occurrence.

To the extent these developments go beyond 0-CFA they establish additional context (called mementoes, tokens or contours) for representing information concerning the *dynamic* call chain. The most common approach links back to the use of call strings in Section 2.5. The Cartesian Product Algorithm [2] was originally developed for object-oriented programs and amounts to the use of "small assumption sets" in Section 2.5.

Another way to establish context is to represent the *static* call chain. This seems first to be described by [80] as part of their so-called "polymorphic splitting" analysis. A more general set-up was formulated in [122] that also argued for the need to base abstract specifications on coinductive methods – bearing in mind that coinductive and inductive methods may coincide as in the case of syntax directed specifications.

Clearly Control Flow Analysis should be combined with Data Flow Analysis to strengthen the quality of the control information as well as providing the data flow information of interest. Our treatment in Section 3.5 only presents the first steps in this direction: a more ambitious approach is outlined in Mini Project 3.4 which is based in [126]. In fact, some authors would claim that the analysis presented in Section 3.1 should not be regarded as a 0-CFA analysis since it does not include data flow analysis (as in Section 3.5) and does not take evaluation order into account (as in Exercises 3.3 and 3.14); in our view these developments are all variations over a theme.

Most of the papers cited above directly formulate a syntax directed specification, perhaps proving it semantically sound, and perhaps showing how to generate constraints so as to obtain an efficient implementation. The use of abstract specifications first appeared in [80, 122] and has the advantage of being more directly applicable to *open systems* (that allow to interface with the library routines provided by the environment) and also to the ideas of Abstract Interpretation of Chapter 4. In particular, the notion of reachability suggests itself rather naturally [21, 60], it becomes clearer how to integrate ideas from Abstract Interpretation into Control Flow Analysis, and one does not inadvertently restrict oneself to *closed systems* only. (The notion of reachability is considered in Mini Project 3.1 which is based on [60].)

Only few papers [125] discuss the interplay between the choice of specification style for the analysis and the choice of semantics. We have used a small-step Structural Operational Semantics rather than a big-step semantics in order to express the semantic correctness also of looping programs. We have used an environment based semantics in order to ensure that we do not "modify" the bodies of functions before they are called, so that function abstractions can meaningfully be used in the value domains of our analysis [125]. As a consequence we have had to introduce intermediate expressions (closures and bindings) and have had to specify the abstract analysis also for intermediate expressions; for the syntax directed specification and the constraint based analysis this was not necessary given that semantic correctness had already been dealt with. Alternative choices are clearly possible but are likely to sacrifice at least some of the generality offered by the present approach.

Control Flow Analysis for other language paradigms. Another main application of Control Flow Analysis has been for *object-oriented languages*: one simply tracks objects rather than functions [3, 124, 137]. As a reminder of the close links between Data Flow Analysis and Control Flow Analysis we should also mention that some approaches [178, 139] are closer to the presentation of Chapter 2. A common theme among the more advanced studies is the incorporation of context (related to k-CFA) and an abstract store [133] (to deal with imperative aspects like method update). To increase the precision, local versions of the abstract store need to exist at all program points, and abstract reference counts are needed to incorporate a "kill" component (in the manner of Chapter 2). We refer to the above literature for further details of how to formulate such analyses and how to choose a proper balance between precision and cost.

Control Flow Analysis for *concurrent languages* has received relatively little attention [22]. However, variations of the techniques presented in this chapter have been used to analyse functional languages extended with concurrency primitives allowing processes to be created dynamically and to communicate via shared locations or channels [78, 57, 60, 22, 23].

In this book we do not consider *logic programming languages*. However, we should point out that Control Flow Analysis also has applications for logic programming languages and that set based analysis was first developed for this class of languages [72, 73].

Set-Constraint Based Analysis. Control Flow Analysis is just one approach to program analysis where the use of constraints pays off. In this chapter we have have taken the following approach: (i) first we have given an abstract specification of when a proposed solution is acceptable, (ii) then we have developed an algorithm for generating a set of constraints expressing that a proposed solution is acceptable, and (iii) finally we have solved the set of constraints for the least solution. For the solution of set constraints in

step (iii), it is unimportant how the constraints were in fact obtained. For this reason, it is often said that set constraints allow the separation of the specification of an analysis from its implementation and that set constraints are able to deal with forward analyses as well as backward analyses and indeed mixtures of these.

Set constraints [72, 7] have a long history [143, 84]. They allow us to express general inclusions of the form

$$S_1 \subseteq S_2$$

where set expressions, S, set variables, V, and set constructors, C, may be built as follows:

$$\begin{array}{rcl} S & ::= & V \mid \emptyset \mid S_1 \cup S_2 \mid S_1 \cap S_2 \mid C(S_1, \cdots, S_n) \\ & \mid & (S_1 \subseteq S_2) \Rightarrow S_3 \mid (S_1 \neq \emptyset) \Rightarrow S_2 \mid C^{-i}(S) \mid \neg S \mid \cdots \\ V & ::= & X \mid Y \mid \cdots \\ C & ::= & \texttt{true} \mid \texttt{false} \mid 0 \mid \cdots \mid \texttt{cons} \mid \texttt{nil} \mid \cdots \end{array}$$

Set constraints allow the consideration of constructors that are not just nullary and this allows us to record also the shape of data structures, so for example $\texttt{cons}(S_1, S_2) \cup \texttt{nil}$ expresses possibly empty lists whose heads come from S_1 and whose tails come from S_2. The associated projection selects those terms (if any) having the required shape, e.g. $\texttt{cons}^{-1}(S)$ produces the heads that may be present in S. We have seen conditional constraints before and it turns out that projection is so powerful that it can be used to code conditional constraints. Finally, it is sometimes possible to explicitly take the complement of a set but this adds to the complexity of the development. (It means that solutions can no longer be guaranteed using Tarski's Theorem and sometimes a version of Banach's Theorem may be used instead.)

The complexity of solving a system of set constraints depends rather dramatically on the set forming operations allowed and therefore many versions have been considered in the literature. We refer to [6, 135] for an overview of what is known in this area; here we just mention [28] for a general result and [70, 10] for some cubic time fragments.

However, it is worth pointing out that many of these results are worst-case results; benchmark results of Jaganathan and Wright [80] shows e.g. that in practice a 1-CFA analysis may be faster than a 0-CFA analysis despite the fact that the former has exponential worst-case complexity and the latter polynomial worst-case complexity. The reason seems to be that the 0-CFA analysis explores most of its polynomial sized state space whereas the 1-CFA analysis is so precise that it only explores a fraction of its exponentially sized state space.

The basic idea behind many of the solution procedures for set constraints is roughly as follows [9]:

1. Dynamically expand conditional constraints, based on the condition being fulfilled, until no more expansion is possible.
2. Remove all conditional constraints and combine the remaining constraints to obtain the least solution.

This is not quite the algorithm used in Section 3.4 where we were only interested in solving a rather limited class of constraints for 0-CFA analysis. When generating the constraints in Table 3.6 we were able to "guess a universe" $\mathbf{Term}_\star$ that was sufficiently large and this allowed us to generate explicit versions of the conditional constraints; in fact a superset of all those to be considered in step 1 of the above algorithm. Therefore our subsequent constraint solving algorithm in Table 3.7 merely needed to check the already existing constraints and to determine whether or not they could contribute to the solution. In practice, the above "lazy" algorithm is likely to perform much better than the "eager" algorithm of Tables 3.6 and 3.7.

Mini Projects

Mini Project 3.1 Reachability Analysis

The syntax directed analysis of Table 3.5 analyses each subexpression of $e_\star$ "exactly once" rather than "at most once" as really called for. In this mini project we shall study one way to amend this.

The idea is to introduce an *abstract reachability component*

$$\widehat{\mathsf{R}} \in \mathbf{Reach} = \mathbf{Lab} \to \mathcal{P}(\{\mathbf{on}\})$$

and to modify the syntax directed analysis to have a relation of the form

$$(\widehat{\mathsf{C}}, \widehat{\rho}, \widehat{\mathsf{R}}) \models'_s e$$

The idea is that $\mathtt{fn}\ x \Rightarrow t_0^{\ell_0}$ has $\{\mathbf{on}\} \subseteq \widehat{\mathsf{R}}(\ell_0)$ if and only if the function is indeed applied somewhere, and that the "recursive call" $(\widehat{\mathsf{C}}, \widehat{\rho}, \widehat{\mathsf{R}}) \models_s t_0^{\ell_0}$ is performed if and only if $\{\mathbf{on}\} \subseteq \widehat{\mathsf{R}}(\ell_0)$.

1. Modify Table 3.5 to incorporate this idea.
2. Show the following analogue of Proposition 3.18: If $(\widehat{\mathsf{C}}, \widehat{\rho}, \widehat{\mathsf{R}}) \models'_s t_\star^{\ell_\star}$, $\{\mathbf{on}\} \subseteq \widehat{\mathsf{R}}(\ell_\star)$ and $(\widehat{\mathsf{C}}, \widehat{\rho}) \sqsubseteq (\widehat{\mathsf{C}}_\star^\top, \widehat{\rho}_\star^\top)$ then $(\widehat{\mathsf{C}}, \widehat{\rho}) \models t_\star^{\ell_\star}$.
3. Determine whether or not the statements

   if $(\widehat{\mathsf{C}}, \widehat{\rho}) \models e_\star$ then $(\widehat{\mathsf{C}}, \widehat{\rho}, \widehat{\mathsf{R}}) \models'_s e_\star$ for some $\widehat{\mathsf{R}}$

   if $(\widehat{\mathsf{C}}, \widehat{\rho}) \models e_\star$ and $(\widehat{\mathsf{C}}, \widehat{\rho}) \sqsubseteq (\widehat{\mathsf{C}}_\star^\top, \widehat{\rho}_\star^\top)$ then $(\widehat{\mathsf{C}}, \widehat{\rho}, \widehat{\mathsf{R}}) \models'_s e_\star$ for some $\widehat{\mathsf{R}}$

   hold in general. ■

Mini Project 3.2 Data Structures

The language considered so far only includes simple data like integers and booleans. In this mini project we shall extend the language with more general data structures:

$$e ::= \cdots \mid C(e_1, \cdots, e_n)^\ell \mid (\texttt{case } e_0 \texttt{ of } C(x_1, \cdots, x_n) \texttt{ => } e_1 \texttt{ or } x \texttt{ => } e_2)^\ell$$

Here $C \in \mathbf{Constr}$ denotes an n-ary data constructor. A data element is constructed by $C(e_1, \cdots, e_n)$: it has tag C and its components are the values of $e_1, \cdots, e_n$. The case construct will first determine the value v_0 of e_0, if v_0 has the tag C then $x_1, \cdots, x_n$ will be bound to the components of v_0 and e_1 is evaluated. If v_0 does not have tag C then x is bound to v_0 and e_2 is evaluated.

As an example we may have $\mathbf{Constr} = \{\texttt{cons}, \texttt{nil}\}$ so we have the following expression (omitting labels) for reversing a list:

```
let append = fun app xs => fn ys =>
             case xs of cons(z,zs) => cons(z,app zs ys)
                     or xs => ys
in fun rev xs => case xs of cons(y,ys) =>
                            append (rev ys) (cons(y,nil()))
                         or xs => nil()
```

To specify a 0-CFA analysis for this language we shall take

$$\widehat{\mathbf{Val}} = \mathcal{P}(\mathbf{Term} \cup \{C(\ell_1, \cdots, \ell_n) \mid C \in \mathbf{Constr}, \ell_1, \cdots, \ell_n \in \mathbf{Lab}\})$$

As before the terms of interest are $\texttt{fn } x \texttt{ => } e_0$ and $\texttt{fun } f\ x \texttt{ => } e_0$ for recording the abstractions. The new contribution is a number of elements of the form $C(\ell_1, \cdots, \ell_n)$ denoting a data element $C(v_1, \cdots, v_n)$ whose i'th component might have been created by the expression at program point ℓ_i.

1. Develop a syntax directed analogue of the analysis in Table 3.5.

2. Modify the constraint generation algorithm of Table 3.6 to handle the new constructs and make the necessary changes to the constraint solving algorithm of Table 3.7.

For the more ambitious: are there any difficulties in developing an abstract analogue of the analysis in Table 3.1? ■

Mini Project 3.3 A Prototype Implementation

In this mini project we shall implement the pure 0-CFA analysis considered in Section 3.3. As implementation language we shall choose a functional language such as Standard ML or Haskell. We can then define a suitable data type for FUN expressions as follows:

```
type var        = string
type label      = int
datatype const  = Num of int | True | False
datatype exp    = Label of term * label
and term        = Const of const | Var of var
                | Fn of var * exp | Fun of var * var * exp
                | App of exp * exp | If of exp * exp * exp
                | Let of var * exp * exp | Op of string * exp * exp
```

Now proceed as follows:

1. Implement the constraint based control flow analysis of Section 3.4; this includes defining an appropriate data structure *constraints* for (conditional) constraints.

2. Implement the graph based algorithm of Section 3.4 for solving constraints; this involves choosing appropriate data structures for the worklist and the two arrays used by the algorithm.

For the more ambitious: generalise your program to perform some of the more advanced analyses, e.g. by incorporating data flow information or context information. ■

Mini Project 3.4 Imperative Constructs

In Section 3.5 we showed how to incorporate Data Flow Analysis into our Control Flow Analysis; this becomes more challenging when imperative constructs are added to the language:

$$e ::= \cdots \mid (\mathtt{new}_\pi\ x\ \mathtt{:=}\ e_1\ \mathtt{in}\ e_2)^\ell \mid (\mathtt{!}\ x)^\ell \mid (x\ \mathtt{:=}\ e_0)^\ell \mid (e_1\ \mathtt{;}\ e_2)^\ell$$

Here $\mathtt{new}_\pi\ x\ \mathtt{:=}\ e_1\ \mathtt{in}\ e_2$ creates a new reference variable x to be used in e_2; it is initialised to e_1, its content is obtained by $\mathtt{!}\ x$, and it may be updated by $x\ \mathtt{:=}\ e_0$. The creation point for the reference variable is indicated by the program point $\pi \in \mathbf{Pnt}$ and the construct $e_1\ \mathtt{;}\ e_2$ merely sequences e_1 and e_2 (and is equivalent to $\mathtt{let}\ x\ \mathtt{=}\ e_1\ \mathtt{in}\ e_2$ when x does not occur in e_2).

One approach to extending the development of Section 3.5 might be to work with an acceptability relation of the form

$$(\widehat{\mathsf{C}}, \widehat{\rho}, \widehat{\mathsf{S}}_\circ, \widehat{\mathsf{S}}_\bullet) \models_{me} e$$

where

- $\widehat{\mathsf{C}} : \mathbf{Lab} \to \widehat{\mathbf{Val}}$ and $\widehat{\mathsf{C}}(\ell)$ describes the values that the subexpression labelled ℓ may evaluate to,
- $\widehat{\rho} : \mathbf{Var} \to \widehat{\mathbf{Val}}$ and $\widehat{\rho}(x)$ describes the values that x might be bound to,
- $\widehat{\mathsf{S}}_\circ : \mathbf{Lab} \to (\mathbf{Pnt} \to \widehat{\mathbf{Val}})$ and $\widehat{\mathsf{S}}_\circ(\ell)$ describes the states that may be possible *before* the subexpression labelled ℓ is evaluated,
- $\widehat{\mathsf{S}}_\bullet : \mathbf{Lab} \to (\mathbf{Pnt} \to \widehat{\mathbf{Val}})$ and $\widehat{\mathsf{S}}_\bullet(\ell)$ describes the states that may be possible *after* the subexpression labelled ℓ is evaluated, and
- $me : \mathbf{Var} \to \mathbf{Pnt}$ and $me(x)$ indicates the creation point for the reference variable x.

One choice of $\widehat{\mathbf{Val}}$ is $\mathcal{P}(\mathbf{Term}) \times L$ where the first component tracks functions and the second component tracks abstract values. It may be helpful to devise a Structural Operational Semantics for the extended language and to use it as a guide when defining the acceptability relation.

A more advanced treatment would involve context information in the manner of Section 3.6. ■

Exercises

Exercise 3.1 Consider the following expression (omitting labels):

```
let f = fn x => x 1
in  let g = fn y => y+2
    in  let h = fn z => z+3
        in  (f g) + (f h)
```

Add labels to the program and guess an analysis result. Use Table 3.1 to verify that it is indeed an acceptable guess. ■

Exercise 3.2 The specification of the Control Flow Analysis in Table 3.1 uses potentially infinite value spaces and this is not really necessary. To see this choose some expression $e_\star \in \mathbf{Exp}$ that is to be analysed. Let $\mathbf{Var}_\star \subseteq \mathbf{Var}$ be the finite set of variables occurring in $e_\star$, let $\mathbf{Lab}_\star \subseteq \mathbf{Lab}$ be the finite set of labels occurring in $e_\star$, and let $\mathbf{Term}_\star$ be the finite set of subterms of $e_\star$. Next define

$$\begin{array}{rcll} \widehat{v} & \in & \widehat{\mathbf{Val}}_\star & = \mathcal{P}(\mathbf{Term}_\star) \\ \widehat{\rho} & \in & \widehat{\mathbf{Env}}_\star & = \mathbf{Var}_\star \to \widehat{\mathbf{Val}}_\star \\ \widehat{\mathsf{C}} & \in & \widehat{\mathbf{Cache}}_\star & = \mathbf{Lab}_\star \to \widehat{\mathbf{Val}}_\star \end{array}$$

and note that these value spaces are finite. Show that the specification of the analysis in Table 3.1 still makes sense when $(\widehat{\mathsf{C}}, \widehat{\rho})$ is restricted to be in $\widehat{\mathbf{Cache}}_\star \times \widehat{\mathbf{Env}}_\star$. ∎

Exercise 3.3 Modify the Control Flow Analysis of Table 3.1 to take account of the left to right evaluation order imposed by a call-by-value semantics: in the clause [*app*] there is no need to analyse the operand if the operator cannot produced any closures. Try to find a program where the modified analysis accepts analysis results $(\widehat{\mathsf{C}}, \widehat{\rho})$ rejected by Table 3.1. ∎

Exercise 3.4 So far we have defined "$(\widehat{\mathsf{C}}, \widehat{\rho})$ is an acceptable solution for e" to mean that

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models e \tag{3.8}$$

but an alternative condition is that

$$\exists (\widehat{\mathsf{C}}', \widehat{\rho}') : (\widehat{\mathsf{C}}', \widehat{\rho}') \models e \ \wedge \ (\widehat{\mathsf{C}}', \widehat{\rho}') \sqsubseteq (\widehat{\mathsf{C}}, \widehat{\rho}) \tag{3.9}$$

Show that (3.8) implies (3.9) but not vice versa. Discuss which of (3.8) or (3.9) is the preferable definition. ∎

Exercise 3.5 Consider an alternative specification of the analysis in Table 3.1 where the condition

$$(\mathtt{fun}\ f\ x \Rightarrow t_0^{\ell_0}) \in \widehat{\rho}(f)$$

in [*app*] is replaced by

$$\widehat{\mathsf{C}}(\ell_1) \subseteq \widehat{\rho}(f)$$

also in [*app*]. Show that the proof of Theorem 3.10 can be modified accordingly. Discuss the relative precision of the two analyses. ∎

Exercise 3.6 Reconsider our decision to use $\widehat{\mathbf{Val}} = \mathcal{P}(\mathbf{Term})$ and consider using $\widehat{\mathbf{Val}} = \mathcal{P}(\mathbf{Exp})$ instead. Show that the specification of the Control Flow Analysis may be modified accordingly but that then Fact 3.11 (and hence the correctness result) would fail. ∎

Exercise 3.7 The operational semantics allow us to rename bound variables without changing the semantics; this is in accord with the language being statically scoped (or lexically scoped) rather than dynamically scoped. As an example

$$((\texttt{fn x => x}^1)^2\ (\texttt{fn y => y}^3)^4)^5 =_\alpha ((\texttt{fn x => x}^1)^2\ (\texttt{fn x => x}^3)^4)^5$$

and clearly the two programs have the same semantics.

However, renaming bound variables changes the acceptability of a solution as well as influences the precision of the analysis specified in Table 3.1. Develop an abstract specification of a 0-CFA analysis that is more faithful to the static scoping than that of Table 3.1; it should agree with the specification of Table 3.1 for expressions that do not have multiple defining occurrences. ∎

Exercise 3.8 In Section 3.2 we equipped FUN with a call-by-value semantics. An alternative would be to use a *call-by-name* or *lazy* semantics. It can be obtained as a simple modification of the semantics of Tables 3.2 and 3.3 by allowing the environments $\rho \in \mathbf{Env}$ to map variables to intermediate terms (and not just values), by deleting the rules $[app_2]$ and $[let_1]$ and then make some obvious modifications to the axioms $[var]$, $[app_{fn}]$, $[app_{fun}]$ and $[let_2]$; in the case of $[var]$ we will take:

$$\rho \vdash x^\ell \rightarrow it^\ell \quad \text{if } x \in dom(\rho) \text{ and } it = \rho(x)$$

Complete the specification of the semantics and show that the correctness result (Theorem 3.10) still holds for the analysis of Table 3.1.

What does that tell us about the precision of the analysis? ∎

Exercise 3.9 Let $\models'_s$ and $\models''_s$ be two relations satisfying the specification of Table 3.5. Show that

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models'_s e \text{ iff } (\widehat{\mathsf{C}}, \widehat{\rho}) \models''_s e$$

by structural induction on e. ∎

Exercise 3.10 Consider Proposition 3.18 and determine whether or not the statement

$$\text{if } (\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star \text{ then } (\widehat{\mathsf{C}}, \widehat{\rho}) \models e_\star$$

holds in general. ∎

Exercise 3.11 Give an example showing that both of the statements

$$\text{if } (\widehat{\mathsf{C}}, \widehat{\rho}) \models e_\star \text{ then } (\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star$$

$$\text{if } (\widehat{\mathsf{C}}, \widehat{\rho}) \models e_\star \text{ and } (\widehat{\mathsf{C}}, \widehat{\rho}) \sqsubseteq (\widehat{\mathsf{C}}^\top_\star, \widehat{\rho}^\top_\star) \text{ then } (\widehat{\mathsf{C}}, \widehat{\rho}) \models_s e_\star$$

fail in general. ∎

Exercise 3.12 Give a direct proof of the correctness of the syntax directed analysis of Table 3.5, i.e. establish an analogue of Theorem 3.10. This involves first extending the syntax directed analysis to the `bind`- and `close`-constructs and next proving that if $\rho \mathcal{R} \widehat{\rho}$, $\rho \vdash ie \rightarrow ie'$ and $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_s ie$ then also $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_s ie'$. ∎

Exercise 3.13 Consider the system $\mathcal{C}^{=}_{\star}[\![e_\star]\!]$ that contains a constraint

$$ls_1 \cup \cdots \cup ls_n = rhs$$

whenever $\mathcal{C}_\star[\![e_\star]\!]$ contains the $n \geq 1$ constraints

$$ls_i \subseteq rhs$$

Show that

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}^{=}_{\star}[\![e_\star]\!] \text{ implies } (\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}_\star[\![e_\star]\!]$$

where $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c (ls = rhs)$ is defined in the obvious way. Also show that

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}_\star[\![e_\star]\!] \text{ implies } (\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}^{=}_{\star}[\![e_\star]\!]$$

holds in the special case where $(\widehat{\mathsf{C}}, \widehat{\rho})$ is *least* such that $(\widehat{\mathsf{C}}, \widehat{\rho}) \models_c \mathcal{C}_\star[\![e_\star]\!]$. ∎

Exercise 3.14 Use the ideas of Exercise 3.3 to develop an improvement of Table 3.8 where expressions are only analysed when absolutely needed. Next develop a syntax directed analysis using the same ideas. Discuss the relationship between the two specifications: are they more closely related than is the case for $\models$ and $\models_s$ of Table 3.1 and 3.5 (see Exercises 3.10 and 3.11)? ∎

Exercise 3.15 Modify the abstract specification of the uniform k-CFA analysis so that it does not record the last k function calls but the last k times we called a different function than in the preceding call: if the calling sequence is [1,2,2,1,1] then 2-CFA records [1,1] but the modified analysis records [2,1]. Discuss which of the two analyses (say for $k = 2$) is likely to be most useful in practice. ∎

Exercise 3.16 Let us consider a language of first-order recursion equation schemes: the programs have the form

$$\texttt{define } D_\star \texttt{ in } e_\star$$

where $D_\star$ is a sequence of function definitions of the form:

$$f(x) = e$$

Here f is a function name, x is the formal parameter and e is the body of the function; the functions defined in D may be mutually recursive and the parameter mechanism is call-by-value. The expressions are given by

$$\begin{array}{lcl} e & ::= & t^{\ell} \\ t & ::= & c \mid x \mid f\ e \mid \texttt{if}\ e_0\ \texttt{then}\ e_1\ \texttt{else}\ e_2 \mid e_1\ op\ e_2 \end{array}$$

where $c \in \mathbf{Const}$ and $op \in \mathbf{Op}$ as before; we shall assume that f and x belong to distinct syntactic categories. As an example we may define the Fibonacci function by the following expression (omitting labels):

```
define fib(z) = if z<3 then 0
                else fib (z-1) + fib (z-2)
in      fib x
```

Define a uniform k-CFA analysis for this language. For $k = 0$ and $k = 1$ compare the development with that for the procedure language in Section 2.5. ■

Exercise 3.17 In Subsection 3.6.2 we covered the Cartesian Production Algorithm based on the assumption that all function abstractions are closed. In this exercise we do not make this simplifying assumption and also we wish to deal with recursive functions. Develop an abstract specification of an analysis

$$(\widehat{\mathsf{C}}, \widehat{\rho}) \models^{ce}_{\delta} e$$

where $\delta \in \Delta$ is as in Subsection 3.6.2 and

$$ce \in \mathbf{CEnv} = \mathbf{Var} \to \Delta$$

Hint: Modify Table 3.10 to deal with functions taking m arguments and to perform the appropriate determination of new contexts in the clause for function application. ■